

西安交通大学

硕士学位论文

周期级多核神经网络芯片模拟器

学位申请人：彭森

指导教师：姚期智教授

合作导师：马恺声教授

类别（领域）：集成电路工程

2026 年 05 月

Design and Implementation of an End-to-End Cycle-Level Simulator for Multi-core NPUs

A thesis submitted to
Xi'an Jiaotong University
in partial fulfillment of the requirements
for the degree of
Master of Engineering

By

Peng Sen

Supervisor: (Associate) Prof. Andrew Chi-Chih Yao

Associate Supervisor: Prof. Kai-Sheng Ma

Integrated Circuit Engineering

May 2026

硕士学位论文答辩委员会

周期级多核神经网络芯片模拟器

答辩人：彭森

答辩委员会委员：

XXXXXXXXXXXX 大学 XXX: XXX (注：主席)

XXXXXXXXXXXX 大学 XXX: XXX

XXXXXXXXXXXX 大学 XXX: XXX

XXXXXXXXXXXX 大学 XXX: XXX

XXXXXXXXXXXX 大学 XXX: XXX

答辩时间：2026 年 5 月 20 日

答辩地点：西安交通大学兴庆校区教二南楼 206 会议室

摘 要

随着深度神经网络与大语言模型在云端与边缘端的广泛部署，人工智能对硬件算力的需求飞速增长。为了提升计算能力，神经网络芯片作为领域专用架构芯片被提出，并从单核向多核架构演进。随着芯片规模的扩大，片上网络拥塞、主存带宽不足与跨核调度复杂度高等系统级问题日益突出。在流片成本高、迭代周期长的背景下，能够在设计阶段给出定量反馈的周期级模拟器，对架构评估与软硬件协同优化具有重要支撑作用。

本文设计并实现了以描述神经网络负载的中间表示为输入的端到端周期级模拟器。通过对神经网络芯片的通用建模和统一任务抽象，将多核执行刻画为“核心级工作负载—显式依赖—数据流”的可解析模型，在统一时间轴下驱动计算、片上互连与片外存储协同推进。模拟过程可记录各核加载、计算、写回及网络背压等阶段周期，输出可复现的瓶颈归因统计，形成“解析—执行—统计”的闭环。

在验证与应用方面，本文构造一系列微基准实验，检验核间数据依赖传播、片上网络和片外存储请求正常响应，以及不同子系统后端差异。与 Synopsys ZeBu 硬件模拟平台在相同调度结果和硬件配置下对 ResNet34/ResNet50 等负载的模拟结果相比，本模拟器的相对误差控制在 $\pm 5.2\%$ 以内，验证了模拟器的准确度。在此基础上，本文对多核神经网络芯片开展了系统性的瓶颈分析与设计空间探索。在四核芯片运行 ResNet50 场景下，本文通过片外存储升级，再到提升 SRAM 带宽的递进式优化，将端到端总周期降低 73.7% (3.80 倍加速)，揭示了“片外带宽瓶颈 $\rightarrow$ 片上端口瓶颈”的逐级迁移规律。本文进一步沿单核算力、核数扩展与批大小三个维度展开 12 组设计空间探索，定量发现：(1) 存储带宽低于计算强度要求下提升算力对吞吐量无贡献；(2) 核数超过负载并行度上限后性能严重退化甚至死锁；(3) 批大小扩展效率仅约 24%，受限于存储带宽争用。

本项目为多核神经网络芯片架构评估与软硬协同优化提供了可复现的周期级模拟基础，通过从精度验证、瓶颈归因到多维设计空间探索的完整实验链，体现了模拟器在正确性校验、瓶颈定位与架构参数优化中的实用价值，可为软硬件协同优化提供可复现的评估能力。

关 键 词：神经网络芯片；多核架构；周期级模拟器；软硬件协同优化

论文类型：应用研究

ABSTRACT

With the widespread deployment of deep neural networks and large language models in the cloud and at the edge, demand for hardware compute is growing rapidly. To increase compute capability, neural network chips are proposed as domain-specific architectures and are evolving from single-core to multi-core designs. As chip scale grows, system-level issues such as on-chip network congestion, insufficient main-memory bandwidth, and high cross-core scheduling complexity are becoming increasingly salient. Against a backdrop of high tape-out cost and long iteration cycles, a cycle-level simulator that can provide quantitative feedback during the design phase plays an important supporting role in architecture evaluation and software–hardware co-optimization.

This thesis designs and implements an end-to-end cycle-level simulator that takes as input an intermediate representation describing neural network workloads. Through general-purpose modeling of neural network chips and a unified task abstraction, multi-core execution is cast as a parsable model of “core-level workload—explicit dependency—data flow”, and compute, on-chip interconnect, and off-chip memory are advanced in a coordinated manner on a unified timeline. During simulation, per-core cycles in loading, computing, writeback, and network backpressure can be recorded to produce reproducible bottleneck attribution statistics, forming a closed loop of “parse—execute—statistics”.

For validation and application, this work constructs a series of micro-benchmark experiments to verify inter-core data-dependency propagation, correct request–response behavior for the on-chip network and off-chip memory, and differences among subsystem backends. Compared with results from the Synopsys ZeBu hardware emulation platform under the same scheduling outcome and hardware configuration for workloads including ResNet34 and ResNet50, the relative error of this simulator is kept within $\pm 5.2\%$, validating its accuracy. On this basis, this thesis carries out systematic bottleneck analysis and design-space exploration for multi-core neural network chips. In a four-core chip running ResNet50, progressive optimization—first upgrading off-chip memory, then increasing SRAM bandwidth—reduces end-to-end total cycles by 73.7% (a $3.80\times$ speedup) and reveals a stepwise bottleneck migration from off-chip bandwidth to on-chip port bandwidth. A further design-space exploration along three dimensions—single-core compute capability, core-count scaling, and batch size—with 12 configurations quantitatively shows that: (1) increasing compute capability yields no contribu-

tion to throughput when memory bandwidth is below what compute intensity demands; (2) scaling core count beyond the workload's parallelism limit severely degrades performance or even causes deadlock; and (3) batch-scaling efficiency is only about 24%, limited by memory-bandwidth contention.

This project provides a reproducible cycle-level simulation basis for evaluating multi-core neural network chip architectures and for software–hardware co-optimization. Through a complete experimental chain spanning accuracy validation, bottleneck attribution, and multi-dimensional design-space exploration, the simulator demonstrates practical value in correctness verification, bottleneck localization, and architectural parameter tuning, and can offer reproducible evaluation capability for software–hardware co-optimization.

KEY WORDS: Neural network chip; Multi-core architecture; Cycle-level simulator;
Software–hardware co-optimization

TYPE OF THESIS: Applied Research

目 录

摘 要	I
ABSTRACT	III
缩写与符号说明	XI
1 绪论	1
1.1 研究背景与意义	1
1.1.1 人工智能发展与神经网络芯片算力演进	1
1.1.2 多核神经网络芯片架构设计挑战	2
1.2 研究现状与不足	6
1.2.1 神经网络芯片模拟器	7
1.2.2 任务调度与编译优化	9
1.2.3 互连与存储优化	10
1.2.4 软硬件协同设计与评估	11
1.3 本文研究内容与章节安排	13
1.3.1 主要研究内容	13
1.3.2 本研究的难点、贡献与创新	14
1.3.3 论文组织结构	15
2 软硬件基础理论与前置技术	16
2.1 多核神经网络芯片体系结构基础	16
2.1.1 脉动阵列与数据流	17
2.1.2 片上 SRAM 与显式搬运	18
2.2 深度学习工作负载的计算与访存特征	19
2.2.1 人工智能与大模型发展对神经网络芯片的需求背景	19
2.2.2 算子与模型形态的多样性	20
2.2.3 大模型推理与注意力、KV Cache 的访存特征	20
2.3 周期级模拟方法与系统级建模	21
2.4 硬件模拟与准确性验证参考：ZeBu	21
2.5 关键工具底座：BookSim2 与 Ramulator2	21
2.5.1 BookSim2：周期级 NoC 模拟	22
2.5.2 Ramulator2：DRAM 时序与调度	22
2.6 多核神经网络芯片编译与调度及调度器 IR	22
2.6.1 编译与调度的挑战与相关工作	22
2.6.2 调度器如何生成 IR 及 IR 的形式与语义	23
2.6.3 调度器 IR 与本文模拟器的衔接	24

3 模拟器整体架构与松耦合接口设计	26
3.1 总体架构概述	26
3.2 前端 IR 解析模块设计：JSON 到统一任务表示	30
3.3 面向对象的松耦合接口设计	32
3.3.1 统一报文类型与接口约定	34
3.3.2 网络接口：简单模式与 BookSim2 封装	34
3.3.3 存储接口：简单模式与 Ramulator2 封装	35
3.3.4 接口化设计的意义	36
4 可配置计算核心（Core）与全系统同步机制	38
4.1 高通用性计算核心建模	38
4.1.1 建模目标与抽象边界	38
4.1.2 状态机设计	39
4.1.3 算子计算时间估算	39
4.1.4 本地 SRAM 建模与生命周期	41
4.1.5 统计输出与可观测性	42
4.2 NoC 与 DRAM 子系统的集成机制	43
4.2.1 多核并发与资源争用	44
4.3 全局事件驱动与跨时钟域同步	46
4.3.1 跨时钟域模拟面临的问题	46
4.3.2 本文的设计：全局周期与多速率推进	46
4.3.3 时序示意与效果	47
4.4 模拟器支撑的研究与探索视角	48
5 模拟器验证与瓶颈及热点分析实验	50
5.1 模拟器基本功能与精度验证（Micro-benchmarks）	50
5.1.1 MB-1：双核点对点依赖	50
5.1.2 MB-2：仅 DRAM 依赖（无核间通信）	52
5.1.3 与 ZeBu 硬件模拟的对比（单核）	53
5.2 真实 AI 负载瓶颈分析（调度器生成 IR）	55
5.2.1 端到端阶段占比分解口径	55
5.2.2 热点核与阶段占比可视化	56
5.3 多核瓶颈分析与优化实验	57
5.3.1 实验设计	57
5.3.2 基线仿真结果（DDR4 单通道）	58
5.3.3 优化仿真结果（HBM 4 通道）	59
5.3.4 进一步优化：提升 SRAM 端口带宽	60
5.3.5 三组配置对比分析	62
5.4 多核吞吐量的设计空间探索	62
5.4.1 单核算力扩展（MAC 规模扫描）	63

5.4.2 核数扩展	63
5.4.3 Batch 扩展效率.....	65
5.4.4 综合结论与设计启示	66
6 结论与展望	67
6.1 结论	67
6.2 展望	68
6.3 工作总结	69
6.4 主要贡献	70
致谢	72
参考文献	73
附录	76
攻读学位期间取得的科研成果	85
答辩委员会会议决议	87
常规评阅人名单	88
声明	

CONTENTS

ABSTRACT (Chinese)	I
ABSTRACT (English)	III
Abbreviations and Notation	XI
1 Introduction	1
1.1 Background and Motivation	1
1.1.1 AI Progress and NPU Performance Trends	1
1.1.2 Challenges in Multi-core Neural Network Chip Design	2
1.2 Related Work and Gaps	6
1.2.1 Neural Network Chip Simulators	7
1.2.2 Scheduling and Compilation	9
1.2.3 Interconnect and Memory Optimization	10
1.2.4 Hardware-Software Co-design and Evaluation	11
1.3 Contributions and Organization	13
1.3.1 Main Contributions	13
1.3.2 Difficulties, Contributions and Innovations	14
1.3.3 Thesis Organization	15
2 Preliminaries	16
2.1 Multi-core Neural Network Chip Architecture	16
2.1.1 Systolic Array and Dataflow	17
2.1.2 Scratchpad SRAM and Explicit Data Movement	18
2.2 Workload Characteristics	19
2.2.1 AI and Large-Model Trends and Demand for Neural Network Chips	19
2.2.2 Diversity of Operators and Model Architectures	20
2.2.3 LLM Inference and Memory Access Patterns of Attention and KV Cache	20
2.3 Cycle-level Simulation	21
2.4 Hardware Emulation and ZeBu as Validation Reference	21
2.5 BookSim2 and Ramulator2	21
2.5.1 BookSim2 for NoC	22
2.5.2 Ramulator2 for DRAM	22
2.6 Compilation, Scheduling, and Scheduler IR	22
2.6.1 Challenges and Related Work	22
2.6.2 How the Scheduler Produces IR and IR Specification	23
2.6.3 Scheduler IR and Simulator Input	24

3	Overall Architecture and Loosely-coupled Interfaces	26
3.1	Architecture Overview	26
3.2	Front-end IR Parsing: JSON to Unified Tasks	30
3.3	Loosely-coupled Interfaces	32
3.3.1	Unified Packet and Interface Contract	34
3.3.2	NetworkInterface: Simple and BookSim2	34
3.3.3	MemoryInterface: Simple and Ramulator2	35
3.3.4	Why Loose Coupling Matters	36
4	Configurable Core and System Synchronization	38
4.1	General-purpose Core Modeling	38
4.1.1	Modeling Goals	38
4.1.2	State Machine	39
4.1.3	Operator Execution Time Estimation	39
4.1.4	Local SRAM Modeling	41
4.1.5	Observables and Statistical Output	42
4.2	Integration of NoC and DRAM	43
4.2.1	Multi-core Concurrency and Resource Contention	44
4.3	Event-driven Scheduling and Cross-clock-domain Synchronization	46
4.3.1	Challenges in Cross-clock-domain Simulation	46
4.3.2	Design: Global Cycle and Multi-rate Ticking	46
4.3.3	Timeline Illustration and Effects	47
4.4	Research and Exploration Enabled by the Simulator	48
5	Simulator Validation, Bottleneck and Hotspot Analysis Experiments	50
5.1	Micro-benchmark Validation	50
5.1.1	MB-1: Two-core P2P Dependency	50
5.1.2	MB-2: DRAM-only Dependencies	52
5.1.3	Comparison with ZeBu Hardware Simulation (Single-core)	53
5.2	Bottleneck Analysis on Scheduler-generated IR	55
5.2.1	Breakdown Methodology	55
5.2.2	Hotspot Cores and Phase Fraction Visualization	56
5.3	Multi-core Bottleneck Analysis and Optimization	57
5.3.1	Experiment Design	57
5.3.2	Baseline Results (DDR4 Single Channel)	58
5.3.3	Optimized Results (HBM 4 Channels)	59
5.3.4	Further Optimization: Wider SRAM Port	60
5.3.5	Three-way Comparative Analysis	62
5.4	Design-Space Exploration for Multi-core Throughput	62
5.4.1	Single-core Compute Scaling	63

5.4.2 Core Count Scaling	63
5.4.3 Batch Scaling Efficiency	65
5.4.4 Summary and Design Implications	66
6 Conclusion and Future Work	67
6.1 Conclusions	67
6.2 Future Work	68
6.3 Summary	69
6.4 Contributions	70
Acknowledgements	72
References	73
Appendix	76
Achievements	85
Decision of Defense Committee	87
General Reviewers List	88
Declarations	

缩写与符号说明

ASIC	Application-Specific Integrated Circuit, 应用专用集成电路
CNN	Convolutional Neural Network, 卷积神经网络
DMA	Direct Memory Access, 直接内存访问
DNN	Deep Neural Network, 深度神经网络
DRAM	Dynamic Random Access Memory, 动态随机存取存储器
DSE	Design Space Exploration, 设计空间探索
Flit	Flow control digit, 流控单元; NoC 中最小传输单位
FPGA	Field-Programmable Gate Array, 现场可编程门阵列
FR-FCFS	First-Ready First-Come-First-Served, 先就绪先服务 (内存调度策略)
GEMM	General Matrix Multiply, 通用矩阵乘
ifmap	input feature map, 输入特征图
IR	Intermediate Representation, 中间表示
JSON	JavaScript Object Notation, 一种轻量级数据交换格式
KV Cache	Key-Value Cache, 键值缓存 (自注意力中的缓存)
LLM	Large Language Model, 大语言模型
MAC	Multiply-Accumulate, 乘加运算
MLIR	Multi-Level Intermediate Representation, 多层中间表示
NoC	Network-on-Chip, 片上网络
NPU	Neural Processing Unit, 神经网络处理器
ofmap	output feature map, 输出特征图
PE	Processing Element, 处理单元
RTL	Register Transfer Level, 寄存器传输级
SRAM	Static Random Access Memory, 静态随机存取存储器
TPU	Tensor Processing Unit, 张量处理单元
VC	Virtual Channel, 虚拟通道

1 绪论

1.1 研究背景与意义

近年来，以深度学习为代表的人工智能技术快速发展，尤其是以 Transformer 为核心架构的大语言模型推动了云端与端侧推理需求的爆发式增长^[1-2]。在算力需求持续攀升、制程红利逐步放缓的背景下，神经网络处理器（Neural Network Processor Unit, NPU）等领域专用架构（Domain Specific Architecture, DSA）芯片成为提升性能与能效的重要路径^[3]。多核神经网络芯片在扩展峰值算力的同时，其系统级行为受存储带宽、片上互连与编译调度策略复杂度的联合约束，在设计阶段就需对架构参数与映射策略进行联合评估与探索。

1.1.1 人工智能发展与神经网络芯片算力演进

人工智能的发展呈现出模型规模与任务复杂度持续上升的态势。从早期卷积网络到 Transformer 与大语言模型，参数量与训练算力需求呈指数级增长。为应对这一趋势，产业界持续推出新一代神经网络芯片与其他 DSA 芯片，其峰值算力与能效在代际迭代中不断提升，与上层模型创新形成相互牵引。图1-1 给出不同年份的典型神经网络芯片算力增长趋势。图1-2 给出深度神经网络所需训练计算量随发布年份的指数级增长趋势，直观反映大模型发展对算力需求的持续攀升^[4]。模型—硬件与训练算力两条线索共同说明：AI 工作负载的演进不断抬高对专用算力的需求，而神经网络芯片则通过工艺、架构与多核横向扩展持续提高芯片算力。

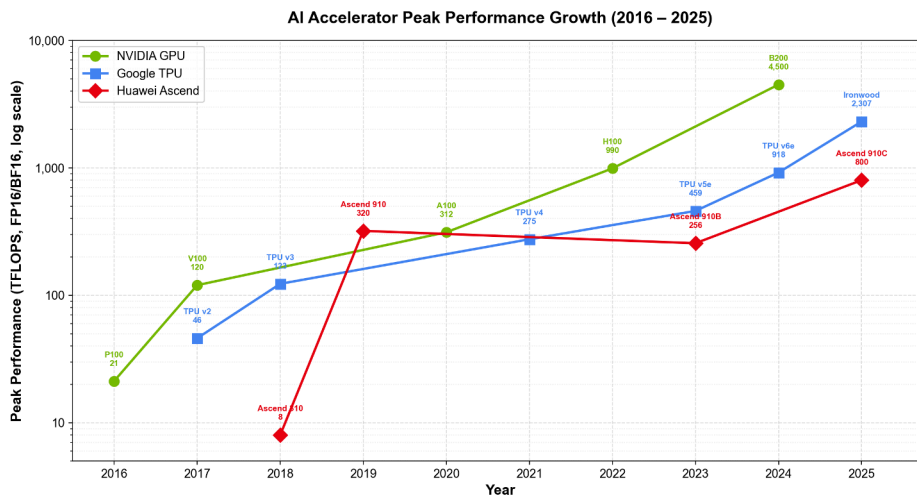


图 1-1 典型神经网络芯片的算力逐年增长（示意）。数据来源：NVIDIA、Google、华为各代产品官方规格（FP16/BF16 峰值算力）



图 1-2 神经网络模型训练计算量随发布年份的增长趋势（训练算力约每 6 个月翻倍）。图
 片来源：Sastry et al.^[4] (arXiv:2402.08797)

神经网络芯片是面向神经网络推理与训练而设计的专用硬件加速器，通常具备高并行度的矩阵/向量运算单元、层次化片上存储以及面向数据流的访存与调度机制，与通用 CPU/GPU 在编程模型与能效比上形成差异化补充。国内外已有多款代表性神经网络芯片落地。例如 Google 的 TPU 系列在云端推理与训练场景中采用脉动阵列作为计算单元的核心架构。华为昇腾、寒武纪思元等国产神经网络芯片在云端与边缘端部署中兼顾多核扩展与软件栈生态。NVIDIA 的推理加速方案（如 NVDLA 等）则与已有 GPU 生态形成协同。Cerebras 则是超大算力多核神经网络芯片的代表产品（如图 1-3），将整片晶圆集成为单一大规模计算平面，将多核扩展推向物理极限。这些芯片在单核算力、多核互联与存储层次上形态各异，但共同面临算力增长时的系统级瓶颈问题。随着单芯片算力规模受限，多核阵列成为扩展神经网络芯片算力与兼顾能效的主流方向，但也引入了片上网络拥塞、主存带宽瓶颈与跨核调度复杂度高等系统级挑战^[5-6]。

在先进工艺下，芯片的流片成本高昂且迭代周期长。为了在设计早期进行架构探索与软硬件协同优化，构建端到端、高保真且可扩展的多核神经网络芯片模拟平台具有重要意义。一方面，周期级模拟能够在统一时间轴下刻画计算、通信与存储的竞争行为，从而定位端到端性能瓶颈。另一方面，模拟平台为映射策略、互连配置与存储参数的设计空间探索（Design Space Exploration, DSE）提供低成本试错环境^[7-8]。从单核到多核阵列、单芯片再到晶圆级形态，算力扩展不断伴随着系统级瓶颈。

1.1.2 多核神经网络芯片架构设计挑战

随着单核神经网络芯片算力逼近工艺与功耗极限，业界普遍采用多核阵列的方式横向扩展峰值吞吐。然而，多核架构在带来线性算力增长的同时，也引发了一系列系统级瓶颈。在扩展的极端形态上，晶圆级引擎等设计将通信、存储与调度挑战更为集中地显现。总体而言，多核神经网络芯片的端到端性能受制于存储、通信与编译调度三个

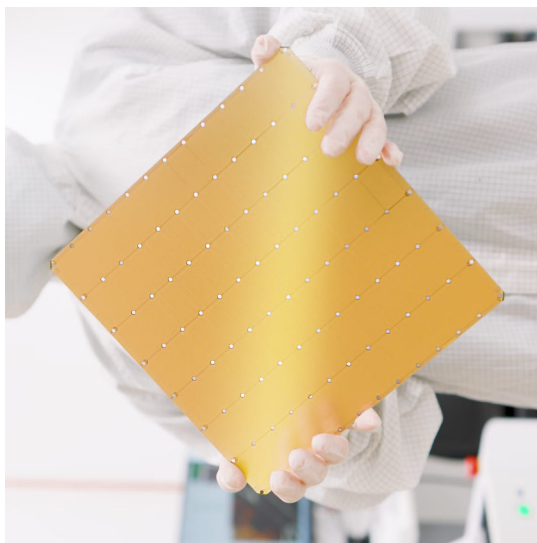


图 1-3 Cerebras 晶圆级引擎示例：算力需求驱动下从单核到多核、从单芯片到晶圆级扩展的典型形态之一。图片来源：Cerebras Systems, Inc. 官网

维度的“墙效应”，三者相互耦合，共同决定了实际可达的有效利用率。

1) 存储墙

“存储墙”是指计算单元的峰值吞吐增速远超片外存储（Dynamic Random-Access Memory, DRAM）的带宽增速所导致的性能瓶颈。在多核神经网络芯片系统中，这一问题尤为突出：当核心数量线性扩展时，峰值算力线性增长，但共享的 DRAM 带宽难以同步扩展，使得存储带宽成为系统吞吐的瓶颈^[5]。

大语言模型进一步放大了上述矛盾。一方面，推理侧的自回归解码在逐 token 生成时往往难以像预填（Prefill）阶段那样充分摊薄通用矩阵乘（General Matrix Multiply, GEMM）类算子，需反复从高带宽存储器（High Bandwidth Memory, HBM）或其他类型 DRAM 拉取海量权重，并伴随 KV Cache 的读写与扩容，单位有效计算所对应的访存量显著上升。另一方面，训练侧除前向与反向中的激活与梯度外，还需保存优化器状态与检查点，在模型宽度与并行维度增加时，片外带宽与容量同时成为扩展效率的约束。权重与 KV 的量化、稀疏化等工程手段可以降低字节搬运量，但在多核与多租户场景下，通道争用与行缓冲冲突仍会使实际带宽低于标称峰值，存储墙以“有效带宽不足”的形式持续存在。

从屋顶线（Roofline）模型^[9]视角看，硬件峰值算力与存储带宽在算术强度（每字

节访存所完成的运算量, FLOP/Byte) 维度上共同构成性能上界: 当工作点的算术强度低于脊点 AI_{ridge} 时, 性能主要由带宽与算术强度的乘积决定, 处于访存受限区。超过脊点后则受峰值算力封顶, 处于算力受限区。大模型推理中的部分注意力、归一化与逐元素算子, 以及解码阶段整体较低的批次并行度, 常使实际工作点更接近访存受限区, 此时单纯提高矩阵运算单元的标称峰值, 对端到端 token 延迟或吞吐的边际收益有限, 系统表现更敏感于 HBM/DRAM 层次的实际带宽。图1-4 给出该屋顶模型的示意。算术强度低于脊点时性能受存储带宽约束 (斜线段), 高于脊点时受峰值算力约束 (水平段)。图中圆点大致标出典型大语言模型 (Large Language Model, LLM) 推理中解码 (Decode, 逐 token、算术强度较低, 贴近带宽斜线) 与预填充 (Prefill, 批处理提示、算术强度相对较高, 常接近算力平顶) 的示意工作点, 非实测数据。

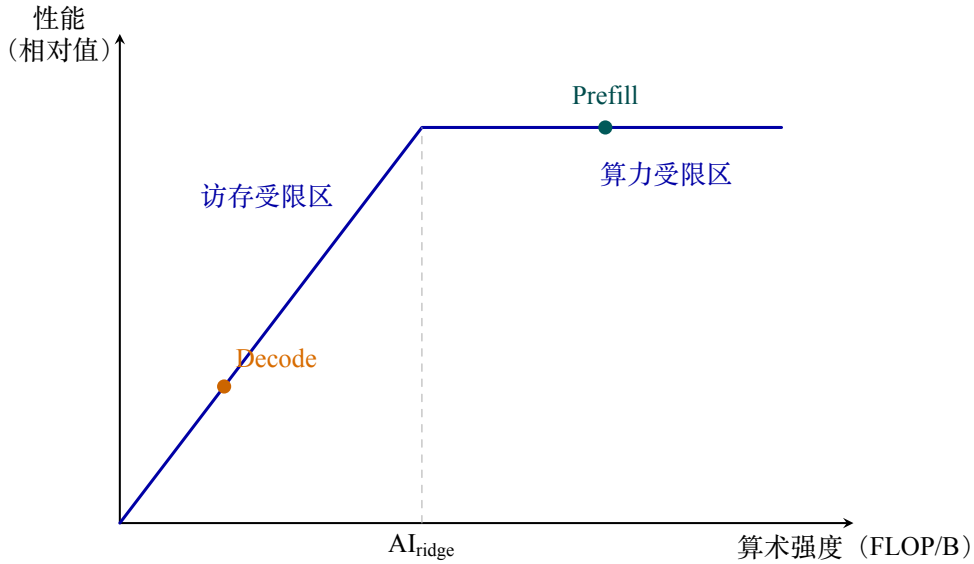


图 1-4 屋顶线模型示意。模型思想见 Williams et al.^[9]

“存储墙”具体表现在以下几个方面: 首先, 在大模型推理中, 自注意力机制的 KV Cache 随序列长度线性增长, 其反复读取的特性使内存访问量远超计算量。FlashAttention 等工作虽通过分块计算减少了片外访存次数^[10-11], 但在多核场景下 KV Cache 的跨核分片仍需经由片上网络搬运, 引入额外的通信延迟与带宽竞争。其次, 在 LLM 推理服务场景中, 多租户并发请求共享有限的 DRAM 通道, PagedAttention 等内存管理策略通过虚拟化技术缓解了碎片化问题^[12], 但多核环境下通道级争用的排队效应仍会造成尾延迟劣化。此外, 从硬件层面看, DRAM 时序参数 (t_{RCD} 、 t_{RP} 、 t_{RAS} 等) 对有效带宽的影响是高度非线性的: 当多个核心同时发起读写请求时, 行缓冲区命中率下降, Bank (存储体, DRAM 的基础结构) 冲突频率上升, 实际可用带宽可能远低于标称峰值。这种竞态行为只有通过周期级的存储控制器模拟才能准确刻画。

2) 通信墙

多核神经网络芯片通过片上网络 (Network-on-Chip, NoC) 实现核间数据传输。当并行策略将神经网络层的计算分配到多个核心上时, 层间的中间激活值、部分和以及权重分片等数据需要经由 NoC 在核心之间搬运。随着核心数量增加和互连直径增大, 通信开销呈现超线性增长趋势^[13-14], 形成“通信墙”。

在张量并行模式下, 每个核心仅计算输出张量的一个分片, 全局规约 (AllReduce) 或全局收集 (AllGather) 等集合通信操作需要在每一层结束时同步全部核心的结果。此时, 最慢路径的传输延迟决定了整体完成时间, 而 NoC 中的链路拥塞与仲裁等待会显著放大这一尾延迟。在流水并行模式下, 虽然核间通信量相对较小, 但流水气泡的大小与核间传输延迟直接相关, 若通信延迟超过计算延迟则流水线效率急剧下降。

此外, NoC 的拓扑结构、路由算法、虚拟通道数量、Flit 大小与注入队列深度等参数共同影响着网络的吞吐与延迟特性。不同参数配置下的性能差异可达数倍, 且与具体的流量模式紧密相关, 难以通过解析模型精确预测。BookSim2 等工具虽然提供了详细的周期级 NoC 模拟能力^[13], 但在独立使用时只能接受合成流量或离线跟踪, 无法与神经网络芯片计算核心的动态行为形成闭环反馈。

3) 编译与调度墙

多核神经网络芯片的编译与调度面临的核心困难在于: 工作负载到硬件的映射空间随核心数量和网络层数呈指数增长, 且映射质量与底层硬件参数强耦合^[6]。具体来说, 编译器需要决定每一层 (或算子组) 的空间切分策略 (在哪些核心上执行)、时序调度顺序 (层间依赖的流水安排) 以及数据布局 (权重与激活值的存放位置)。每一项选择都会影响计算效率、通信开销与存储占用, 三者之间存在复杂的权衡关系。

Gemini 等工作提出了层流水空间映射编码 (Layer-Pipeline Spatial Mapping) 与模拟退火搜索框架^[6], 能够在多核架构上进行映射与架构的联合探索。MLIR 等多层中间表示基础设施也为神经网络芯片后端的编译优化提供了统一的抽象与变换框架^[15-16]。然而, 编译器生成的调度方案的实际效果, 往往取决于运行时 NoC 拥塞、DRAM 排队延迟等系统行为, 单纯依靠编译期的代价模型难以准确评估。这就产生了“编译-执行语义鸿沟”: 编译器的代价模型通常基于峰值带宽、理想延迟等静态假设, 而实际系统中的资源竞争与时序交互是动态且非线性的。缩小这一鸿沟需要一个能够执行编译器生成调度方案并给出周期级反馈的端到端模拟平台, 从而实现编译与架构的闭环协同优化。

4) 三墙耦合与模拟器需求

上述三类挑战并非彼此独立, 而是高度耦合的。例如, 增加 DRAM 通道数虽然缓解存储墙, 但可能因 NoC 上的存储请求流量增加而加剧通信墙。调整映射策略以减少

跨核通信，可能导致单核 SRAM (Static Random Access Memory, 静态随机存取存储器) 不足而增加 DRAM 溢写，反过来加重存储墙。这种多维耦合特性使得对任何单一子系统的孤立优化都可能在系统层面产生意外的负面效果。

因此，构建一个能够将计算、通信与存储三者纳入统一时间轴的端到端周期级模拟平台，成为多核神经网络芯片架构设计的关键需求。这样的模拟器不仅能够定位性能瓶颈的根因，更能为编译器与架构师提供定量可靠的反馈，支撑大规模设计空间探索。上述需求构成了本文研究工作的出发点。

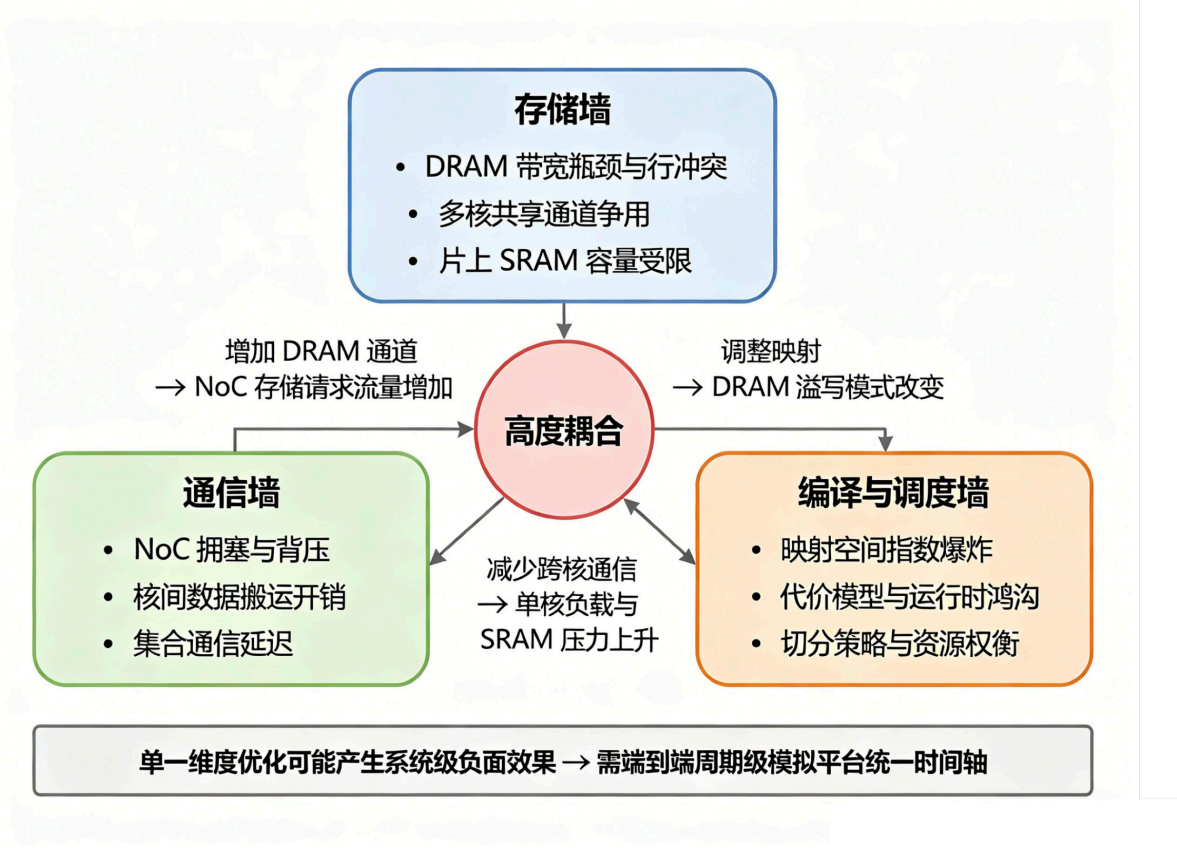


图 1-5 多核神经网络芯片面临的“三墙”及其耦合关系

1.2 研究现状与不足

本节围绕多核神经网络芯片的周期级模拟、任务调度与编译优化、互连与存储建模、以及软硬件协同设计与评估四个方向，系统综述国内外相关研究进展，分析其在多核神经网络芯片系统级分析与闭环优化中的能力边界，并逐项指出其与本文研究目标之间的差距，由此引出本文所构建的 IR 驱动、松耦合、可扩展的端到端多核神经网络芯片模拟平台的研究动机。

1.2.1 神经网络芯片模拟器

在多核架构下，模拟器不仅是验证正确性的工具，更是软硬件协同设计的核心引擎。图1-6 简要给出了多核神经网络芯片模拟器在架构探索中的作用示意，同时也是本篇工作提出的模拟器的工作流程图。模拟器接收上游编译/调度器生成的中间表示 (Intermediate Representation, IR, 包含工作负载与数据依赖) 以及底层硬件架构参数 (如计算单元数量、片上静态随机存取存储器容量、片上互连拓扑与主存带宽等) 作为输入。在统一的时间轴下，模拟器驱动计算核心、片上网络 (NoC) 与片外存储 (DRAM) 协同推进，并输出端到端总周期、阶段分解统计 (计算、通信、访存等待占比) 以及资源利用率等定量指标。基于这些反馈，架构师与编译器开发者可以形成闭环优化：例如当发现访存等待占比过高时，可调整存算配比 (增加 SRAM 或 DRAM 通道)。当发现网络背压严重时，可优化互连拓扑与路由。或者反馈给调度器以改进数据切分与映射策略。

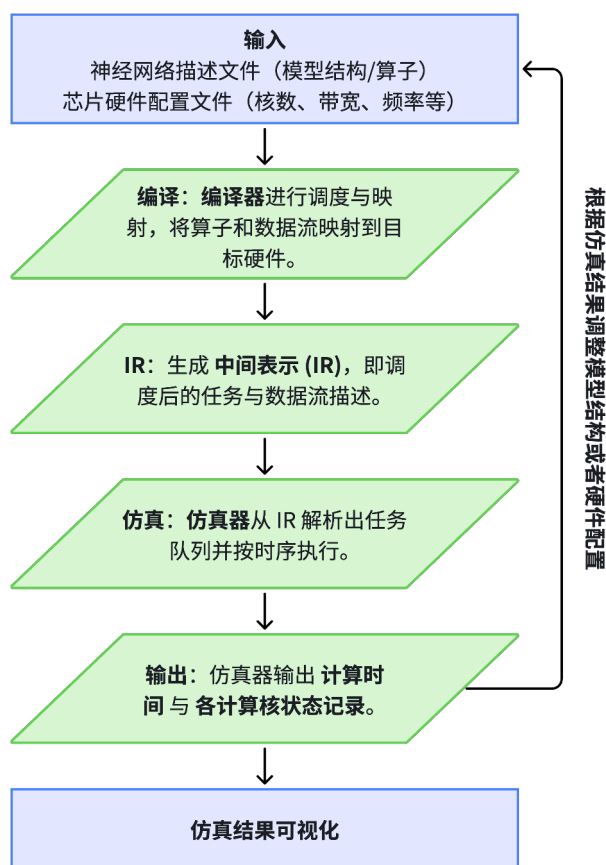


图 1-6 多核神经网络芯片模拟器工作流程图

面向神经网络加速器的模拟与评估工具，按建模抽象层次与覆盖范围可大致分为三类：（1）以计算阵列数据流与复用分析为核心的解析模型，关注峰值吞吐、利用率与

能耗等一阶指标；(2) 兼顾片上存储层次、调度映射与互连行为的架构级周期模拟器，能够反映算子级到层级的运行时开销；(3) 将编译栈、调度器输出与系统级时序打通的端到端模拟框架，支持在统一时间轴上观察多核、多时钟域与存储争用等耦合效应。不同层次的工具在建模精度、仿真速度与集成复杂度之间各有取舍，难以被单一工具同时满足。

在解析与数据流层面，早期研究多聚焦于单颗加速器或单一脉动阵列。Eyeriss 从片上数据复用与能效出发，系统研究了卷积数据流（如权重固定、输出固定、行固定）对能耗的影响，为后续数据流分类与评估奠定了基础^[17]。Google TPU 公开了首款大规模部署的云端推理脉动阵列设计与实测数据，确立了“指令受限—计算受限—存储受限”的系统级分析范式^[18]。NVDLA 作为开源的推理加速器 IP，提供了从硬件 RTL 到编译栈的参考实现，为学术界研究“软件栈—硬件行为”链路提供了可复现的对象^[19]。在上述硬件参考平台之上，SCALE-Sim 针对脉动阵列数据流给出了轻量级周期级估算工具，可快速枚举不同数据流配置下的利用率与访存需求，广泛用于数据流与阵列规模的早期评估^[20]。MAESTRO 进一步以数据为中心描述 DNN 在可配置加速器上的数据复用、延迟与能耗，为数据流驱动的解析评估提供了统一抽象^[21]。TETRIS 则把 3D 存储与加速器数据流放在同一优化闭环中分析，揭示了带宽/容量层次对算子性能的关键影响^[22]。这一类工作普遍以解析模型或周期级乘法累加（Multiply-Accumulate, MAC）计数为主要手段，在算子级或网络层级提供性能上界分析，但对多核 NoC 竞争、DRAM 排队与跨核同步等动态系统级行为的刻画能力有限，尤其在多核扩展到十核乃至数十核时，其假设的近理想带宽利用与零争用条件与真实系统行为偏离较大^[23]。

在架构级与系统级模拟层面，随着多核神经网络芯片与异构 SoC 逐渐成为主流，研究者开始把片上网络、片外存储与多核同步纳入统一的模拟闭环。ONNXim 针对多核神经网络芯片设计了周期级模拟流程，重点刻画多核与多租户场景下 NoC 与 DRAM 的竞争行为，能够对端到端延迟与吞吐进行定量评估，为多核神经网络芯片的系统级瓶颈分析提供了可用的工具基础^[7]。在更广泛的系统级模拟领域，将通用处理器模拟框架 gem5 与 NVDLA 加速器模型相结合的工作，实现了从编译、调度到系统执行的端到端链路，为“软件栈—硬件行为”的一体化评估提供了参考架构^[24]。此外，近年来针对稀疏化、流式张量与新型数据流的模拟工作（如基于张量抽象的事件驱动模拟器^[25]），也拓展了模拟器在新兴算子形态与编译策略下的适用范围。这类工作相对解析模型提供了更贴近真实行为的系统级指标，但大多在编译/调度器的输入接口、子模块可插拔性以及可解释统计输出等工程能力上仍存在欠缺，较难直接服务于“一次改映射、一次改硬件配置”的高频迭代场景。

综合国内外现状，面向多核神经网络芯片架构探索与编译协同的周期级模拟工具仍存在三方面典型不足。第一，上游编译/调度器输出的 IR 与模拟器输入格式往往脱

节：模拟器多依赖自定义的踪迹或简化的工作负载描述，无法直接消费调度器生成的、带有显式依赖与数据流向的 IR，导致“改映射即改模拟输入”的迭代成本高，且难以保证语义一致。第二，NoC 与 DRAM 子系统的建模多为内置固定实现，缺少统一的抽象接口与可插拔后端设计，研究者难以在不改动核心执行逻辑的前提下替换为不同精度（如简单延迟模型 vs. 周期级 NoC/DRAM 模拟器）或不同配置进行对照实验，设计空间探索的变量隔离性不足。同时，子模块的版本升级与替换往往牵连大量执行层代码改动，制约了工具链的可演化性。第三，模拟输出多为端到端总周期或简单分类统计，缺乏与“计算—通信—存储”阶段一一对应的可解释指标（如每核不同状态时间占比、显式 NoC 背压周期、DRAM 请求排队周期等），不利于瓶颈归因与架构参数敏感性分析，也不利于把模拟结果直接反馈给调度器以驱动映射改进。本文针对上述不足，设计以调度器 IR 为输入的端到端模拟流程、面向异构子模块模拟器的松耦合 NoC/DRAM 接口、以及可复现的阶段分解与热点统计体系，为多核神经网络芯片的系统级研究提供一套“IR 输入—周期级执行—可解释输出”闭环的模拟基础设施。

1.2.2 任务调度与编译优化

神经网络在神经网络芯片上的高效执行依赖编译与调度两端的协同：编译器负责算子融合、数据布局与循环变换，调度与映射则决定计算与数据在空间（多核）与时间（流水、批处理）上的分配。这两端的决策相互耦合——循环变换的可行性受限于硬件数据流与片上存储容量，而映射的优劣又取决于编译器所暴露出的并行度与复用机会——因而现代神经网络芯片编译栈往往需要在图级、算子级与映射级之间建立端到端的可评估闭环。

在编译基础设施方面，多层中间表示框架（Multi-Level Intermediate Representation, MLIR）为领域专用后端提供了可扩展的 Dialect 与变换管道，使得从高层图表示到硬件相关指令的 lowering 过程更加模块化与可复用^[15]。面向具体神经网络芯片的编译栈在此基础上不断丰富，例如面向高通 Hexagon NPU 的 MLIR 编译栈，展示了从模型到神经网络芯片指令的完整链路及在真实硬件上的优化实践^[16]。在更通用的编译框架层面，TVM 以张量表达式与学习式代价模型为核心，结合 AutoTVM/Ansor 等自动调度机制，在多种加速器与 GPU 上实现了算子级的跨平台优化^[26]，为自定义神经网络芯片后端提供了可借鉴的调度语言与搜索算法。这类工作侧重单核或单芯片内的算子优化与指令生成，对多核映射与跨核数据流的形式化描述相对较少。同时，其所依赖的代价模型大多以峰值算力与理想带宽为基础，缺乏对多核 NoC 与 DRAM 动态行为的刻画能力。

在映射与调度层面，多核场景下的空间划分与流水编排是一个组合优化问题，搜索空间随核心数与网络深度快速增长。Timeloop 与 Accelergy 为加速器映射与能耗建模提供了一套以循环嵌套变换为核心的分析方法，被广泛用于单核与多级存储层次上的

映射枚举^[27-28]。CoSA 将映射问题形式化为约束优化并用求解器在合法映射空间中搜索近优解^[29]。Interstellar 借用 Halide 的调度语言描述 DNN 到加速器的映射，通过系统化枚举数据复用与并行策略分析设计空间^[30]。面向更大规模的多核/多芯片系统，Gemini 将层流水与空间映射编码为统一的表示 (Layer-Pipeline Spatial Mapping)，并采用动态规划算法与模拟退火算法在映射空间中搜索，实现了在多核架构上对映射与架构参数的联合探索，其输出的调度结果以结构化 IR 描述各核工作负载及依赖关系^[6]。近年来，面向缓存与片上存储的分配策略也出现了专门化的优化工作，例如 SET 通过系统化的张量切分与存储规划提升利用率^[25]，BufferProspector 从缓冲分配角度系统分析并搜索片上存储的最优方案^[31]，显示出“调度—映射—存储分配”的联合优化正在成为新的研究重点。

然而，上述映射与调度方法普遍依赖对“执行代价”的估计以驱动搜索，而代价模型通常基于峰值算力、理想带宽、固定延迟等简化假设，难以反映实际运行时的 NoC 拥塞、DRAM 排队与行冲突等多核系统级效应。因此，映射与调度算法的有效性往往需要在更贴近真实系统行为的模拟环境中二次验证。若模拟器无法直接消费调度器 IR 并反馈周期级指标，则只能通过事后踪迹注入或手工改写输入进行间接评估，迭代效率与结论可信度均受限，难以形成“调度搜索—模拟评估—反馈改进”的短周期闭环。本文通过 IR 驱动、闭环执行的设计，使调度器输出可直接驱动模拟并得到阶段级统计与可解释的瓶颈归因，为“编译—调度—架构”的协同优化提供可复用的评估闭环。

1.2.3 互连与存储优化

多核神经网络芯片的端到端性能受片上网络与片外存储的时序与竞争行为影响显著，相关研究依赖高保真的 NoC 与 DRAM 建模工具，并需要把二者与计算核心的执行时序在同一时间轴下对齐，才能反映真实系统中的争用、排队与背压传播等动态效应。

在片上网络方面，BookSim（及其后续版本 BookSim2）被广泛用作周期级 NoC 模拟器，支持多种拓扑（如 Mesh、Torus、Fat-Tree）、路由算法（如 XY、自适应路由）、虚拟通道数与缓冲深度、分片与数据包的划分等可配置维度，能够刻画链路仲裁、排队与尾延迟等拥塞效应，是 NoC 研究中事实上的参考工具^[13]。近年来，面向高带宽与多流并行的开源 NoC 设计（如 FlooNoC）在物理层与协议层提供了可配置的互连模型，为集成到全系统模拟中提供了可选组件^[14]。这类工具在单独使用时，通常接受合成流量（如均匀随机、热点、置换流量）或由其他工具生成的数据包级踪迹（Packet-Level Trace）作为输入，无法根据神经网络芯片核心的实时执行状态动态产生请求，因而难以体现“计算完成—发起传输—对端消费”的依赖链与背压传播。在多核神经网络芯片场景下，NoC 与计算和存储的耦合评估仍依赖上层模拟框架把核心状态、读写事务与 NoC 推进过程统一组织起来，这对模拟器的集成能力与接口抽象提出了较高要求。

在片外存储方面, DRAM 的时序约束、Bank 并行度与调度策略对有效带宽影响巨大。Ramulator2 以模块化与可扩展的方式实现了多种 DRAM 标准 (如 DDR3、DDR4、LPDDR、HBM 等) 的时序与调度建模, 支持行缓冲、刷新策略与多种调度算法 (如 First-Ready First-Come-First-Served, FR-FCFS), 使排队延迟、行命中率与冲突行为可在模拟中复现^[8]。在多核神经网络芯片中, 多个核心并发访问 DRAM 会共享通道与 Bank, 请求交织与行冲突会导致实际带宽远低于标称峰值, 且对通道数、Bank 数、地址映射策略等参数高度敏感。若叠加以大模型推理所特有的权重/KV Cache 密集访存特征, 这一矛盾在吞吐与延迟两端都会被进一步放大^[5]。只有将 DRAM 模型与多核执行时间轴对齐的周期级模拟, 才能系统性地评估这类动态效应在不同架构配置下的表现。

此外, 面向多核神经网络芯片的互连与存储研究还表现出若干值得关注的新趋势: (1) 存储墙问题推动了片上存储层次的精细化建模与搜索, 如缓冲分配与张量切分的联合优化^[25,31]; (2) 高带宽片外存储 (如 HBM) 在云端与高端推理平台上的普及, 使得研究者需要在模拟中区分 DDR、LPDDR 与 HBM 等不同标准在带宽—延迟—并行度上的差异; (3) 计算与通信的耦合评估越来越强调“事件驱动”的逻辑: 即请求的发起时刻不再独立于核心执行, 而必须由核心状态机在运行时动态触发, 以真实反映依赖闭环。

综合来看, NoC 与 DRAM 的优化若脱离具体映射与负载特征进行孤立分析, 难以反映端到端效果。若要与映射和调度联动评估, 又需要统一的模拟平台在相同时间基准下驱动 NoC 与 DRAM 请求、并汇总可解释的阶段级统计。然而, 现有模拟框架大多将 NoC/DRAM 后端以硬编码方式嵌入执行层, 难以在不改动核心代码的前提下切换不同后端或不同精度模型, 阻碍了“简单模型快速迭代—高保真模型精细评估”的分层研究路径。本文针对这一痛点, 通过定义 NoC 子系统与 DRAM 内存子系统的松耦合接口, 将 BookSim2 与 Ramulator2 封装为可插拔后端, 在保持执行层逻辑不变的前提下支持简单后端与全后端的对照实验与设计空间探索, 弥补了互连与存储优化研究与多核神经网络芯片端到端评估之间的衔接不足。

1.2.4 软硬件协同设计与评估

随着神经网络芯片在架构参数空间 (如处理单元数量、片上缓存容量、互连带宽等) 与软件映射空间 (如数据流策略、层间调度、算子融合等) 上的设计选择日益增多, 软硬件协同设计 (Hardware-Software Co-design) 成为兼顾性能与效率的关键方法论。其核心思想在于, 将算法、编译映射与硬件架构作为统一的优化空间进行联合探索, 而非在固定硬件上优化软件或在固定映射下搜索硬件参数。

在预 RTL 阶段的快速架构评估方面, Aladdin 提出了基于动态数据依赖图的预 RTL 加速器性能与功耗模拟方法, 能够在不编写 RTL 的前提下对定制加速器进行大规模设

计空间探索^[32]。SCALE-Sim 对脉动阵列的不同数据流配置进行快速周期估算，为硬件参数与映射的联合评估提供了轻量级工具^[20]。这类工作通过降低评估成本来扩大可搜索的设计空间，但多聚焦于单核或单个计算阵列，对多核 NoC 争用与共享存储竞争的建模有限。

在映射与架构的联合搜索方面，除已述的 Gemini^[6] 与 Timeloop/Accelergy^[27-28] 外，CoSA 将加速器的调度问题形式化为约束优化，利用求解器在合法映射空间中高效搜索近优解^[29]。Interstellar 借用 Halide 的调度语言描述 DNN 到加速器的映射，通过系统化枚举数据复用与并行策略来分析设计空间^[30]。这些方法在编译映射与硬件参数之间建立了定量关联，但其评估所依赖的代价模型多为解析模型或简化的周期估计，难以体现 NoC 拥塞、DRAM 行冲突等多核系统级效应。

在算法与硬件的端到端联合优化方面，Hardware-aware NAS 在搜索网络结构时将目标硬件的延迟或能耗纳入评估函数^[33]，使搜索结果能同时满足精度与部署约束。与之类似的能效评估工作也在神经网络芯片的高能效单元设计层面不断深入，例如对低功耗计算单元与神经网络芯片结合的架构探索^[34]。但这类方法通常以固定硬件平台或预设代价模型为前提，对硬件侧的参数可调性与系统级动态行为考虑不足。当硬件架构本身也处于探索阶段时，仍需模拟器提供对不同架构配置下的执行代价估计与瓶颈归因。

另一个影响协同设计评估质量的关键因素，是“评估对象粒度”与“优化对象粒度”的匹配程度。若评估工具仅能给出端到端总周期或简单的计算/通信/存储三分类，则在多核神经网络芯片这类多维耦合系统中，研究者难以判断某次改动究竟改善了哪一环节、又是否引入了新的瓶颈。近年来以 Sze 等综述为代表的文献指出^[35]，高效神经网络硬件的设计与评估必须覆盖“算子—数据流—存储层次—互连”多个抽象层次，并要求评估工具能够对其中每一层提供可解释的量化指标。这对模拟器在统计输出上的工程设计提出了超出“总周期正确性”的更高要求。

综上，软硬件协同设计的有效性高度依赖评估工具在“精度—效率—可解释性”三者之间的平衡。解析模型速度快但精度受限于静态假设。RTL 仿真精度高但迭代成本过大。周期级模拟器——特别是能够直接消费编译/调度输出并在统一时间轴下反映多核系统行为的端到端模拟平台——填补了二者之间的精度—效率空白，为“改映射或改架构后快速得到可信反馈”的协同优化循环提供了必要的基础设施。本文所设计的模拟器正是面向这一需求，以 IR 为输入统一编译—模拟接口、以松耦合抽象支持子系统后端可插拔、以阶段级统计与热点可视化提升结果可解释性，使软硬件协同探索具备低迭代成本与可信的定量依据。与解析类工具相比，本文模拟器在牺牲一定速度的前提下补全了 NoC 背压与 DRAM 争用等系统级行为的刻画。与完整 RTL 或硬件模拟平台相比，本文模拟器在牺牲一定精度的前提下换取了远低于硬件流程的迭代成本与更丰

富的可观测性，从而在多核神经网络芯片架构探索与编译协同场景下占据一个独特的定位。

1.3 本文研究内容与章节安排

1.3.1 主要研究内容

本文围绕面向芯片早期设计阶段的“端到端周期级多核神经网络芯片模拟平台”的构建与利用展开。该平台以工作负载为最小执行单元，在统一时间轴上刻画计算、通信与存储三者的系统级争用，以支持流片前的架构参数与映射策略快速扫描，而不追求指令级或寄存器传输级的核内精度。围绕该定位，本文在平台设计、系统集成与实验验证三方面开展研究，具体研究内容与所做工作如下。

(1) 端到端模拟框架的设计与实现。以 Gemini 调度器生成的结构化 IR (JSON 格式)^[6] 为芯片工作负载输入，以芯片描述文件为可配置参数，设计并实现从 IR 解析到周期级执行、再到统计与踪迹输出的完整模拟流程。模拟引擎在统一全局周期下推进多颗计算核心，每核按“依赖到齐—计算—写回”的语义执行工作负载，并通过拉取式数据流与片上网络、片外存储交互：核间数据依赖经由 NoC 请求/响应完成，与 DRAM 的读写经由存储接口完成。计算、通信与存储三者共享同一时间轴，从而能够刻画排队、背压与资源争用对端到端周期的影响。为支持不同精度的评估需求，NoC 与 DRAM 均抽象为统一接口，并提供“简单延迟模型”与“周期级高保真后端” (BookSim2、Ramulator2) 两种实现，可在不修改执行层逻辑的前提下切换或对照，便于设计空间探索时的变量隔离与可复现对比。

(2) 统一任务抽象与可配置核心建模。将上游 IR 中的核心网格、每核 workload 序列及数据依赖解析为与格式无关的统一任务表示 (IRData/Workload)，执行层仅依赖该抽象，从而实现“内容与形式分离”，在 IR 格式演进时仅需扩展解析器。单核执行采用依赖驱动的五态状态机 (IDLE、LOADING、COMPUTING、WRITEBACK、DONE) 建模，核心计算能力与本地 SRAM 容量均参数化配置 (如乘加器数量、向量处理器单元数量、SRAM 容量)，以支持后续对计算资源与存储配置的扫描实验。在多时钟域场景下，通过全局周期与可配置的 DRAM 时钟频率实现多速率推进，在保证因果一致的前提下刻画计算核心/片上网络与 DRAM 频率差异对端到端性能的影响。

(3) 基于模拟器的实验验证与分析。为体现模拟器的实用价值，本文在模拟平台上开展从精度验证、多核瓶颈分析到设计空间探索的系统性实验。微基准与 ZeBu 对照：通过双核点对点依赖 (MB-1) 与仅 DRAM 依赖 (MB-2) 验证依赖因果、核间通信与 DRAM 闭环及子模块使用不同后端差异。与 Synopsys ZeBu 硬件模拟平台对比单核模拟结果，将相对误差控制在 $\pm 5.2\%$ 以内。真实 AI 负载瓶颈与热点分析：以调度器生

成的 ResNet34、ResNet50 等神经网络模型的 IR 驱动模拟，对端到端周期进行阶段分解（计算、NoC 背压、存储等待），并辅以各核阶段占比热力图等可视化，实现可解释归因。多核瓶颈分析与递进式优化：在 2×2 四核配置下，以 ResNet50 为负载，构造存储受限基线（DDR4 单通道），通过 DRAM 升级（DDR4 $\rightarrow$ HBM 4 通道，峰值带宽提升约 6.7 倍）与 SRAM 端口加宽（256 $\rightarrow$ 1024 B/cycle）两步递进式优化，将总周期从 238 803 降至 62 818 ($3.80 \times$ 加速)，揭示了“DRAM 带宽 $\rightarrow$ SRAM 端口带宽”的瓶颈逐级迁移现象。多核吞吐量设计空间探索：沿单核算力（乘法累加单元（Multiply-Accumulate, MAC）数量取 256/512/1024）、核数扩展（ $2 \times 2/4 \times 4/8 \times 8$ ）与 Batch 大小（1/4）三个维度展开 12 组配置仿真，发现存储受限状态下提升 MAC 算力无效、核数超过负载并行度上限后性能反而退化（ 8×8 死锁）、Batch 扩展效率仅约 24% 等定量结论，为芯片设计前期的参数定界提供了快速反馈依据。

综上，本文既完成了模拟平台从 IR 输入到周期级输出、从可插拔 NoC/DRAM 到多时钟同步的完整设计与实现，又基于该平台开展了从精度验证、瓶颈归因到多维设计空间探索的系统性实验，使模拟器不仅作为多核神经网络芯片研究的评估工具，也通过具体实验体现其在正确性校验、瓶颈定位与架构参数优化中的实际价值。

1.3.2 本研究的难点、贡献与创新

本文研究中的主要难点与贡献、创新点概括如下。

研究难点。（1）统一时间轴下的闭环耦合：在单一模拟引擎内将计算核心、NoC 与 DRAM 按周期推进并保证依赖与因果一致，需精细设计事件顺序与多时钟域同步机制。（2）松耦合下的异构集成：在不过度暴露 BookSim2/Ramulator2 内部细节的前提下，通过统一接口封装并支持与执行层无缝切换，保证对比实验的变量隔离与可复现性。（3）可解释的瓶颈量化：建立与“计算—通信—存储”阶段对应的统计口径与模拟记录踪迹输出，使端到端周期可分解、可归因，支撑架构与映射的针对性优化。

主要贡献与创新。（1）IR 驱动的端到端闭环（低成本快速反馈）：以调度器生成 IR 和硬件描述文件为输入，实现从解析、执行到统计的完整周期级模拟链路，使“改映射即改输入”的迭代与验证成本显著降低。（2）模块化设计与跨模块同步（可扩展能力）：通过子系统抽象接口及多速率推进机制，在同一框架下支持不同子模块后端与多时钟域建模，为设计空间探索提供可控、可对比的评估基础。（3）高精度模拟与系统性设计空间探索：通过微基准、ZeBu 对照（误差 $\pm 5.2\%$ 以内）与真实负载瓶颈分解及热点可视化等实验，系统展示模拟器在正确性校验与可解释归因上的实用价值。在此基础上，进一步开展多核瓶颈递进式优化（DDR4 $\rightarrow$ HBM $\rightarrow$ SRAM 端口加宽， $3.80 \times$ 加速）与多维设计空间探索（MAC 规模、核数、Batch 三维度 12 组配置），定量揭示瓶颈迁移规律、核数扩展上限与 Batch 扩展效率等设计启示，体现仿真平台在流片前快速参数定界

中的实际价值。

1.3.3 论文组织结构

本文共六章，按“问题与现状—基础与工具—架构与实现—实验与验证—总结与展望”的逻辑展开。第一章（绪论）在“研究背景与意义”中阐述算力需求与神经网络芯片发展脉络，并作为其子节分析多核神经网络芯片的“三墙”挑战及耦合关系，继而综述模拟器与编译调度等研究现状与不足，并概括本文研究内容、难点与创新点及章节结构。第二章（软硬件基础与前置技术）介绍多核神经网络芯片体系结构、工作负载特征、周期级模拟方法及 ZeBu/BookSim2/Ramulator2 的角色与集成方式，说明编译与调度挑战及调度器 IR 与模拟器的衔接，为第三、四章做铺垫。第三章（模拟器整体架构与松耦合接口设计）从总体架构、输入解析与对外接口阐述设计，给出 IR 驱动数据流与每周期推进顺序，定义 NoC/DRAM 抽象接口与可插拔后端及 BookSim2/Ramulator2 封装思路。第四章（可配置计算核心与全系统同步机制）介绍核心五态状态机、加载—计算—写回语义及拉取式数据流，说明 NoC/DRAM 运行时集成与跨时钟域推进。第三、四章合为“结构”到“行为”的完整叙述。第五章（模拟器验证与瓶颈及热点分析实验）围绕模拟器开展微基准与 ZeBu 总周期对照等精度验证实验，基于调度器生成 IR 对真实负载做端到端阶段分解及各核阶段占比热力图等瓶颈与热点分析。在此基础上，以 2×2 四核 ResNet50 为场景，通过 DDR4 $\rightarrow$ HBM $\rightarrow$ SRAM 带宽递进式优化揭示瓶颈逐级迁移，并沿 MAC 规模、核数与 Batch 三维度开展 12 组配置的设计空间探索，定量给出核数扩展上限、Batch 扩展效率等设计启示。第六章（总结与展望）总结平台设计、接口与实验工作，归纳贡献与不足，并对多核扩展、编译—模拟闭环及带宽/能耗约束下的架构配置等方向进行展望。

2 软硬件基础理论与前置技术

绪论指出多核神经网络芯片面临存储墙、通信墙与编译调度墙的耦合挑战，并提出了以端到端周期级模拟支撑架构探索与瓶颈归因的研究目标。本章介绍后续模拟器设计与实验所依赖的软硬件基础与前置技术。首先概述多核神经网络芯片的体系结构基础，建立硬件组成与数据流的直观认识。其次说明深度学习工作负载的计算与访存特征，以明确模拟器需要刻画的负载形态。然后简述周期级模拟方法论，并介绍硬件模拟平台 ZeBu 作为后文准确性验证的参考依据。接着说明本文所集成的关键工具底座 BookSim2 与 Ramulator2。最后介绍多核神经网络芯片编译与调度的挑战与相关工作（含 SET、Gemini 等），以及调度器 IR 与本文模拟器的衔接。本章为第三、四章的整体架构设计与核心建模奠定基础。

2.1 多核神经网络芯片体系结构基础

为便于后文理解模拟器所抽象的计算、存储与互连模块，本小节先给出通用神经网络芯片的典型结构概览，再分别介绍脉动阵列与数据流、以及片上 SRAM 与显式搬运，与本文建模的对应关系。神经网络芯片在计算、数据流与存储层次等方面的设计空间与权衡可参见综述^[35]。

在芯片级，神经网络芯片常采用多核布局：处理单元（Processing Element, PE）阵列构成计算核心，周围分布本地缓存或输入输出缓冲，通过片外接口与主存或其他芯片互联。各计算核心之间由片上网络（NoC）连接，形成可扩展的多核拓扑。业界代表性设计如 Google TPU 采用脉动阵列与高带宽片内存储实现数据中心级推理加速^[18]，NVDLA 等开源架构提供了可配置的卷积与数据处理器模块，便于软硬件协同评估^[19]，华为 Ascend Da Vinci AI Core 则以矩阵立方单元、向量单元与片上分层缓存的组合为代表^[36-37]。上述设计在矩阵阵列拓扑、缓存层次与专用/共享策略上各有侧重，但在核内功能组成上高度相似，本文第四章将在其共性基础上抽象出通用的单核结构并据此参数化建模。多核扩展在提升峰值算力的同时，也使得核间数据搬运与共享存储访问成为系统级瓶颈。NoC 的拓扑与路由、以及各核与片外存储的带宽分配，共同决定了实际可达的利用率和尾延迟。在单核或单簇内部，典型功能模块包括：控制单元负责指令或调度序列的解析与下发。数据传输单元包含 DMA 与 NoC 路由器，负责片内与片外数据的搬运及与邻核的通信。全局缓存（或层次化 SRAM）作为输入特征图、权重与输出特征图的暂存与共享区。处理单元阵列（如脉动阵列）执行矩阵乘、卷积等 MAC 密集运算。向量处理单元则负责激活、归一化等逐元素或向量运算。数据在“全局缓存 ↔ 处理单元阵列 ↔ 向量处理单元”以及“数据传输单元 ↔ 片外 DRAM”之间按调度流

动。图2-1给出上述通用结构的示意，便于读者建立对神经网络芯片组成与数据流的大致认识。左半部分是芯片级多核布局（处理单元、本地缓存/输入输出、片外接口及互连），右半部分描述了单核/簇内部功能模块与数据流（控制单元、数据传输单元、全局缓存、处理单元阵列、向量处理单元及输入特征图/权重/输出特征图流向）。如绪论所述，Cerebras 等晶圆级设计将多核扩展推向极致。本文模拟器对“处理单元阵列”抽象为可配置的乘加器数目与向量单元数目，对“全局缓存/本地 SRAM”抽象为容量与生命周期约束，对“数据传输单元”中的 NoC 与片外接口则通过网络接口与存储接口对接，详见第三、四章。

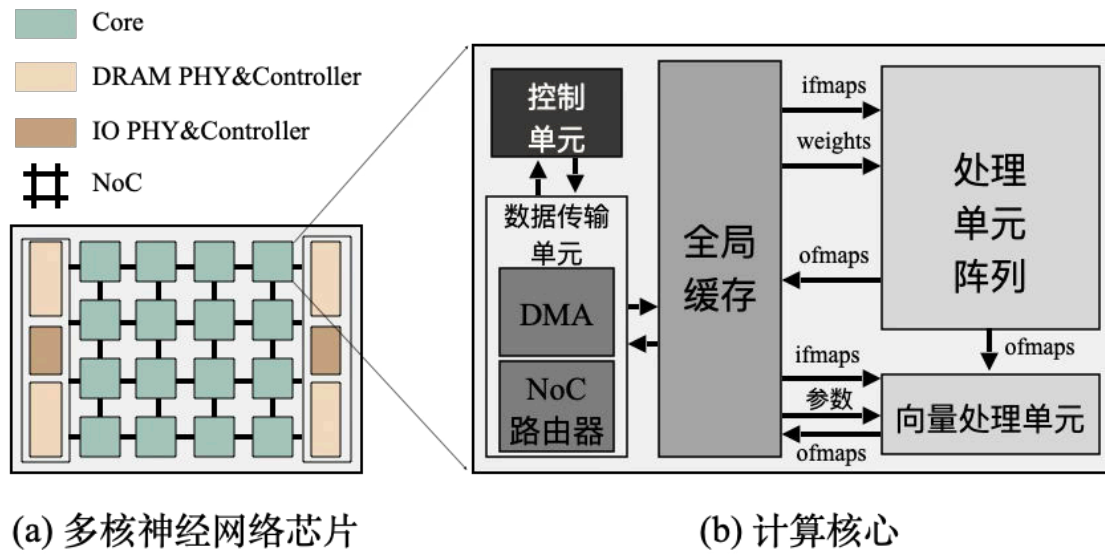


图 2-1 一种通用神经网络芯片的典型结构示意图

2.1.1 脉动阵列与数据流

脉动阵列通过规则的数据流与邻近通信实现高数据复用，是神经网络芯片中常见的核心计算结构。数据在阵列内按“脉动”节奏流动，同一数据在多个 PE 间复用，从而降低对片外带宽的依赖。不同数据流策略（如权重驻留、输出驻留、行驻留等）会改变权重与激活在阵列上的驻留方式，进而影响片上 SRAM 占用、核间/核外数据搬运量以及 DRAM 访问模式，从而在系统层面表现为不同的计算—通信—存储平衡。图2-2展示了多核神经网络芯片架构下的片上数据流示意。张量数据通过片外 DRAM 加载至片上网络（NoC），在不同计算核心的 SRAM 之间进行传输与复用，完成计算后再将最终结果写回 DRAM。这种显式的数据搬运与通信机制显著降低了对片外带宽的依赖。Eyeriss 等工作系统刻画了行驻留等数据流对能效与复用率的影响^[17]。MAESTRO 则以数据为中心对 DNN 映射的复用、性能与硬件代价进行快速解析建模，为设计空间探索提供了工具基础^[21]。本文关注多核系统级瓶颈与设计空间探索。模拟器对单核计算采用“可参

数化的乘加器数目与向量单元数目等吞吐能力 + 显式数据搬运依赖”的抽象，不展开阵列内部数据流细节。实验中可通过调节计算资源参数与映射方式，间接体现数据流与映射对端到端性能的影响。

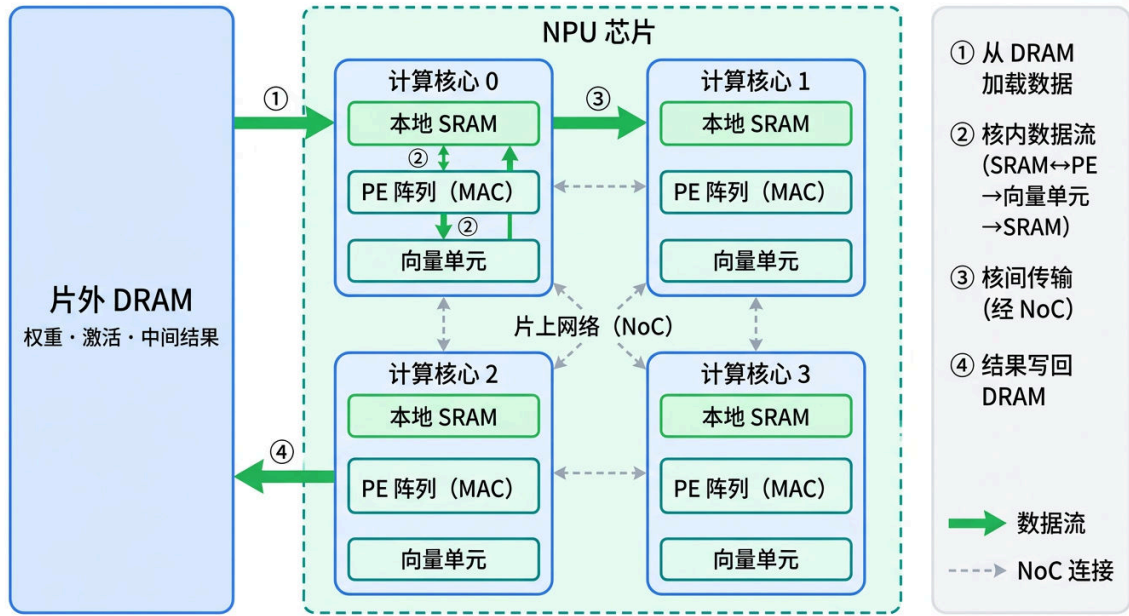
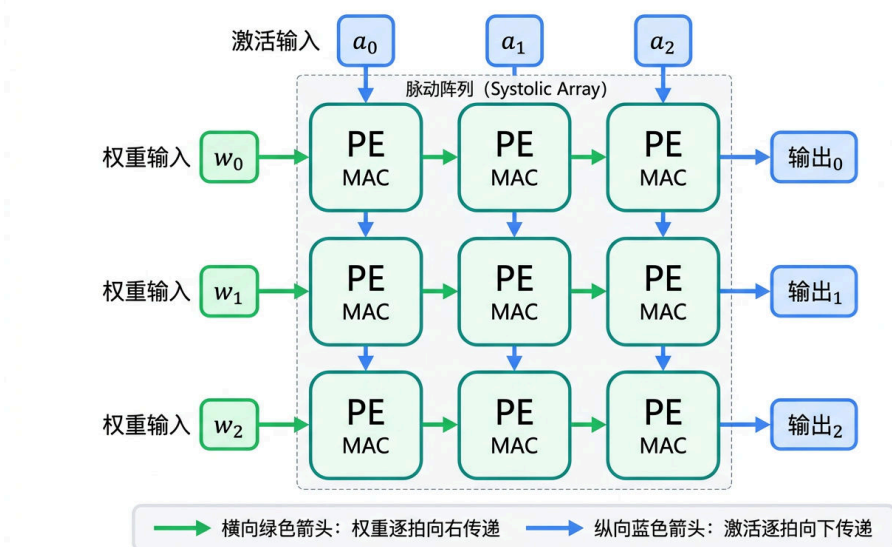


图 2-2 多核神经网络芯片片上数据流示意图

在微架构层面，脉动阵列内部的数据流可进一步细化。图2-3给出了一个 3×3 脉动阵列的数据流示意。权重沿水平方向逐拍向右传递（绿色箭头），激活沿垂直方向逐拍向下传递（蓝色箭头），每个 PE 在每拍接收来自左侧的权重与上方的激活，执行一次乘加（MAC）操作后将数据传递给相邻 PE。部分和在阵列右侧输出。这种规则的数据流动使得每份数据在多个 PE 间被复用，从而以较低的片外带宽需求实现高吞吐的矩阵运算。

2.1.2 片上 SRAM 与显式搬运

与传统 CPU 的缓存不同，NPU 常采用编译器或调度器显式管理的片上缓存（常用 SRAM 构成）。编译与调度在空间上把张量切分为小块，在时间上决定何时搬运、何时计算以及搬运与计算的重叠程度。双缓冲等技术可在计算当前块的同时预取下一块，从而提高计算单元利用率。片上容量、端口带宽与数据驻留生命周期共同决定了可选的分块粒度与复用效率，也是多核系统中“存储墙”形成与缓解的关键因素^[5]。本文模拟器对每核配置本地 SRAM 容量，并通过传输标识与引用计数刻画多消费者共享同一输出时的驻留需求，从而在周期级推进中反映 SRAM 约束对可行映射与端到端周期的影响。

图 2-3 3×3 脉动阵列数据流示意图

2.2 深度学习工作负载的计算与访存特征

在明确神经网络芯片的典型结构与存储、互连层次之后，本节在人工智能与大模型发展的背景下，从需求背景、算子形态与推理访存特征三方面总结运行于神经网络芯片上的深度学习工作负载的计算与访存特点，以便理解模拟器所需刻画的负载形态及其对架构与调度的要求。

2.2.1 人工智能与大模型发展对神经网络芯片的需求背景

近年来，以深度学习为代表的人工智能技术在计算机视觉、自然语言处理、多模态理解等任务上取得显著进展，模型规模与数据量持续增长，对算力与能效提出了更高要求^[3]。基于 Transformer 架构的大规模预训练模型（大模型）在自然语言理解与生成、代码生成、逻辑推理等场景中成为主流形态，参数规模从数亿至数万亿不等。云端与边缘侧的部署与推理需求推动了专用加速器与多核神经网络芯片的发展^[2]。模型演进、训练算力需求与神经网络芯片性能代际提升之间的呼应关系已在第一章绪论图1-1与图1-2中概括给出（其中图1-2刻画训练算力随年份的指数增长趋势^[4]）。大模型在训练与推理阶段均表现出强烈的计算密集与访存密集特征。随序列长度、批次大小与模型深度的变化，计算与存储、通信的瓶颈会发生转移，这对周期级模拟与多核调度评估提出了明确需求。因此，将深度学习与大模型负载的特征纳入前置知识，便于后文理解模拟器刻画的层间依赖、数据量与阶段划分同真实负载的对应关系。

2.2.2 算子与模型形态的多样性

深度学习模型可视为由张量算子构成的计算图，不同算子在算术强度、访存形态与可融合性上差异显著。卷积神经网络（Convolutional Neural Network, CNN）类模型以卷积与全连接（GEMM）为主，计算图相对规整，层间数据量可预测。Transformer 类模型则引入自注意力、层归一化等算子，访存模式与序列长度和批次大小强相关。量化等方法在保证精度的前提下降低访存与计算量，是部署阶段常用的优化手段^[23]。GEMM、卷积等 MAC 密集算子计算量大、访存相对集中，在充足算力与带宽供给下更容易接近峰值吞吐。归约、归一化、激活与逐元素算子则往往带宽受限，其性能对片上和片外内存带宽更为敏感。批量大小、序列长度与特征维度等参数直接决定每层的张量体积与访存量。多核映射时需在算力均衡与数据搬运代价之间折中。多核神经网络芯片模拟器需要能够区分这类差异，以便在周期级模型中合理分配计算周期与访存延迟。

2.2.3 大模型推理与注意力、KV Cache 的访存特征

以 Transformer 为代表的基础模型在云端与端侧被广泛部署，其自注意力机制具有显著的长序列与 KV Cache 特征。朴素自注意力的计算与存储复杂度随序列长度 L 呈 $O(L^2)$ 增长，在长序列场景下输入输出与带宽成为主要瓶颈。FlashAttention 等工作通过分块计算与重计算权衡降低了片外访存与带宽压力^[1,10-11]。在大模型推理服务中，KV Cache 的容量与访问模式对内存碎片与带宽利用率影响巨大，PagedAttention 及 vLLM 等通过分页式 KV Cache 管理显著提升了吞吐^[12]。本文模拟器以调度器 IR 为输入，执行层仅依赖“工作负载序列 + 数据依赖 + 数据量”的统一任务表示，不区分上游模型是 CNN 还是 Transformer——二者在 IR 层面只是数据量与依赖拓扑不同的工作负载项。因此，模拟器的建模机制与瓶颈分析方法论对模型类型具有通用性。后文实验选择 ResNet 系列负载，原因有二。其一，当前可用于精度验证的“金标准”——Gemini-Compiler-IR 项目中的 ZeBu 硬件仿真踪迹——仅覆盖 ResNet34 与 ResNet50 的调度 IR 与对应执行踪迹。在缺少 Transformer 负载的 ZeBu 参考踪迹的条件下，以 ResNet 为基准可保证验证的公平性与可比性。其二，ResNet 兼具卷积与全连接等 MAC 密集算子，已能在实验中展示从计算受限到存储受限的瓶颈转换、DRAM 带宽墙、SRAM 端口带宽墙、NoC 争用与核数扩展上限等多核系统级关键现象。Transformer 负载在这些维度上表现为更大的 KV Cache 与更强的访存压力，但并不改变模拟器的建模机制与分解口径。待上游调度器支持 Transformer 类负载的 IR 生成后，可在不修改模拟器代码的前提下直接运行与分析（见第六章展望）。

2.3 周期级模拟方法与系统级建模

从模拟范式看，踪迹驱动方法以离线生成的执行踪迹回放为主，执行闭环较弱，难以自然反映因资源争用与反压导致的时序变化。执行驱动方法则以任务或指令驱动系统推进，能够体现排队、背压与依赖满足的连锁效应，更适合多核神经网络芯片的端到端分析与瓶颈归因^[24]。本文采用执行驱动的周期级模拟。模拟器按全局周期步进，每一拍可更新核心状态、NoC 报文与 DRAM 请求的进展，事件顺序与资源占用在时间轴上一致，从而在保证因果正确的前提下得到可重复的周期估计。以调度器输出的工作负载与依赖为驱动，在统一时间轴上推进各核心与 NoC、DRAM，使“依赖到齐—计算—写回”的因果与竞争行为在模拟中可复现。周期级精度意味着每个全局周期内各子系统的状态更新与事件顺序可被显式建模，从而能够捕捉 NoC 拥塞、DRAM 行缓冲冲突等对端到端延迟的非线性影响，为设计空间探索提供可信的定量依据。近期多核神经网络芯片模拟器（如 ONNXim）也强调对 NoC 与 DRAM 竞争的建模，以提升系统级瓶颈刻画能力^[7]，与本文思路一致。

2.4 硬件模拟与准确性验证参考：ZeBu

周期级模拟器的结果是否可信，需要与更接近真实硬件的参考进行对比。除流片实测外，基于硬件模拟的原型验证是常用参考手段。将 RTL 综合并部署到硬件模拟平台（如基于现场可编程门阵列（Field-Programmable Gate Array, FPGA）或专用模拟机的原型系统），在接近真实时钟与时序约束下运行，得到指令级或周期级的执行踪迹，可作为“金标准”用于校验模拟器的周期估计。

ZeBu 是业界常用的硬件模拟平台之一，能够将神经网络芯片等设计的 RTL 在模拟环境中运行，并输出每条指令的执行时间区间（即起始周期与结束周期）。在相同调度配置与相同输入 IR 的前提下，ZeBu 运行得到的踪迹可汇总为总执行周期（即各指令结束周期的最大值），与本文模拟器在相同 IR、相同核心数等配置下得到的端到端总周期进行对比。若二者相对误差在可接受范围内（如 5%–15%），则可为模拟器的准确性提供佐证。实践中常采用单核或与 ZeBu 相同的核心数配置进行对比，以保证调度与数据流一致、便于归因差异。本文第五章在开展微基准与真实负载实验时，将视需要引入 ZeBu 踪迹与模拟器结果的对比，以说明模拟器在周期级行为上与参考实现的一致性。本节对 ZeBu 的角色与定位做预先说明，为第五章中与 ZeBu 的对比实验提供背景。

2.5 关键工具底座：BookSim2 与 Ramulator2

本文模拟器在“高保真”模式下将片上网络与片外存储分别委托给 BookSim2 与 Ramulator2，以复用其成熟的周期级建模能力。本节简要说明二者在本文中的角色，为第

三、四章的接口设计与集成提供依据。

2.5.1 BookSim2: 周期级 NoC 模拟

BookSim (及其后续版本 BookSim2) 提供可配置的周期级 NoC 模拟, 支持多种拓扑 (如 Mesh、Torus)、路由算法 (如 XY、自适应路由)、虚拟通道 (Virtual Channel, VC) 数量与缓冲深度、以及分片与数据包的划分^[13]。Mesh 等规整拓扑便于随核心数扩展且路由规则简单。虚拟通道在同一物理链路上提供多条逻辑通道, 有助于缓解队头阻塞、提升吞吐。模拟按周期推进, 每周期内完成路由计算、VC 分配、交换仲裁与链路传输, 从而能够刻画多流并发时的排队与背压。通过配置拓扑与路由, 可研究拥塞、仲裁与尾延迟对端到端性能的影响。本文通过封装 BookSim2 作为片上网络接口的一种实现, 在保持与执行层解耦的前提下, 将多核产生的核间读请求与响应映射为 NoC 上的数据包流, 并接收其周期级投递结果以驱动依赖满足与状态推进。

2.5.2 Ramulator2: DRAM 时序与调度

主存方面, DRAM 的时序约束 (如 t_{RCD} 、 t_{RP} 、 t_{RAS})、Bank 并行度与调度策略 (如 FR-FCFS) 对有效带宽影响显著。其中 t_{RCD} 为行激活延迟, t_{RP} 为预充电时间, t_{RAS} 为行有效至预充电的最短间隔。行缓冲命中可大幅缩短访问延迟, 而 Bank 冲突与行关闭则引入额外周期。多核并发时排队与调度顺序会显著改变实际带宽利用率。Ramulator2 以模块化、可扩展的方式实现了多种 DRAM 标准 (如 DDR4) 的时序与调度建模, 支持通道数、Bank 组织、行缓冲与刷新等配置, 便于研究排队、行命中与冲突对多核并发访存的影响^[8]。本文通过封装 Ramulator2 作为片外存储接口的一种实现, 将核心发起的读写请求注入 Ramulator2, 并按模拟器配置的存储与核心时钟比进行多速率推进, 使 DRAM 行为与核心/NoC 时间轴对齐, 支撑端到端周期与存储瓶颈分析。

2.6 多核神经网络芯片编译与调度及调度器 IR

2.6.1 编译与调度的挑战与相关工作

在多核神经网络芯片上, 将深度神经网络 (DNN) 工作负载映射到各计算核心并确定层间/算子间的执行顺序与数据放置, 是编译与调度的核心问题。不同映射与调度策略会导致数量级级的性能与能效差异。

在编译与中间表示方面, TVM 等端到端优化编译器通过图级与算子级优化、学习式代价模型与多后端支持, 实现了从高层模型到 FPGA、专用集成电路 (Application-Specific Integrated Circuit, ASIC) 等多种加速器的性能可移植^[26]。MLIR 等多层中间表示基础设施为领域专用后端提供了可扩展的抽象与变换管道, 便于从高层图表示下译

到硬件相关指令^[15]。面向具体神经网络芯片的编译栈（如 Hexagon-MLIR）在此基础上实现了从模型到神经网络芯片指令的完整链路^[16]。在架构与映射评估方面，Timeloop 对 DNN 加速器的工作负载映射与性能评估提供了系统化方法，Accelergy 则从架构级对能效进行估计，二者常配合用于设计空间探索^[27-28]。MAESTRO 以数据为中心刻画 DNN 数据流与映射的复用、延迟与能耗，为快速解析评估提供了工具^[21]。

在层间与多核调度方面，SET^[25] 针对分片式加速器，对层间调度空间进行了形式化定义与系统探索，为各层计算资源的分配与执行顺序提供了可操作的搜索空间，避免了仅依赖启发式规则带来的次优解。Gemini^[6] 则面向多核架构，提出层流水空间映射编码，并结合图划分与模拟退火进行映射与架构的协同探索，其输出的调度方案以结构化 IR 形式描述，可直接驱动硬件模拟或周期级模拟器执行。图2-4给出了多核架构下典型的工作负载映射示意。模型各层被分配至不同的计算核心，通过核间通信实现层间流水与空间并行，从而最大化硬件利用率并降低端到端延迟。Tetris 等工作则从 3D 堆叠存储与数据放置角度探讨了可扩展的神经网络加速与带宽利用^[22]。上述工作分别从编译基础设施、架构—映射评估与层间/多核调度等角度，为多核神经网络芯片的编译调度与本文所采用的调度器 IR 提供了理论与工具基础。

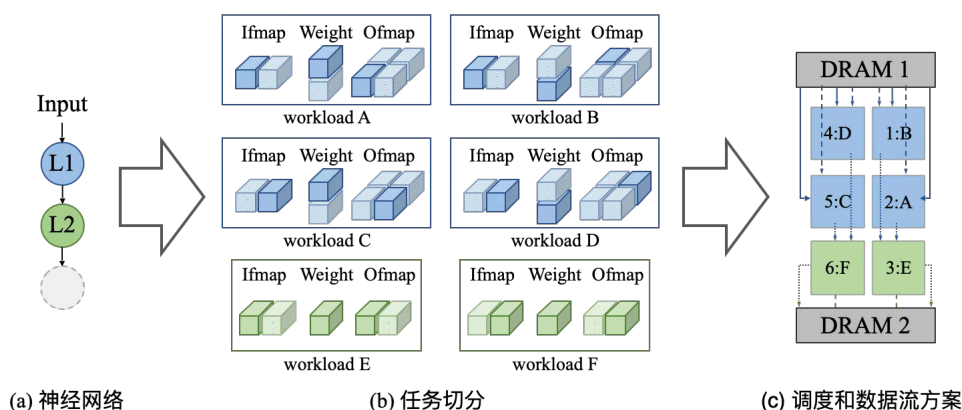


图 2-4 多核神经网络芯片上的层间流水与空间映射示意图

2.6.2 调度器如何生成 IR 及 IR 的形式与语义

本模拟器以 Gemini 调度器^[6]输出的 IR 为工作输入，因此有必要明确 IR 的来源、生成过程及其应包含的信息与语义。

IR 的来源与生成过程。调度器在完成层间/层内映射与执行顺序搜索后，将结果编码为结构化 IR 并写出为 JSON 文件。以 Gemini^[6] 为例：在给定核心网格与 SRAM 等约束下，通过层流水空间映射编码与模拟退火等搜索得到“每一层（或算子组）分配到哪些核心、以何种顺序执行、数据从何而来、写往何处”。随后将这些决策转化为 IR 中的核心网格规模、每核上的工作负载列表，以及各工作负载的输入依赖与输出描述。因

此, IR 本质上是“映射与调度结果”的可执行表示。模拟器或硬件后端只需按 IR 中的依赖与数据量推进, 无需再做映射决策。

IR 应包含的内容与语义。为驱动周期级模拟, IR 至少需表达以下几类信息。(1) 核心网格与全局约束: 如核心数量 (或 $x \times y$ 网格)、可选的全局 SRAM 容量上限等, 用于约束资源与规模。(2) 每核工作负载序列: 每个核心对应一个工作负载列表, 列表中的每一项表示“在该核上执行的一个计算单元”, 通常对应某一层或某一算子在某一数据分片上的执行。每项需包含算子类型、输出张量规模 (如输出特征图尺寸、权重尺寸等), 以便模拟器推算计算量与访存量。(3) 依赖关系: 每个工作负载的输入由输入缓冲 (或输入依赖) 描述, 每一项引用一个传输标识 (或等价标识), 表示该数据由哪一个生产者的哪一输出产生。模拟器据此建立“数据产生 $\leftrightarrow$ 传输/写回 $\leftrightarrow$ 被消费”的依赖链, 只有依赖到齐后该工作负载才能进入计算阶段。(4) 输出与消费者: 每个工作负载的输出描述给出其输出对应的传输标识, 供后续工作负载或 DRAM 写回引用。多消费者可共享同一传输标识, 由模拟器通过引用计数或等价机制管理驻留与释放。此外, 若调度器区分了来自 DRAM 的读与来自他核的读, IR 中可显式标注数据来源 (如 DRAM 读入、核间读), 以便模拟器分别走存储接口与 NoC 接口。综上, IR 的形式是“核心网格 + 每核工作负载列表 + 每工作负载的依赖与输出标识及数据量”。模拟器据此即可在统一周期轴下按依赖推进, 发起 NoC/DRAM 请求并统计周期与瓶颈。

2.6.3 调度器 IR 与本文模拟器的衔接

由上可见, Gemini 等框架输出的调度器 IR 以 JSON 形式承载上述核心网格、每核工作负载序列、各工作负载的输入依赖与输出描述及数据量等信息。每个工作负载对应某一层 (或算子) 在某一核心上的执行单元, 依赖关系与数据量在 IR 中显式给出, 便于模拟器按“依赖满足—计算—写回”的语义推进。本文模拟器即以此类 IR 为可执行的上游输入。本小节从 Gemini 与模拟器的分工与衔接、以及 Gemini-Compiler-IR 项目^①所提供的基准调度 IR 与 ZeBu 踪迹在第五章实验中的使用两方面, 说明调度器 IR 在本文研究中的角色与意义。

Gemini 与本文模拟器的关系。从功能分工看, Gemini 负责“映射与调度”的搜索与决策。在给定核心网格、SRAM 容量等约束下, 通过层流水空间映射编码与模拟退火等算法, 确定每一层分配到哪些核心、以何种顺序执行、数据从何而来与写往何处, 并将结果编码为结构化 JSON IR 写出。本文模拟器则不参与映射搜索, 仅作为“执行后端”。它以 Gemini 输出的 IR 为工作负载输入, 在统一周期轴上按依赖推进各核心的加载、计算与写回, 并同时推进 NoC 与 DRAM 的请求与响应, 从而得到端到端周期与瓶颈统计。二者在逻辑上形成“Gemini 生成调度 $\rightarrow$ IR 描述 $\rightarrow$ 模拟器执行并反馈周期与瓶颈”

^① 开源仓库: <https://github.com/SET-Scheduling-Project/Gemini-Compiler-IR>

的流水线。这一分工的意义在于：调度器专注于映射空间探索与决策，模拟器专注于在给定调度下对周期级行为与瓶颈的刻画。模拟器输出的周期与阶段分解既可验证某一调度方案的实际效果，也可作为后续映射搜索或架构参数设计提供定量依据，从而支撑“编译—架构”协同优化。

Gemini-Compiler-IR 项目与 ZeBu 踪迹在本文中的作用。Gemini-Compiler-IR 项目汇集与 Gemini/SET 工作流兼容的调度 IR，以及在同一调度下于 Synopsys ZeBu 硬件仿真平台得到的执行踪迹，被用作评估编译调度与周期级建模的参照^[6]。本文第五章的验证与瓶颈分析实验采用该项目发布的 JSON 调度文件（如 ResNet34、ResNet50 在单核、不同批大小等配置）作为模拟器输入，并采用同一批调度在 ZeBu 上测得的总周期作为对照。调度文件描述已确定的映射与依赖，ZeBu 踪迹则提供与之一致的指令级执行时间参考，二者同源，使模拟器输出可在相同调度、相同配置下与硬件仿真公平对比。本文模拟器前端将上述 JSON IR 解析为与格式无关的统一任务表示（含整体解析数据与各工作负载项），并在统一周期轴下与 NoC、DRAM 后端耦合推进，形成“调度器 IR $\leftrightarrow$ 解析 $\leftrightarrow$ 执行 $\leftrightarrow$ 统计”的链路。第三、四章将在此基础上分别介绍模拟器的整体架构与松耦合接口设计、以及可配置计算核心与全系统同步机制。

3 模拟器整体架构与松耦合接口设计

本章与第四章共同介绍本文模拟器的设计与实现。本章侧重整体架构、输入解析与对外接口，回答“有哪些模块、如何衔接、NoC/DRAM 以何种方式接入模拟器”。第四章侧重单核执行模型、NoC/DRAM 在运行时的集成方式与全系统同步机制，回答“核心如何按依赖推进、如何与接口交互、多时钟域如何对齐”。两章合起来形成从“结构”到“行为”的完整叙述。

3.1 总体架构概述

第二章已对多核神经网络芯片的架构特征、周期级模拟的基本范式，以及 Book-Sim2、Ramulator2、ZeBu 等第三方模拟与硬件模拟平台的能力与定位做了系统介绍。基于上述背景与本文研究目标，本章的整体架构设计需要同时满足三方面的工程约束：一是能够直接消费调度器生成的结构化 IR，从而把“修改映射—重新编译—重新评估”的迭代成本降到最低。二是能够在不同精度的 NoC/DRAM 建模之间灵活切换，以支撑“简单后端快速回归、全后端精细评估”的分层研究路径。三是能够在统一的时间轴上对齐核心、片上网络与片外存储三个子系统的推进过程，并输出可解释的阶段级统计以支持瓶颈归因与参数敏感性分析。下文的总体架构与后续章节的核心建模、实验验证均围绕这三条约束展开。

本文模拟器面向多核神经网络芯片的端到端周期级评估需求，采用“IR 驱动—统一任务抽象—周期推进—统计追踪”的总体思路。整体由配置与入口、IR 解析、模拟引擎、计算核心阵列以及 NoC/DRAM 接口五部分构成。配置与入口负责读取 JSON 形式的模拟配置与调度器 IR 路径，模拟器据此完成加载、初始化与运行。

设计目标与原则。架构设计遵循三项原则。(1) 以 IR 为唯一调度输入，使“改映射即改 IR 路径”，避免模拟器与上游调度器在输入格式上脱节。(2) 统一任务抽象，将不同格式的 JSON IR 解析为与格式无关的统一任务数据结构，执行层仅依赖该抽象，实现“内容与形式分离”。(3) 可替换的 NoC/DRAM 后端，通过抽象接口与配置项在简单延迟模型与周期级高保真后端之间切换，便于变量隔离与两档精度评估。

模块划分与职责边界。五部分职责如下。配置与入口：解析命令行与配置文件，提供 IR 路径、核心/NoC/DRAM/拓扑等参数，不参与周期推进。IR 解析：按格式将 JSON 转为统一任务表示，完成拓扑排序与引用计数等后处理，产出供执行层消费的统一任务数据。模拟引擎：持有拓扑与地址分配器，每周期按固定顺序推进各核心、收集报文、注入 NoC/DRAM、推进子系统、投递响应，并汇总统计。计算核心阵列：每核维护工作负载队列与状态机，产生读/写请求报文、消费到达的响应，不直接调用 NoC/DRAM

接口。NoC/DRAM 接口：对外提供“注入—推进—取回”的抽象，内部由配置选择简单或 BookSim2/Ramulator2 实现。模块间通过稳定的数据结构与接口协议衔接，便于单点扩展与测试。图3-1 给出了模拟器各模块的构成与数据流关系的整体示意，后续小节将据此依次展开各模块的设计决策与内部机制。

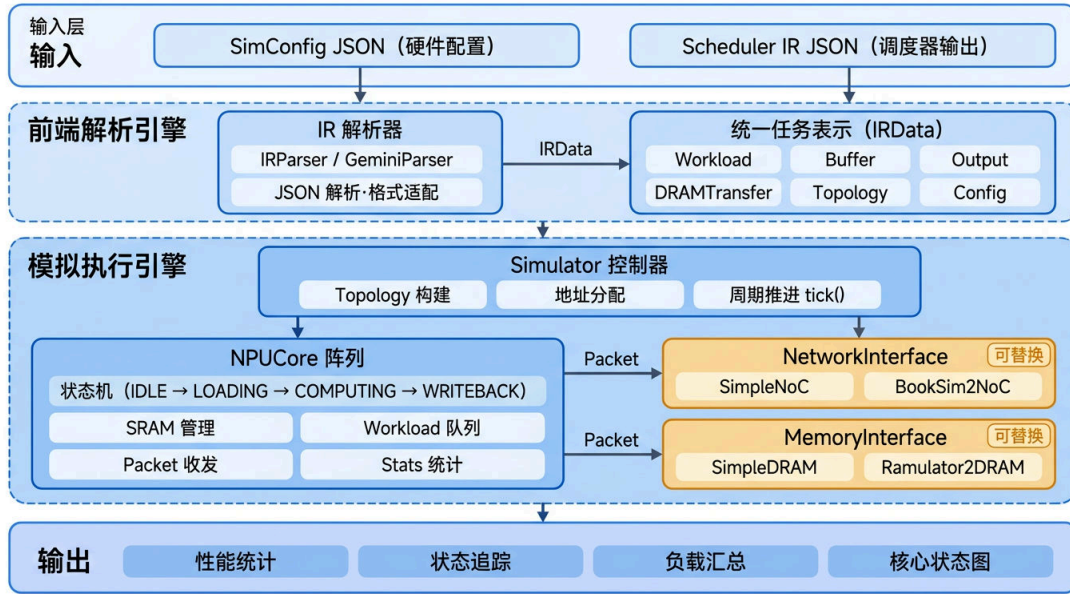


图 3-1 模拟器总体架构示意图

关键设计决策与权衡。上述模块切分背后有三项核心设计决策。(1)“IR 驱动”而非“踪迹驱动”。常见的加速器模拟器采用预先录制的访存或执行踪迹作为输入，适合在真实硬件上采样后离线重放。但对架构探索场景而言，每次修改映射或调度都需要重新录制踪迹，迭代成本高且难以在新架构上获得。本文以调度器直接生成的、带有显式依赖与数据流向的 IR 作为输入，使“改映射即改 IR”，在映射修改与参数扫描的场景下具有显著更低的迭代开销。(2)“执行层不直接操作拓扑与地址”。本文将拓扑解析（核心 ID 到 NoC 节点、DRAM 控制器路由）与地址分配（传输标识到物理地址）集中在引擎侧的辅助结构中，使计算核心只需以逻辑核心 ID 与传输标识为基本单位产生请求。这种分层有利于在后续支持更复杂的拓扑（如带 DRAM 控制器聚簇的非对称 Mesh）时，只需扩展拓扑与路由策略，而无需触动核心状态机。(3)“NoC/DRAM 走抽象接口而非直接调用第三方 API”。考虑到 BookSim2、Ramulator2 等第三方模拟器在调度粒度、时钟模型、完成通知机制上差异显著，本文在执行层与第三方模拟器之间引入统一的网络接口与存储接口，把差异封装在各自的封装器内部。这使得后续替换或新增子模块后端时，执行层逻辑完全不受影响。三项决策共同服务于同一目标：在保证端到端周期级精度的前提下，把“映射改动”“架构参数改动”与“子模块后端改动”三条迭代路径相互解耦。

配置与入口。用户通过命令行或配置文件指定 IR 路径与可选配置路径。配置为 JSON 格式，包含核心网格横/纵维度（可为 0 表示从 IR 补齐）、单核计算与 SRAM 参数、NoC 与 DRAM 参数（含后端类型、时钟比等）、拓扑（如 DRAM 控制器位置）及输出目录。用户可基于“同一 IR、不同配置”做对照实验。

拓扑构建与 DRAM 控制器部署。模拟器在初始化阶段根据核心网格维度与可选的 DRAM 控制器位置列表构建 Mesh 拓扑。计算核心按行优先顺序放置在网格上，行索引 i 、列索引 j 的核心映射到 NoC 节点 $i \times W + j$ (W 为网格宽度)。当配置中给出 DRAM 控制器坐标时，控制器作为独立节点挂在 Mesh 边缘或角落（可在配置文件中选择不同策略），网格宽高自动扩展以容纳所有节点。此时 DRAM 请求需经 NoC 发往控制器节点，响应沿 NoC 返回，NoC 流量中同时包含核间通信与 DRAM 访存两类数据包。若未配置控制器位置，则 DRAM 走直连通道绕过 NoC，核间与 DRAM 流量完全隔离。拓扑构建完成后会执行合法性校验，确保计算核心与 DRAM 控制器不占用同一 NoC 节点，否则抛出异常终止。

拓扑选型的考量。本文模拟器选择 Mesh 作为默认 NoC 拓扑，主要基于三方面考量。一是与工业实践对齐：当前主流多核神经网络芯片（如 Google TPU、NVDLA 多核版本、国内若干多核神经网络芯片）在物理版图上普遍采用规则的二维 Mesh 或类似的片上网络，Mesh 拓扑能够直接对应到实际神经网络芯片的版图布局，模拟结果对工程研发具有参考价值。二是路由规则简单且可解释：XY 路由等确定性路由在 Mesh 上无死锁、无活锁，路由结果可由核心坐标直接推导，便于在分析阶段手工核对核间通信路径，也便于第五章中结合核坐标与阶段占比进行热点归因。三是与第三方 NoC 模拟器的兼容性：BookSim2 原生支持 Mesh 并以之为最成熟的测试拓扑，相关配置项与验证路径最为完善。对于 Torus、Fat-Tree、Dragonfly 等更复杂的拓扑，虽然其对角径与带宽特性在超大规模系统中具有优势，但在当前多核神经网络芯片的规模（数核至数十核）下，Mesh 的直径和平均跳数尚未成为系统瓶颈，且更复杂拓扑的引入会加大调度器侧映射策略的复杂度。因此，本文将 Mesh 作为默认实现，同时在拓扑模块中保留接口扩展点，便于后续研究中替换为其他拓扑。

当多个 DRAM 控制器挂在 NoC 上时，需要决定每次 DRAM 请求发往哪个控制器。模拟器支持三种路由策略。（1）最近节点策略：按发起请求的核心与各控制器之间的曼哈顿距离选择最近者，适合规则 Mesh 拓扑下减少平均跳数。（2）地址哈希策略：以请求携带的物理地址对控制器数取模，将不同地址段分散到不同控制器，提高多通道并行度。（3）轮转策略：按请求的全局发出顺序依次轮转到各控制器，实现请求级的负载均衡。策略由配置中的路由策略字段指定，默认为最近节点。上述拓扑与路由策略使“核心 ID—NoC 节点—DRAM 控制器”三者之间的映射透明化。引擎与核心仅按逻辑核心 ID 和传输标识操作，拓扑负责完成物理节点的翻译与路由决策。

DRAM 地址分配。拓扑确定了 DRAM 请求的路由路径，而请求中携带的物理地址则由地址分配器在加载 IR 后统一分配。分配器遍历 IR 中的 DRAM 读/写表以及各工作负载的输入缓冲与输出传输，收集所有涉及 DRAM 的传输标识及其数据大小。对同一传输标识在多处出现时取最大字节数，避免因多消费者引用导致的重复或不一致。随后按传输标识排序，依次为每条传输分配对齐到缓存行（默认 64 字节）的连续地址段。排序保证分配结果与传输标识的自然序一致，使地址空间布局确定且可复现。核心在发起 DRAM 读/写请求时，引擎根据传输标识查询分配器填入物理地址，供 Ramulator2 等后端按 Bank 与行进行时序建模。该分配策略的优势在于简单、确定且对齐良好，有利于 DRAM 后端的行缓冲命中率。后续可扩展为按张量维度的分区布局，以支持更复杂的地址映射研究。

可配置的硬件与系统参数一览。模拟器当前支持的配置维度如下。具体语义与对周期的影响在第四章核心建模及本章 NoC/DRAM 小节中说明。(1) 核心规模与数据位宽：核心网格的横向与纵向维度、数据元素位宽。(2) 单核计算与片上存储：MAC 单元数与向量单元数（分别用于 MAC 密集算子与向量/逐元素算子的周期估算）、SRAM 容量与读/写带宽，并可区分 PE（卷积/全连接）与向量处理器（Vector Processor, VP）（用于池化/逐元素操作）的带宽。(3) 网络接口：最大未完成请求数、注入/弹出队列深度。(4) NoC：Flit 大小、跳数延迟与路由器延迟、后端类型（简单模式或 BookSim2）、时钟比、虚拟通道数与缓冲深度等。BookSim2 模式下还可指定外部配置文件路径。(5) DRAM：通道数、后端类型（简单模式或 Ramulator2）、时钟比、存储标准与组织（如 DDR4/HBM2）、时序与调度策略（如 FR-FCFS）、地址映射方式等。Ramulator2 模式下可指定外部 YAML 配置。(6) 拓扑：DRAM 控制器在 NoC 上的位置与路由策略（如最近节点策略）。上述参数均可通过 JSON 配置或命令行覆盖，便于在同一 IR 下进行设计空间探索与变量隔离实验。

数据流与初始化。模拟器启动后执行“加载 IR”。内部根据格式创建解析器（当前支持 Gemini 格式），将 JSON IR 解析为统一的任务数据，包含核心网格、每核工作负载列表、DRAM 读/写表等。若配置中未显式指定核心规模，则从解析结果中的核心网格横向与纵向维度补齐。若 IR 中带有调度器给出的 SRAM 容量需求，则据此自动调整每核 SRAM 容量约束。随后根据拓扑配置构建拓扑，用于将逻辑核心 ID 映射到 NoC 节点以及可选的 DRAM 控制器节点。地址分配器根据 IR 中的 DRAM 读/写表为每次传输分配物理地址，供后续 DRAM 请求携带。完成上述准备后，初始化核心：创建计算核心数组（每核绑定核心配置、SRAM 配置、网络接口配置与数据位宽），将各核工作负载列表按核注入各核心，并根据 NoC 与 DRAM 的后端类型分别实例化网络接口与存储接口（简单模式或 BookSim2/Ramulator2 封装）。

运行循环与每周期顺序。模拟器在“所有核心均已完成”为真前循环执行“单周期推

进”并递增全局周期。单次推进的顺序为：(1) 各计算核心按当前全局周期推进，产生待发送的 NoC 包或 DRAM 请求。(2) 引擎从各核心收集发出的报文，根据目的节点是逻辑 DRAM 还是他核，分别注入存储接口或网络接口。若拓扑将 DRAM 挂在 NoC 上，则 DRAM 请求先封装为发往 DRAM 控制器节点的 NoC 包。(3) 若有上一周期未注入的 DRAM 响应，则在本周期尝试注入 NoC。(4) 推进 NoC 与 DRAM 一个周期。(5) 从 NoC 按节点取回本周期到达的包并投递给对应核心或 DRAM 控制器。(6) 从 DRAM 取回本周期完成的响应并投递给核心（或经 NoC 再投递）。该顺序保证“先产生请求、再推进网络与存储、再投递响应”，依赖满足与状态更新在时间轴上因果一致。模拟结束后统计总周期、总工作负载数、NoC 包数、DRAM 读写次数等，并可导出状态追踪与工作负载汇总表供第五章分析使用。

模拟结束后的输出与统计。运行结束后输出三类结果。(1) 端到端统计：总周期、总工作负载数、NoC 注入/投递包数、DRAM 读/写请求与响应数，用于与 ZeBu 等参考对比。(2) 每核统计：各核加载、计算、写回、NoC 背压停滞等阶段周期数，用于阶段分解与瓶颈归因。(3) 可选的踪迹与汇总：启用踪迹选项时导出每核每周期状态序列与工作负载级起止周期汇总等文件。输出目录可由配置或命令行指定，支持时间戳子目录以避免覆盖。

3.2 前端 IR 解析模块设计：JSON 到统一任务表示

(1) IR 的含义与定义。

在展开解析器与格式细节之前，先明确本节所称 IR (Intermediate Representation, 中间表示) 在本模拟器中的含义与作用。本文中，IR 指调度器（或编译/映射工具链）输出的、描述“已调度任务如何在多核神经网络芯片上执行”的结构化数据。其内容通常包括：核心网格规模、各核上按依赖关系排列的工作负载列表、每项工作负载的算子类型与数据范围、核间与核—DRAM 之间的数据传输（含传输标识、源宿、大小等），以及可选的 SRAM 容量需求等。IR 不描述硬件微架构细节（如具体 NoC 路由、DRAM 时序）。它描述的是“做什么、在哪做、依赖谁”的逻辑视图。模拟器据此驱动周期级执行并统计性能。因此，IR 实质上是调度器与模拟器之间的接口约定。上游生成 IR，下游解析 IR 并转化为统一任务表示，二者通过 IR 的格式与语义解耦。

第二章已给出调度器 IR 的来源、生成过程及其应包含的形式与语义。本节结合本项目实现，说明模拟器前端如何消费该 IR：解析器接口、Gemini 格式的适配、以及统一任务表示与后处理流程。

解析器抽象与创建。为避免执行层与具体 IR 格式强耦合，本文定义解析器抽象。对外仅暴露“按路径解析并返回统一任务数据”的接口。根据格式字符串可创建具体实

现（当前支持 Gemini 格式）。后续若支持其他调度器输出格式，只需新增解析器并注册，模拟引擎与核心逻辑无需修改。

Gemini 格式与解析流程。Gemini 解析器读入 JSON 根节点后，先读取全局字段：核心网格横向与纵向维度、批切分参数。若存在“缓冲区容量”，则判定为调度器 IR 并记录所需 SRAM 容量。除上述保留键外，专用键表示 DRAM 段，其下为 DRAM 读/写表。其余数字键表示核心 ID，对应值为该核上的工作负载数组。每个工作负载的解析包括：工作负载 ID、层名、层类型（或调度器 IR 中的 PE/VP/DT 等），映射为统一的算子类型（见下段）。再读取输出张量/权重等范围与尺寸、可选的解析周期。输入依赖数组解析为“缓冲需求”（每项包含传输标识、来源列表：来源类型、源核、大小、传输标识）。输出解析为“输出传输”（含传输标识与目的列表：目的类型、目的核、目的工作负载、层名）。调度器 IR 与汇编器 IR 在部分字段上存在位与字节的差异，解析器据此做换算，保证写入工作负载结构的尺寸为字节单位，供执行层与统计使用。

目前支持的算子类型。模拟器在统一任务表示中支持以下算子类型，算子类型与执行层计算时间估算及 SRAM 带宽选择一致。卷积与全连接为 MAC 密集型，计算时间按矩阵阵列算力的上界式估算，SRAM 使用 PE 带宽。池化如 MaxPool、AveragePool、GlobalAveragePool 按输出元素数与向量单元吞吐的上界估算，使用 VP 带宽。逐元素运算如 Add、Relu、Mul、Sub 等，同样按向量单元吞吐的上界估算，使用 VP 带宽。点对点传输对应数据搬运或 DT 类层，估算与带宽模型同逐元素。自定义在未显式匹配时归入此类，可按解析周期或默认规则估算。解析时由 IR 的层类型（如 Gemini 的 PE/VP/DT）与层名前缀（如 Conv、Gemm、MaxPool、Relu）映射为上述类型。计算时间估算的详细公式与可配置的 MAC 单元数/向量单元数见 4.1.3 节。

后处理与统一表示。解析完各核工作负载后，解析器执行三项后处理。（1）按依赖对每核内工作负载拓扑排序，保证消费某传输标识的工作负载排在产生该传输的工作负载之后。（2）根据各传输标识被引用的次数补全输出的引用计数，供第四章核心在写回完成后做 SRAM 释放。（3）若调度器给出的核心 ID 非连续，则重映射为从 0 起的紧凑 ID，便于与核心数组下标一致。最终得到的统一任务数据包含核心网格规模、批切分、所需 SRAM 容量、各核工作负载列表、DRAM 读/写表。执行层仅依赖上述稳定数据结构，不直接依赖 JSON 键名。当上游 IR 格式演进时，仅需扩展或替换解析器实现，从而满足“内容与形式分离”的长期演进需求。

统一任务表示与执行层协议。解析器产出的统一任务表示在内存中由两级结构组成。顶层的“全局任务数据”持有：核心网格维度、批切分参数、可选的全局 SRAM 容量需求、按核心 ID 索引的工作负载列表数组，以及 DRAM 读/写表（每条记录含传输标识、数据量、源/宿等，供地址分配与 DRAM 请求构造）。每个“工作负载”项包含：工作负载 ID、层名、算子类型、输入缓冲列表（每项含传输标识、来源类型与源核/DRAM、

数据大小)、输出列表(每项含传输标识、目的列表与引用计数)、以及可选的解析周期或数据范围。执行层(第四章)仅通过“当前核的工作负载队列”“每项的缓冲需求与输出”“传输标识与引用计数”驱动状态机与 SRAM 管理,不依赖 JSON 键名或具体字段命名。因此新增或调整 IR 格式时,只需在解析器内完成映射与单位换算,统一表示与执行层协议保持不变。

IR 与配置的协同。模拟器在加载 IR 后,部分配置项可由 IR 内容补齐或覆盖,以降低用户配置负担并保证与调度结果一致。例如,若配置中核心网格的横/纵维度为 0 或未给出,则从 IR 的网格维度补齐。若 IR 中带有调度器给出的缓冲区容量需求,且大于配置中的 SRAM 容量,则自动将每核 SRAM 容量上调至满足需求,避免因配置过小导致逻辑错误。反之,计算资源(如 MAC 单元数、向量单元数)、NoC/DRAM 后端类型与参数等仍以配置为准,便于在同一 IR 上扫描不同硬件参数进行设计空间探索。

IR 解析中的典型语义对齐问题。尽管本节把解析器抽象得较为简洁,实际适配调度器 IR 的过程仍需处理若干语义层面的不一致问题,这些问题对于后续接入新的 IR 格式同样具有参考价值。第一,单位语义不一致:同一传输的数据量在调度器 IR 中可能以比特为单位、在汇编器 IR 中以字节为单位,也可能以张量元素个数给出。解析器需要结合数据位宽完成换算并统一到字节,否则 DRAM 地址分配与 SRAM 占用计算会出现系统性偏差。第二,算子类型命名空间不一致:调度器往往按其内部抽象(如 Gemini 的 PE/VP/DT)来标注工作负载,而执行层需要的是面向硬件的算子类型(卷积/全连接/池化/逐元素/点对点)以便选择对应的计算单元与 SRAM 带宽。解析器以“层类型+层名前缀”做联合映射,兼顾调度器自身类别与常见网络中的层命名惯例。第三,核心 ID 空间不紧凑:调度器为便于优化搜索,可能跳号或不使用全部核心(如仅在 2×2 子阵列上调度),解析器需在后处理中重映射为从 0 起的紧凑 ID,以便与核心数组下标一致并避免空索引。第四,传输引用计数的隐式依赖:同一输出传输可能被多个下游工作负载消费,而 IR 通常只显式给出“消费者列表”,解析器需据此推断引用计数,为第四章所述的“引用计数归零后释放 SRAM 占用”机制提供正确依据。这四类对齐工作封装在解析器内部,使得执行层与统一任务表示对 IR 格式的演进保持稳定,也为后续接入基于 MLIR 或自定义 JSON 的新 IR 格式提供了清晰的适配模板。

3.3 面向对象的松耦合接口设计

模拟引擎通过两个抽象接口与片上网络、片外存储交互:网络接口与存储接口。二者均以统一的“报文”结构为请求/响应单元,便于核间读请求与 DRAM 读/写请求共用同一数据形式。当拓扑将 DRAM 控制器挂在 NoC 上时,DRAM 流量可经 NoC 转发。采用抽象接口的核心动机在于:不同的子模块模拟器在请求粒度、数据表示、推进方式与完成通知机制上各不相同。例如 BookSim2 以 Flit 为最小调度粒度并由外部逐周期驱

动，Ramulator2 则以缓存行为请求单元并通过异步回调通知完成。若执行层直接调用各模拟器的原生应用程序接口（Application Programming Interface, API），则更换或新增后端时将牵连大量代码改动。通过在执行层与具体模拟器之间引入统一的接口层，将各模拟器的差异封装在接口实现内部，执行层仅依赖“注入—推进—取回”的抽象语义，从而在兼容现有模拟器的同时为接入其他 NoC 或 DRAM 模拟工具预留扩展空间。

图3-2从模块划分与可插拔后端的角度概括接口化松耦合架构带来的主要好处。

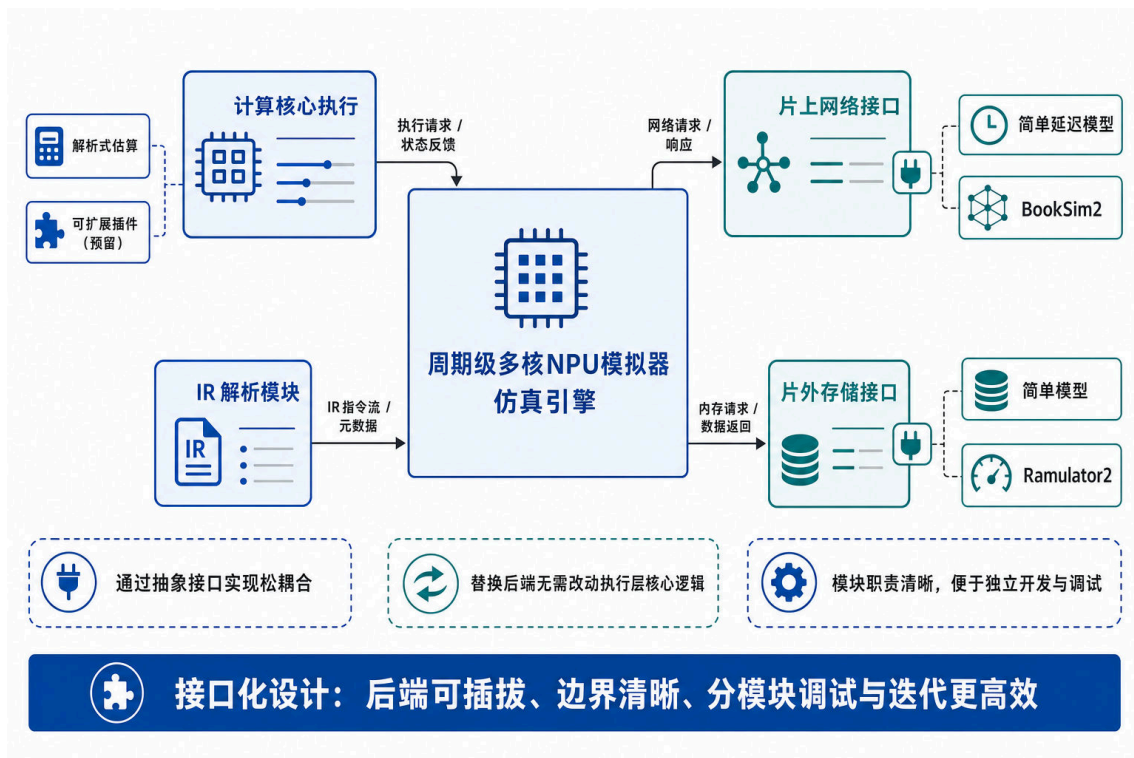


图 3-2 接口化松耦合架构示意：抽象接口便于接入不同 NoC/DRAM 等后端，模块边界清晰，利于独立开发与调试。

拓扑与地址分配在接口使用中的角色。在将报文注入 NoC 或 DRAM 之前，模拟引擎依赖两类辅助结构。拓扑根据配置的核心网格与可选的 DRAM 控制器位置，将逻辑核心 ID 映射为 NoC 节点 ID，并确定 DRAM 请求应发往的控制器节点或直连存储接口。当 DRAM 控制器挂在 NoC 上时，拓扑还负责“核心 → DRAM 控制器”的路径选择（如最近节点策略）。地址分配器在加载 IR 后根据 DRAM 读/写表为每条传输分配物理地址（按缓存行 Cacheline 对齐）。核心在发起 DRAM 读/写请求时由引擎将对应地址填入报文，供存储接口与 Ramulator2 等后端使用。拓扑与地址分配使“逻辑核心与传输标识”与“NoC 节点与 DRAM 地址”解耦。接口层仅需按报文中的目的节点与地址转发。

配置驱动的后端选择。NoC 与 DRAM 的具体实现由配置中的后端类型字段决定（可选简单模式、BookSim2 或 Ramulator2）。模拟引擎在初始化时通过工厂方法根据配置项创建对应的网络接口与存储接口实例。执行层仅持有抽象接口指针，不依赖具体

后端类型。切换后端时仅需修改配置文件并重新运行，无需改动核心与引擎代码，从而实现“两档精度”与变量隔离。

报文生命周期与引擎侧数据流。报文从产生到被消费的路径可简述如下。核心在“加载”或“写回”阶段将待发送的读/写请求放入核内发送队列。引擎在每周期推进后遍历各核，收集本周期产生的报文，根据目的（他核或 DRAM）填入地址等信息后注入网络接口或存储接口。NoC/DRAM 在后续周期内推进。完成传输或访存后，本周期到达的响应由引擎按节点或目的核取回并投递到对应核心的接收入口。核心在下一周期推进开头处理到达的报文（写 SRAM、标记就绪、回复远端读请求等）。该数据流保证“请求先于响应被处理、响应在后续周期对核心可见”，与第四章所述的因果顺序一致。

3.3.1 统一报文类型与接口约定

报文结构。报文包含：类型（读请求、读响应、写请求、写响应）、源/目的节点编号（目的为 DRAM 时使用保留编号）、传输标识、数据大小（字节）、DRAM 物理地址（由地址分配器分配）、注入周期/投递周期（用于简单模式与统计）。NoC 注入时可根据 flit 大小计算分片数（读请求仅头片，写请求为头片加载荷），供 BookSim2 等后端做分片传输。

网络接口。接口提供的能力包括：按节点数与 NoC 配置初始化。注入报文并返回是否成功。每周期推进网络一次。按节点取回本周期到达的包。查询某节点是否可注入（背压判断）。查询当前周期。根据配置中的后端类型可选择简单延迟模型或 BookSim2 封装。

存储接口。接口提供的能力包括：按 DRAM 配置初始化。提交读/写请求并返回是否接受。每周期推进 DRAM 一次。取回本周期完成的响应包。可选地统计子请求总数。根据配置中的后端类型可选择简单固定延迟模型或 Ramulator2 封装。

3.3.2 网络接口：简单模式与 BookSim2 封装

简单 NoC 模式。不依赖外部 NoC 模拟库，按曼哈顿距离与配置的跳数延迟、路由器延迟计算端到端延迟。注入时记录投递周期并放入在途列表，每周期仅递增内部周期，到达时间已到的包在本周期取回。支持 NoC 与全局周期的时钟比，将 NoC 周期按比例折算到全局周期，便于与多时钟域实验一致。该模式无排队、无拥塞，适合快速回归与趋势对比。

BookSim2 封装。封装第三方 BookSim2，在初始化时根据 NoC 配置生成或加载 BookSim2 配置文件（拓扑、虚拟通道数、缓冲深度、路由与分配延迟等），将逻辑节点 ID 映射为 BookSim2 子网与节点。注入时将报文转为 BookSim2 的 Flit 序列（头 flit 携带源/目的/传输标识/大小等元数据），按时钟比进行多速率推进。每周期查询各节点投

递的 Flit，按报文编号重组为完整报文后返回。执行层仅调用网络接口的“注入—推进—取回”能力，不依赖 BookSim2 的虚拟通道分配器、交换分配器等内部细节，从而实现松耦合与可替换性^[13]。

BookSim2 集成中的关键工程问题。将 BookSim2 作为模拟器的 NoC 后端，需要解决报文与 Flit 序列之间的双向转换以及节点编号映射两类问题。

在报文到 Flit 序列的转换方面，模拟器内部以一个完整的报文为传输单元，而 BookSim2 以 Flit 为最小调度粒度。注入时封装器根据报文的数据大小与配置的 Flit 字节数计算所需 Flit 数。读请求通常仅携带元信息，映射为单个头 Flit。读响应与写请求则需在头 Flit 之后附加若干载荷 Flit。头 Flit 中编码源节点、目的节点、报文编号、传输标识与数据大小等元信息，供接收端在 Flit 到达后重组为完整报文。封装器为每个注入的报文分配递增的报文编号，并在内部维护“报文编号到原始报文”的映射表。

在 Flit 到报文的重组方面，BookSim2 在每周期按节点投递已到达的 Flit。封装器遍历所有目的节点，收集本周期到达的 Flit，并按头 Flit 中的报文编号进行聚合。当某报文编号对应的所有 Flit 均已到达时，从映射表中取出原始报文并标记投递周期，返回给引擎供后续核心消费。尾 Flit 的到达标志着该报文传输完成。这一机制使 BookSim2 内部的分片传输对执行层完全透明。

在节点编号与 Mesh 拓扑的对齐方面，模拟器的拓扑按“核心网格 + 可选 DRAM 控制器节点”构建 Mesh 拓扑网络，节点按行优先编号。BookSim2 的 Mesh 拓扑同样按行优先编号，但其节点数、宽度与高度需要与模拟器拓扑完全一致。封装器在初始化时将模拟器的 Mesh 宽度与高度写入 BookSim2 配置，并确保注入与投递时使用相同的节点 ID 空间，避免因编号偏移导致路由错误。多速率时钟桥接的具体机制（累加器与条件推进）在第四章跨时钟域同步中详述。

3.3.3 存储接口：简单模式与 Ramulator2 封装

简单 DRAM 模式。固定读延迟与写延迟（如各 100 周期），提交请求时根据时钟比折算到全局周期，将响应放入在途列表并在投递周期到达时取回。无 Bank/行冲突、无排队，适合与高保真模式对照以隔离“时序与调度”的影响。

Ramulator2 封装。封装 Ramulator2，在初始化时根据 DRAM 配置加载或生成 Ramulator2 配置（通道数、Bank 组织、时序、调度策略等），并将报文的字节数与地址拆分为若干 cache-line 粒度子请求注入 Ramulator2 前端。Ramulator2 按自身时钟与时序推进，完成子请求后回调，封装器将同一传输标识的多个子请求聚合为一条读/写响应返回。支持 DRAM 时钟比，实现 DRAM 控制器与核心/NoC 的多速率推进（详见第四章）。执行层仅通过存储接口的“提交请求—推进—取回响应”交互，不依赖 Ramulator2 的内部 API，从而保证松耦合^[8]。

Ramulator2 集成中的关键工程问题。与 BookSim2 类似, Ramulator2 的集成涉及请求粒度适配、异步回调聚合与地址语义对齐三方面。

在请求粒度适配方面, 模拟器中的一次 DRAM 读或写对应 IR 中一个传输标识, 数据量通常为数 KB 至数 MB。而 Ramulator2 的前端以缓存行 (通常 64 字节) 为请求粒度。封装器在提交请求时将报文的物理地址与字节数拆分为多个缓存行对齐的子请求, 依次注入 Ramulator2 的请求队列。若某周期 Ramulator2 的请求队列已满, 则剩余子请求在后续周期重试。封装器维护每个传输标识的“已注入子请求数”与“已完成子请求数”两个计数器。

在异步回调与响应聚合方面, Ramulator2 在内部完成某个子请求后通过回调通知封装器。封装器在回调中递增对应传输标识的已完成计数。当某传输标识的所有子请求均已完成时, 封装器构造一条完整的读或写响应报文, 放入可取回的完成队列。引擎在当前周期结束的“取回响应”阶段获取该报文并投递给对应核心。由此, 多个缓存行级别的子请求对执行层表现为单次读/写操作的完成, 简化了核心与引擎的逻辑。

在地址语义对齐方面, 前文所述的地址分配器为每个传输标识分配了缓存行对齐的连续地址段。封装器在拆分子请求时从该基地址起按缓存行步进, 生成的每个子请求地址落入 Ramulator2 的地址空间后经其内部的地址映射 (通道—Bank 组—Bank—行—列) 转化为具体的 DRAM 位置。不同传输标识的地址段互不重叠, 避免了 Ramulator2 内部因地址冲突导致的非预期行缓冲行为。若配置中指定了多通道, 地址的高位或低位交织可将不同传输的子请求分散到不同通道, 提高并行度。

3.3.4 接口化设计的意义

从论文实验角度, 松耦合接口的意义体现在四点。(1) 变量隔离: 替换 NoC/DRAM 后端不改变模拟器推进逻辑与核心逻辑, 对比“简单模式与 BookSim2”或“简单模式与 Ramulator2”时仅改配置项, 减少混杂因素。(2) 两档精度: 可先用简单模式完成回归与趋势分析, 再用 BookSim2/Ramulator2 给出拥塞与时序敏感的结论。(3) 兼容异构子模块模拟器: 不同第三方模拟器在请求粒度 (Flit vs. 缓存行)、时钟模型 (外部逐周期驱动 vs. 自身时钟推进) 与完成通知方式 (同步查询 vs. 异步回调) 上差异显著。接口层将这些差异封装在各自的封装器实现内, 使执行层无需关注后端是 BookSim2、Ramulator2 还是其他模拟工具, 降低了集成新子模块模拟器的工程代价。(4) 可扩展性: 上述兼容机制也为后续引入新的 NoC 拓扑模型、DRAM 标准 (如 HBM3、LPDDR5) 或自定义统计插件提供了统一的扩展点, 使设计空间探索具备持续演进能力。

简单模式与高保真模式的适用场景。两种后端在精度与开销上各有侧重, 便于按实验目的选择。简单 NoC/DRAM 模式无排队、无拥塞, 延迟为距离或固定常数的理想化模型, 运行快、结果可复现。它适合快速回归 (如验证解析与状态机正确性)、趋势对

比（如扫描核心数、MAC 单元数时观察总周期随参数的大致走向），以及和高保真模式对照时作为基线，以隔离“仅因拥塞与时序”带来的差异。高保真模式周期级建模 NoC 的虚拟通道、仲裁与排队、DRAM 的 Bank/行缓冲与调度策略，能捕捉多核并发下的拥塞、背压与行冲突。它适合瓶颈归因（如第五章的阶段分解与 NoC 背压停滞/存储等待占比），以及设计空间探索中需要区分“计算墙、通信墙、存储墙”时的定量分析。

接口层支持的后续扩展方向。上述网络接口与存储接口在抽象语义层面并未绑定具体的 NoC 拓扑或 DRAM 标准，因此本架构在不修改执行层的前提下，可以向多个方向自然扩展。在 NoC 侧，可以接入支持新型拓扑的模拟器（如 Garnet、FlooNoC 等），用以评估 Torus、Fat-Tree、层次化 Mesh 等更复杂拓扑在大规模多核神经网络芯片下的通信特性。在 DRAM 侧，由于接口仅要求“注入读/写请求—每周期推进—取回完成响应”的统一语义，Ramulator2 之外的 DRAM 模拟器、以及 HBM3、HBM3E、LPDDR5 等更新的标准只需在封装器内完成时钟比与地址分配的对齐即可接入，便于在未来研究中对比不同存储介质与控制器调度策略对端到端性能的影响。在统计与可观测性方向，可以围绕统一报文结构插入额外的统计插件（如按张量类型分类的 NoC 流量统计、按 Bank 维度的 DRAM 行命中率分析），无需对核心或引擎逻辑做侵入式修改。这种基于接口的可扩展性，使本文模拟器不仅服务于本文的实验需求，也为后续围绕多核神经网络芯片的架构与系统研究提供了一个长期可演进的工程载体。

4 可配置计算核心（Core）与全系统同步机制

第三章已给出模拟器的总体架构、IR 解析流程以及 NoC/DRAM 的接口抽象（网络接口、存储接口）。本章在此基础上展开执行层的设计。具体包括：单核如何按依赖驱动推进、如何通过上述接口发起请求与接收响应。多核并发下核间通信与 NoC/DRAM 资源争用如何体现、如何保证因果一致与可复现。最后说明全局周期与多时钟域如何同步，从而完成从“接口协议”到“可运行系统”的闭环。

4.1 高通用性计算核心建模

4.1.1 建模目标与抽象边界

多核神经网络芯片端到端性能受计算、通信与存储共同影响。为在系统级研究中兼顾通用性与可解释性，本文以调度器 IR 中的“工作负载”为最小执行单元，对核心执行过程进行依赖驱动建模。核心在满足输入依赖后进入计算，计算完成后将输出写入本地 SRAM 并供其他核心拉取或写回 DRAM。该抽象能够覆盖 MAC 密集算子（卷积、全连接）与向量/逐元素算子（池化、激活等），并保留“等待依赖到齐”“注入队列满导致背压”“写回 DRAM 拖尾”等关键瓶颈形成机制，便于在第五章对端到端周期做阶段分解与归因。

从方法论上，本模拟器面向芯片早期设计阶段的架构参数与配置探索，以工作负载而非单条指令为最小建模粒度，这一取舍旨在“可解释性”与“仿真成本”之间取得平衡。指令级模拟能刻画更细的流水与调度行为，但多核、多工作负载场景下事件规模大，难以快速扫描配置。工作负载级抽象将调度器已决策的依赖与数据量作为输入，使端到端周期可归结为“计算—加载—写回”等阶段的叠加。由此便于回答“瓶颈在计算、通信还是存储”“增加某类资源能否线性缩短时间”等系统级问题，为架构探索与调度—硬件协同研究提供可操作的评估界面。而核内微架构与指令流水等细节被有意简化，以换取跨配置的快速扫描能力。

单核在模拟中的职责边界为：维护当前工作负载队列与当前下标、按状态机推进、在“加载”阶段向 NoC/DRAM 发起读请求并消费到达的读响应、在“写回”阶段向 DRAM 发起写请求并等待写响应、响应他核对本核 SRAM 中已就绪数据的读请求。核心不直接调用 NoC/DRAM 的注入或发请求接口，而是将待发送的报文放入内部发送队列，由模拟引擎在每周期推进后统一从各核收集报文并注入网络或存储。到达的包由引擎按节点或目的核投递到各核的接收入口，核心在下一周期推进开头先处理本周期到达的报文。由此保证“同一周期内先推进核心、再收集发送、再推进 NoC/DRAM、再投递”

的因果顺序。

4.1.2 状态机设计

本文将单核执行抽象为五态状态机：空闲 (IDLE)、加载 (LOADING)、计算 (COMPUTING)、写回 (WRITEBACK)、完成 (DONE)。状态转移在单周期推进内按当前状态分支执行。

空闲。若当前工作负载下标已达队列末尾，则转入“完成”并结束该核。否则取当前工作负载，根据其输入依赖中所有数据源的传输标识填入“待加载集合”。若待加载集合为空（无输入依赖），则直接转入“计算”并设置剩余计算周期。否则转入“加载”，等待依赖到齐。

加载。每周期尝试发起取数请求。对尚未请求过的传输标识，若已在本地 SRAM 且已就绪则从待加载集合移除。否则若来源为“他核”则构造发往该核的读请求，若来源为 DRAM 则构造发往 DRAM 的读请求，放入发送队列（若队列已满则统计“因 NoC 背压停滞”的周期数并本周期不再注入）。当“所有缓冲已加载”（即待加载集合为空）时，转入“计算”并设置剩余计算周期。

计算。每周期剩余计算周期减一。减至 0 时将当前工作负载的输出在 SRAM 中分配并标记为就绪（引用计数由 IR 后处理给出），并释放已消费的输入缓冲（对输入传输标识执行引用释放）。若存在输出目的为 DRAM，则转入“写回”并初始化待写回队列。否则当前工作负载完成，当前下标自增，回到“空闲”。

写回。每周期按 SRAM 读带宽从当前待写回块读取数据，凑满可发送的写请求后构造写请求（目的为 DRAM）放入发送队列，并将该传输标识加入“待写回完成集合”。当“所有写请求已发出且写响应均已收回”时，当前工作负载完成，回到“空闲”。

完成。该核不再参与推进。

每周期推进内，在状态分支之前先执行以下操作。处理本周期投递来的包。将 NoC/-DRAM 读响应数据按写带宽写入 SRAM 并标记就绪。对他核发来的读请求，若本核 SRAM 已就绪则回复读响应，否则继续挂起。图4-1 给出状态机示意。

4.1.3 算子计算时间估算

单个工作负载的计算周期由算子类型、张量尺寸与计算核心微架构三者共同决定。与精细的数据流仿真器不同，本模拟器面向芯片早期设计阶段的架构探索，核内采用上界式估算以换取跨配置扫描的速度与结果的可比性，不对流水填充、片上供数折损与分块对齐损失等微观效应做显式建模。

核内通用抽象。主流神经网络芯片的核内微架构在具体取舍上各异，但通常呈现出相似的功能组成：面向矩阵乘的规则阵列承担 MAC 密集计算，向量单元承担归约、

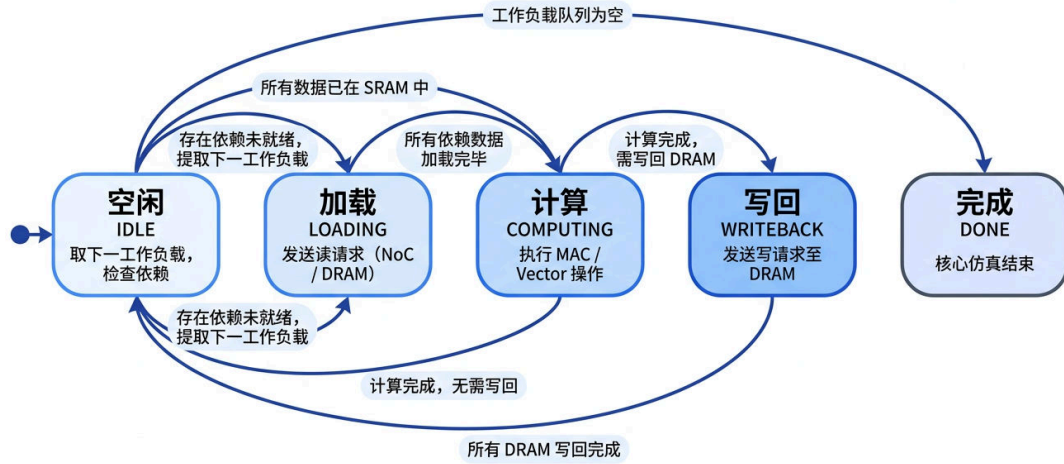


图 4-1 核心五态状态机示意图

逐元素与非线性计算，标量单元负责控制与地址生成，配以由编译器/调度器显式管理的片上 SRAM 以及负责片内/片外数据搬运的 DMA 引擎，代表性设计如 Google TPU 的 MXU^[18]、华为 Ascend Da Vinci 的 Cube Unit^[36-37]、NVDLA 的卷积核心^[19] 与 Eyeriss 的脉动阵列^[17] 等。本模拟器综合上述共性，在执行层将单个计算核心参数化为矩阵阵列（每周期 N_{MAC} 次 MAC，对应配置字段 `mac_units`）、向量单元（每周期 N_{VP} 个元素，对应 `vector_units`）、片上缓存（容量与读写带宽可配）与数据搬运通路四部分。具体产品在阵列拓扑、缓存层次与专用/共享策略上的差异，可通过参数扫描或子模块替换表达，但不作为核内精细建模的对象。

基于朴素数据流的计算时间估算。本模拟器按核内微架构的尺寸，对算子做朴素的数据流模拟：MAC 密集算子（卷积、全连接）按矩阵阵列每周期 N_{MAC} 次 MAC 的吞吐沿算子各维度逐 tile 累积阵列占用周期；向量类算子（池化、逐元素等）按向量单元每周期 N_{VP} 个元素的吞吐累积；数据搬运类工作负载按搬运带宽估算。当算子的张量维度与阵列或向量单元尺寸不整除时，按向上取整记入完整阵列周期，从而显式反映维度一阵列不对齐造成的尾效应。张量尺寸信息从 IR 中各工作负载的输出范围与权重尺寸字段解析获得，具体维度展开次序与分块策略不作为本文核内建模的对象。

该估算仅使用微架构的若干尺寸参数（ N_{MAC} 、 N_{VP} 、SRAM 容量与带宽等），不建模流水填充与排空、片上供数折损以及阵列内部利用率损失等二阶效应，属于计算时间的上界式估算：在卷积主干等计算密集层上接近真实，在小批量 GEMM、含 Softmax/LayerNorm 等多 pass 归一化的场景下偏乐观。片上缓存的供数约束不在本估算中体现，而是经由状态机的加载阶段按 SRAM 读写带宽与背压机制（见后文本地 SRAM 建模小

节) 在系统级时间轴上显式刻画, 从而保证“供数受限”在端到端周期中仍可观测。

取舍与扩展空间。上述核内简化是面向早期架构探索的有意取舍: 核内的分块策略、循环顺序、数据驻留方式以及流水填充等因素在同一算子上可带来数倍的执行周期差异, 其精细建模本身是神经网络芯片研究中独立的课题^[17,21,27]。本模拟器将计算周期的产生过程实现为可替换的接口, 与核心状态机、NoC/DRAM 推进等系统级推进逻辑解耦。后续可在不改动系统级仿真框架的前提下, 将 4.1.3 节的上界式估算替换为含利用率折损、片上供数约束与流水填充的完整模型, 或接入面向特定微架构的数据流仿真器 (如 SCALE-Sim、MAESTRO、Timeloop 等^[20-21,27]), 从而在保持系统级评估能力的前提下按需提升核内精度。

4.1.4 本地 SRAM 建模与生命周期

本地 SRAM 按“容量 (字节) + 读/写带宽 (字节每周期)”建模, 内部以传输标识为键维护条目, 每项包含大小、数据类型 (输入特征/权重/输出特征)、是否已就绪、引用计数 (仍在引用该数据的消费者数)。分配时在容量允许下登记条目并累加已用容量。标记就绪将对应条目置为可消费。释放引用将引用计数减一, 减至 0 时回收容量。这样, 多消费者共享同一输出时, 该输出在 SRAM 中保留至最后一个消费者释放引用, 与第二章所述 IR 语义及解析器的引用计数后处理一致。

SRAM 拉取策略。第四章后文所述核间与 DRAM 的拉取式数据流, 在“网络与存储接口”层面体现为消费者发起读请求、数据以读响应形式到达。在本地 SRAM 层面, 本文采用与之衔接的到达后再占位、就绪后才可消费的策略, 可概括为三点。(1) 请求不等于已入 SRAM。核心在“加载”阶段发出读请求, 仅表示向对端或 DRAM 索要数据, 并不在本地预先为整块数据预留可消费条目。数据只有在读响应到达并被核心处理之后, 才进入本地 SRAM 管理流程。(2) 响应到达后的分配与写入。读响应到达时, 仿真器先按当前工作负载对该输入的引用关系确定引用计数, 并尝试为对应传输标识执行容量分配。若当前剩余容量不足, 则将该响应暂存于重试队列, 在后续周期中随其他数据释放或配置变化再次尝试分配, 避免因瞬时峰值误判为永久失败。分配成功后, 将本次加载的字节数记入“待写入 SRAM 的剩余量”, 并在每个周期按有效写带宽 (见下) 消耗剩余量。当某传输标识对应的剩余量减至零时, 将该条目标记为就绪, 并从“待加载集合”中移除。由此, 网络/存储侧完成传输与片上写入带宽填满本地 SRAM 并置就绪在时间上可分步刻画, 便于区分“远端延迟”与“片上写口瓶颈”。(3) 核间读对生产者 SRAM 的拉取。消费者向生产者发起的读请求, 仅在生产者本地 SRAM 中该传输标识已就绪时立即回复读响应。否则请求挂起, 待生产者侧计算完成并将输出写入本地 SRAM 并标记就绪后再回复。生产者侧在成功向消费者发送读响应时, 按语义对相应条目做引用释放。这与 (1) (2) 共同构成“谁消费、谁拉取、何时在本地变为可消

费”的闭环。

带宽与类型分档。有效写带宽与有效读带宽可根据当前工作负载类型在 PE 与 VP 两档配置间选取（例如卷积、全连接走 PE 档带宽，其余走 VP 档），与配置中统一写带宽字段并存。若将某档带宽配置为 0，则仿真器退化为“当拍内完成整块逻辑写入并标记就绪”，用于与带宽受限模式对照。每周期更新 SRAM 统计中的峰值占用，供第五章分析 SRAM 占用与映射可行性。峰值占用与引用计数语义的结合，使得“某调度在给定 SRAM 容量下是否可行”“若缩小容量何处会成为瓶颈”等问题可在仿真中直接观测，为映射与缓冲配置的可行性研究提供定量支撑。

图4-2 给出上述 SRAM 管理模型的整体结构与数据生命周期示意。左侧为数据来源（他核经 NoC、DRAM 经存储接口），中部为当前计算核心内的本地 SRAM 管理（包括容量分配、条目表与写带宽分档），右侧为数据去向（消费者核心经 NoC、DRAM 写回）。下方的生命周期流程展示了数据从分配、写入、就绪、消费到回收的完整过程，其中多消费者共享的数据在 SRAM 中保留至最后一个消费者释放引用。

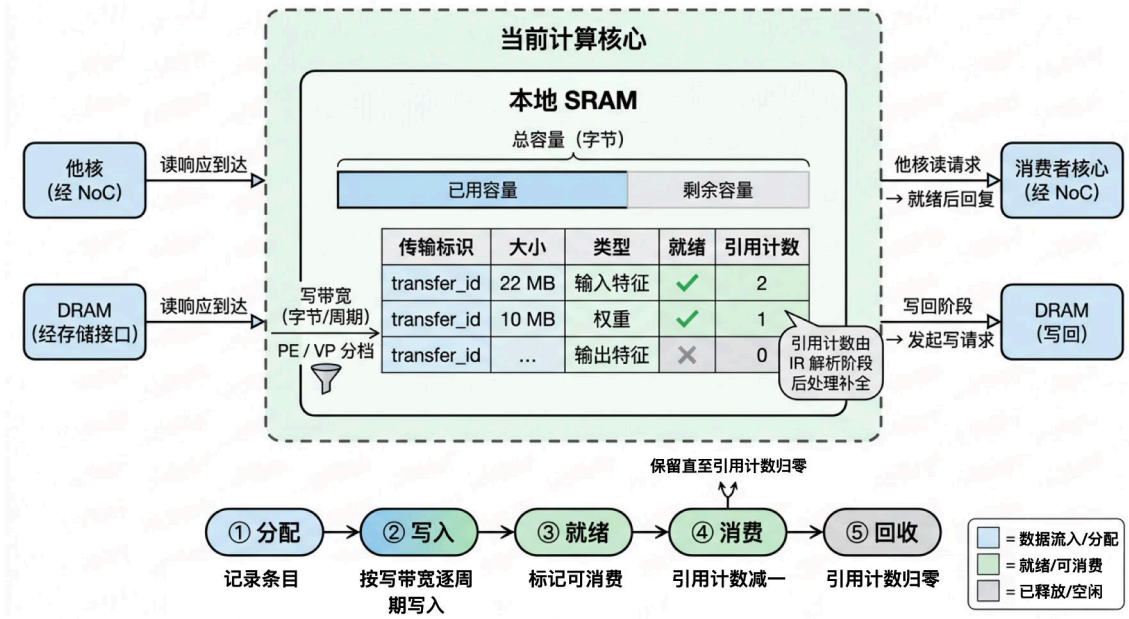


图 4-2 本地 SRAM 管理模型与数据生命周期示意图

4.1.5 统计输出与可观测性

执行层在每核、每周期上的行为最终体现为可导出的统计与踪迹，这些输出是连接“设计”与“分析”的桥梁。仿真结束后，每核可汇总得到在空闲、加载、计算、写回及因 NoC 背压停滞各阶段的周期数，以及完成的工作负载数、加载/写回的数据量、SRAM 峰值占用等。若开启状态追踪，还可按周期记录各核的状态转移序列（如从加载转入计算、从计算转入写回等），进而生成按核、按时间排列的阶段甘特图。上述统计与踪迹

不改变仿真语义, 但为端到端周期的阶段分解、瓶颈归因以及“热点核”识别提供了统一口径。第五章将基于这些可观测量, 对真实负载进行“计算—访存—NoC 背压”的占比分解, 并对多核场景下的工作负载时间线进行可视化, 从而在系统级回答“时间花在哪里”“哪些核或哪些阶段主导了总周期”等问题。

4.2 NoC 与 DRAM 子系统的集成机制

第三章定义的网络接口与存储接口为执行层提供了统一的注入、推进与响应提取能力。本节说明模拟引擎与核心如何在使用这两类接口的前提下, 将核间通信与 DRAM 访问纳入统一模型。

拉取式数据流与报文统一。核间数据采用拉取式 (Pull-Based) 数据流。消费者核心在“加载”阶段根据缓冲需求中的来源列表, 向生产者核心 (来源类型为“他核”时按源核 ID) 或 DRAM (来源类型为 DRAM 时) 发起读请求。生产者核心在处理到达报文时收到读请求。若本地 SRAM 中该传输标识已就绪, 则构造读响应放入发送队列。否则将请求加入“待回复的远端读请求”列表, 在后续周期中当数据就绪且发送队列未满时再回复。DRAM 读请求由引擎根据地址分配器填入物理地址后交给存储接口。写请求同样目的为 DRAM, 由引擎注入 DRAM, DRAM 完成后再以写响应经引擎投递回核心, 核心在处理到达报文时从“待写回完成集合”移除对应传输标识。所有报文均使用统一的报文结构 (类型、源/目的、传输标识、数据大小、地址等), 便于 NoC 与 DRAM 共用同一抽象^[8,13]。

从研究角度, 拉取式数据流与统一报文使“核间读请求—响应”与“DRAM 读/写请求—响应”在模拟中的处理路径一致, 便于在对比实验中有意隔离变量。例如比较“仅 DRAM 依赖”与“含核间依赖”的负载时, 可明确区分 NoC 流量与 DRAM 流量的贡献, 为第五章中微基准设计与后端差异分析提供清晰的语义基础。

引擎侧收集与投递。在单周期推进中, 各核心推进后, 引擎遍历各核的待发送报文。若目的为 DRAM, 则用地址分配器填入物理地址, 再根据拓扑决定是经 NoC 发往 DRAM 控制器节点还是直接交给存储接口。若目的为他核, 则必要时将源/目的转为 NoC 节点 ID 后注入网络接口。投递 NoC 包时按节点取回本周期到达的包并交给对应核心的接收入口 (或交给 DRAM 控制器节点再转发给存储接口)。投递 DRAM 响应时从存储接口取回本周期完成的响应, 根据目的投递给对应核心 (若 DRAM 挂在 NoC 上, 则响应先经 NoC 再在下一周期投递)。拓扑负责核心 ID 与 NoC 节点 ID 的映射、DRAM 控制器节点 ID 以及多控制器时的路由策略, 从而支持“仅计算核挂 NoC”与“NoC 上挂 DRAM 控制器”两种部署方式。

背压与队列约束。核心侧发送队列容量由网络接口配置限定。当队列满时, 新发出的请求无法入队, 本周统计“因 NoC 背压停滞”的周期数, 请求在下一周期重试。NoC

侧“是否可注入”在 BookSim2 封装中反映注入缓冲是否满，引擎可将无法注入的 DRAM 响应缓存在待注入队列并在后续周期重试注入，从而形成背压传递，使端到端周期能够反映拥塞与排队效应。

4.2.1 多核并发与资源争用

对于数据流驱动的多核神经网络芯片，同一全局周期内多核并行推进，核间通信与共享资源的争用会直接影响端到端周期与瓶颈分布，仅讨论单核状态机不足以刻画系统行为。本小节说明多核并发在模拟中的体现、可能出现的竞态与争用现象，以及本文如何通过确定的推进顺序与资源建模来刻画并复现这些效应。

多核并发的体现。在单周期推进中，引擎先对各核心依次执行一次推进，再从所有核心收集本周期发出的包并注入 NoC/DRAM。因此，每一全局周期内，多个核心可能同时处于“加载”（向不同生产者或 DRAM 发起读请求）、“计算”或“写回”。它们产生的请求在同一周期或相邻周期内集中进入 NoC 与 DRAM 的队列，形成对共享链路与存储资源的并发访问。依赖关系由 IR 中的传输标识与各核的“待加载集合”保证。消费者只有在收到对应读响应并标记就绪后才会离开“加载”，但“何时收到”取决于 NoC 的投递延迟与 DRAM 的排队与调度，二者均受多核并发流量影响。

竞态与争用现象。（1）NoC 争用。多核同时向 NoC 注入时，同一链路或路由器上的多包需仲裁与排队。BookSim2 的虚拟通道分配、交换分配与路由延迟会随流量增大而放大尾延迟，部分核心的响应包可能因拥塞晚于“理想延迟”到达，从而拉长其“加载”阶段并统计到更多“因 NoC 背压停滞”周期。（2）DRAM 争用。多核的读/写请求进入同一 DRAM 控制器后，Bank/行缓冲与调度器（如 FR-FCFS）会引入排队与行冲突。Ramulator2 按时序与调度策略推进，同一周期内完成的响应数受通道与 Bank 并行度限制，部分请求的响应会延后完成，体现为消费者核心在“加载”或“写回”阶段等待更久。（3）生产者侧背压。当消费者向生产者发起读请求时，生产者需在数据就绪后回复读响应。若生产者本周期发送队列已满（因其他响应或本核写回占用），则无法注入该响应，请求在“待回复的远端读请求”中挂起，消费者端待加载集合无法清空，形成跨核的背压链。上述现象在真实多核数据流执行中均存在，本文通过“多核每周期同时推进 + 统一收集注入 + NoC/DRAM 周期级建模”将其显式刻画为可观测的周期与阶段占比。

确定性顺序与可复现性。模拟器内部不引入随机调度。同一周期内核心按数组下标顺序推进，收集报文按相同顺序遍历并注入，NoC/DRAM 的仲裁与调度由配置与请求到达顺序确定。因此，多核并发下的“谁先被服务”由确定的规则与数据依赖决定，不存在未定义的竞态。同一配置与同一 IR 下，总周期与各核各阶段占比可完全复现。争用与排队体现在“同一资源被多核同时使用时，部分请求被延迟”的客观结果上，而非模拟器实现的不确定性。

死锁与核间同步。在拉取式数据流与有限发送队列的约束下，多核推进可能陷入死锁。若干核心均在“加载”或“写回”阶段等待来自他核或 DRAM 的响应，而对方因自身队列已满或尚未产生数据无法注入响应，形成相互等待、全局不再推进的局面。死锁的成因可从两方面理解。(1) 依赖与资源环。若调度 IR 中存在跨核的依赖环（例如核心 A 的某工作负载依赖核心 B 的输出，核心 B 的某工作负载又依赖核心 A 的输出，且二者在时间上重叠或请求顺序导致互相阻塞），则在无额外同步或请求排序约束时，可能出现“A 等 B、B 等 A”的循环等待。IR 本身描述的是数据依赖关系，调度器在生成 IR 时通常假定执行按某种顺序或缓冲容量充足，但并未在 IR 中显式编码“防死锁”的调度约束，因此 IR 在语义上并不保证无环或可运行性。(2) 队列与背压的级联。当生产者核心的发送队列已满时，无法向消费者回复读响应，消费者会一直停留在“加载”并可能持续发起重试。若 NoC 或 DRAM 的注入侧也因背压无法接受新包，则请求与响应在多个节点上形成阻塞链，在极端情况下可能演变为系统级停滞。

图4-3 给出多核并发下请求汇聚与资源争用的示意。多核在各自 LOADING/WRITEBACK 中产生的请求经引擎统一注入 NoC/DRAM，NoC 与 DRAM 在共享队列与仲裁下推进，响应再经投递回到各核，部分核心因排队或背压而在 LOADING/WRITEBACK 上停留更久，从而在端到端周期与阶段分解中体现核间通信与共享资源的影响。争用与背压在模拟中并不依赖随机调度，而是由确定的推进顺序与请求到达顺序产生。因此“因 NoC 背压停滞”的周期数、各核在加载/写回阶段的占比等均可稳定复现，并可作为第五章中多核瓶颈分解与热点核分析的数据基础。当负载从单核扩展到多核时，非零的 NoC 背压占比与各核阶段分布的差异，恰好用于通信瓶颈和热点核等设计问题。

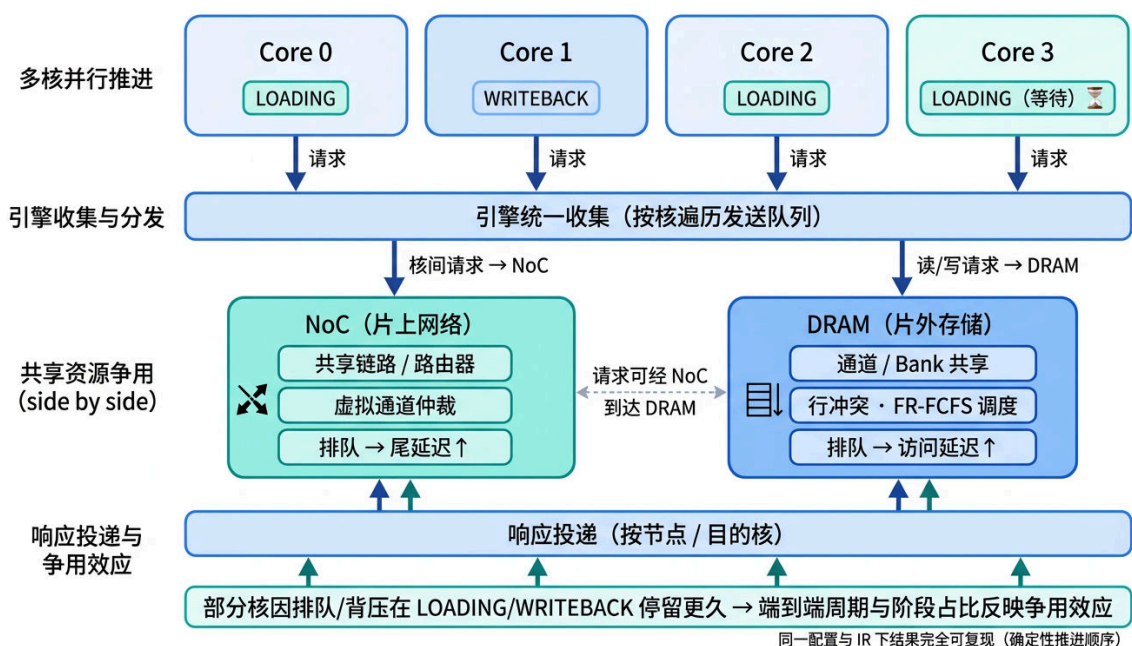


图 4-3 多核并发下请求汇聚与 NoC/DRAM 争用示意图

4.3 全局事件驱动与跨时钟域同步

模拟器以全局周期为唯一时间基准。全局周期计数器在每次“单周期推进”时递增一次，同一推进内各模块的顺序固定为“先核心、再收集注入、再 NoC/DRAM 推进、再投递”，从而保证请求先于响应、响应在后续周期被核心消费的因果关系。然而，当核心、NoC 与 DRAM 控制器运行在不同时钟域时，若仍按“每全局周期各子系统均推进一周期”的假设建模，既无法反映真实频率差异，也可能在因果顺序与周期统计上产生歧义。本节先说明跨时钟域模拟面临的主要问题，再介绍本文的设计与实现，最后用图示归纳多速率推进的时序关系及可达效果。

4.3.1 跨时钟域模拟面临的问题

(1) 时间基准不统一。真实芯片中，计算核心、NoC 路由器与 DRAM 控制器往往由不同 PLL 或时钟域驱动，三者频率可能为 1 GHz / 500 MHz / 266 MHz 等不同组合。若模拟器仅使用“单一周期计数器”且每拍让所有子系统各推进一周期，则等价于假设三者同频，无法区分“DRAM 控制器更慢”带来的排队与带宽折减。若改为为每个子系统维护独立时钟并异步推进，则事件顺序与“谁先谁后”的判定会变得复杂，且请求在“核心周期 t ”发出、响应在“DRAM 周期 t' ”完成时，需要明确 t 与 t' 的对应关系才能正确投递响应并统计端到端延迟。

(2) 因果一致与响应可见性。请求在某一时刻注入 NoC 或 DRAM 后，响应应在“该子系统已处理该请求并完成传输/访问”之后才对核心可见。若子系统的“内部周期”与“全局周期”不一致，则必须约定响应何时被认为“已完成”，是以全局周期为刻度还是以子系统自身周期为刻度。若约定不当，可能出现“响应在逻辑上早于请求被处理”的因果错误，或同一全局周期内事件顺序依赖实现细节而难以复现。

(3) 周期统计与带宽语义。端到端总周期通常以核心或系统主时钟为统计口径。当 DRAM 以较低频率运行时，其“每 DRAM 周期处理能力”不变，但“每全局周期平均处理能力”会按频率比下降。本模拟器在保持因果正确的前提下，让 DRAM 的等效带宽与“每全局周期可完成的请求数”随时钟比合理变化，以便扫描时钟比时观察到“内存变慢 → 端到端周期增加”的预期趋势。

4.3.2 本文的设计：全局周期与多速率推进

本文采用“单一全局周期 + 多速率推进”的策略，在不动引擎调用顺序的前提下，让 NoC 与 DRAM 的“等效频率”可配置。

统一时间基准。以全局周期计数器为唯一时间轴，所有“何时发生”的判定均以该周期为刻度。核心每全局周期推进一次。NoC 与 DRAM 的“推进”由引擎在每全局周期调

用一次，但接口内部可根据“时钟比”决定本全局周期内子系统是否推进、以及推进几次。

时钟比语义。配置项“NoC 时钟比”与“DRAM 时钟比”的约定为：时钟比 = 核心频率 / 子系统频率。因此时钟比 = 1 表示子系统与核心同频（每全局周期推进一周期）。时钟比 = 2 表示子系统以核心一半频率运行（每 2 个全局周期推进 1 周期）。时钟比 < 1 表示子系统频率高于核心（每全局周期可能推进多次，本项目通过累加器的累加与扣减实现）。这样，配置与真实“核心 1 GHz、DRAM 500 MHz”的对应关系为 DRAM 时钟比 = 2，便于与数据手册或 RTL 时钟设定对齐。

累加器实现。BookSim2 封装与 Ramulator2 封装内维护浮点累加器。每次被引擎调用时（即每全局周期一次）累加 1。当累加值不小于时钟比时，执行一次内部网络或存储的推进，并扣减时钟比（若时钟比大于 1，可用循环保证不漏推进）。因此，在任意一段全局周期数 G 内，NoC/DRAM 的“等效推进次数”为 $G/\text{时钟比}$ （取整由累加器自然体现）。既满足因果（请求先进入队列，随后仅在子系统推进时被处理），又使响应完成时间随时钟比增大而延后，从而在端到端周期上体现“内存更慢”的效果。

4.3.3 时序示意与效果

图4-4 与表4-1 给出在 DRAM 时钟比为 2 时，连续若干全局周期内核心、NoC 与 DRAM 的推进关系。核心每全局周期推进一次。DRAM 仅在累加值达到时钟比时推进，因此第 1、3、5 拍才各推进一次，等效于“每 2 个全局周期 1 个 DRAM 周期”。请求在某一全局周期注入 DRAM 后，需等待 DRAM 内部经过若干次推进才会在“取回响应”时返回。由于 DRAM 推进频率降低，同一请求的“从注入到响应可见”的全局周期数会随时钟比增大而增加，从而在总周期与瓶颈分析中体现跨时钟域影响。

全局周期（唯一时间轴）： $g = 0 \quad 1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6 \quad \dots$
核心 / NoC（时钟比=1）：每拍推进一次。
DRAM（时钟比=2）：仅在累加值 ≥ 2 时推进，即 $g = 1, 3, 5, \dots$ 各推进 1 次。等效“每 2 全局周期 1 DRAM 周期”。
累加器变化示例： $g = 0$ 时 $+1 \rightarrow 1$ ， $1 < 2$ 不推进。 $g = 1$ 时 $+1 \rightarrow 2$ ， $2 \geq 2$ 推进并 $-2 \rightarrow 0$ 。依此类推。

图 4-4 跨时钟域多速率推进时序示意（DRAM 时钟比 = 2 时，每 2 个全局周期 DRAM 推进 1 次）

效果小结。（1）因果一致。请求在某一全局周期被提交或注入后，仅在后续全局周期中、当子系统内部已推进并完成该请求时，响应才会在“取回响应”或“按节点取回包”时出现并投递给核心，不会出现“响应早于请求”。（2）可复现。同一配置与同一 IR 下，总周期与各阶段占比由确定的时钟比与累加器逻辑决定，无随机性或实现依赖。（3）可

表 4-1 多速率推进示例 (DRAM 时钟比 =2)

全局周期 g	0	1	2	3	4	5	6	...
Core 推进	✓	✓	✓	✓	✓	✓	✓	每拍一次
NoC 推进 (ratio=1)	✓	✓	✓	✓	✓	✓	✓	每拍一次
DRAM 推进 (ratio=2)	–	✓	–	✓	–	✓	–	每 2 拍一次
累加器 (DRAM)	1	2→0	1	2→0	1	2→0	1	累加至 ≥ 2 则推进并扣减

配置与可探索。通过将 DRAM（或 NoC）的时钟比设为大于 1，模拟器即可在不改动核心与引擎代码的前提下刻画“内存/网络相对更慢”的场景，第五章也对此进行了参数扫描与“存储墙/通信墙”敏感性分析。

4.4 模拟器支撑的研究与探索视角

前文从执行层设计出发，给出了单核状态机、NoC/DRAM 集成与跨时钟域同步的具体实现。本节从“可做什么研究”的角度，归纳上述设计所支撑的几类系统级研究与探索，并部分与第五章的验证与实验形成呼应。

瓶颈归因与阶段分解。端到端执行时间由计算、核间通信与片外访存共同决定。但在缺乏周期级可观测量的情况下，难以回答“时间主要花在计算、等待数据加载还是写回 DRAM”“多核扩展后通信是否取代计算成为主导瓶颈”等问题。本模拟器在每核上统计加载、计算、写回及因 NoC 背压停滞的周期数，并可选择性地导出按周期的状态转移踪迹，从而将总周期分解为“计算占用”“访存相关等待”与“NoC 背压”等可解释分量。该分解口径具有明确的语义（与状态机阶段一一对应）且可复现，适用于对调度器生成的 IR 进行事后分析。第五章将基于该口径对 ResNet 等真实负载进行占比分解，得到“计算约占多少、访存约占多少”的定量结果，为调度优化与资源调配提供依据。在多核场景下，非零的 NoC 背压占比与各核阶段分布的差异还可用于识别“热点核”与通信密集时段，与第五章中的热点核与工作负载甘特图分析相对应。

设计空间探索与参数敏感性。多核神经网络芯片的设计空间涉及核心计算能力、片上缓存容量、NoC 拓扑与缓冲、DRAM 通道与时序等多维参数。在流片前或缺乏多组硬件实测时，仿真成为探索该空间的主要手段。本模拟器将核心 MAC/向量单元数、SRAM 容量与带宽、NoC 与 DRAM 的后端与时钟比等抽象为配置项，使得“增加计算单元能否线性缩短时间”“何时瓶颈从计算迁移到通信或存储”“不同负载（如 ResNet34 与 ResNet50）在同一配置空间上的最优工作点是否一致”等问题可通过参数扫描与对比实验回答。第五章将针对“核心乘法累加单元数”与“NoC 虚拟通道数”两个维度进行一维扫描与二维联合探索，展示边际收益递减与瓶颈迁移现象，并比较不同负载的最优配置差异。

多核扩展与争用分析。单核性能分解仅能反映“该核上计算与访存的占比”。当负载

被调度到多核时,核间依赖、NoC 拥塞与 DRAM 排队会引入新的延迟,部分核心可能在加载或写回阶段因等待共享资源而停留更久。本章多核并发与资源争用小节已说明,模拟器通过“多核每周期同时推进、统一收集注入、NoC/DRAM 周期级建模”将争用与背压显式刻画为可观测的阶段占比与停滞周期。在此基础上,可进一步开展多核扩展实验。固定负载与调度策略,逐步增加核数或改变拓扑,观察总周期、各核阶段占比以及“因 NoC 背压停滞”周期随核数变化的趋势,从而评估扩展效率与通信瓶颈的显现条件。

跨时钟域与存储墙敏感性。真实系统中,计算核心、NoC 与 DRAM 控制器往往运行于不同时钟域,存储带宽与延迟随 DRAM 频率与时序变化。若仿真假设所有子系统同频,则无法量化“内存变慢”对端到端周期的影响。本章引入的时钟比与多速率推进机制,使 DRAM (或 NoC) 的等效频率可相对于核心单独配置,在保持因果一致与可复现的前提下,将“每全局周期 DRAM 可完成的请求数”随时钟比增大而降低,从而在总周期与阶段分解中体现“存储墙”的加剧。这一能力支撑如下研究。在固定负载与核心配置下,扫描 DRAM 时钟比,观察总周期及访存阶段占比随时钟比的变化曲线,用于评估系统对内存带宽/延迟的敏感程度。

面向微架构的算子编译优化。对于给定的算子类型、张量尺寸与核内微架构,分块策略、循环顺序与数据驻留方式的选择会显著影响计算阵列的有效利用率与片上缓存的访问效率,使得同一算子在同一硬件上的实际执行周期可相差数倍,属于神经网络芯片研究中独立的优化课题。本模拟器中计算周期的产生过程实现为可替换接口(见 4.1.3 节),与核心状态机、NoC/DRAM 推进等系统级逻辑解耦。后续可在不改动系统级仿真框架的前提下,将当前的上界式估算替换为含利用率折损、片上供数约束与流水填充的完整模型,或接入面向特定微架构的算子编译器与数据流仿真器。由此,模拟器可同时评估算子级优化(改变计算周期)与系统级参数调整(改变 NoC/DRAM 配置)对端到端性能的联合影响,为“编译—微架构—系统”三层协同优化提供统一的评估平台。

5 模拟器验证与瓶颈及热点分析实验

本章全部真实负载实验均采用 ResNet34 与 ResNet50。选择该系列负载的原因在于：(1) 精度验证所需的”金标准”(Gemini-Compiler-IR 项目发布的 ZeBu 硬件仿真踪迹) 目前仅覆盖 ResNet 系列的调度 IR 与对应执行踪迹，在此条件下方可进行公平对比。(2) ResNet 兼具卷积与全连接等典型算子，已能展示从计算受限到存储受限的瓶颈转换、DRAM 带宽墙、SRAM 端口带宽墙、NoC 争用与核数扩展上限等多核系统级关键现象。需要指出的是，模拟器以调度器 IR 为输入，执行层不区分上游模型类型，因此本章所建立的验证方法与瓶颈分析框架对 Transformer 等其他负载同样适用（见第 2.2 节与第六章展望）。

5.1 模拟器基本功能与精度验证 (Micro-benchmarks)

本文模拟器支持两档 NoC/DRAM 后端，后文统称为 simple 后端与 full 后端。Simple 后端采用简化的 NoC 与 DRAM 模型：网络与存储请求使用固定延迟或轻量排队，不引入 BookSim2/Ramulator2 的细粒度时序（如 DDR 的行列延迟、bank 冲突等），适用于快速校验与对外对比时消除 DRAM 建模差异。Full 后端采用 BookSim2 作为 NoC 模型、Ramulator2 作为 DRAM 模型（DDR4 时序、bank 冲突、请求排队等），用于多核设计空间探索时保留 NoC/DRAM 的拥塞与排队效应。

本节通过可控微基准与硬件模拟平台对照，验证模拟器的三项核心能力：(1) 核间依赖的因果一致性。(2) DRAM 请求/响应闭环在 simple 与 full 后端下的差异。(3) 单核下与 ZeBu 硬件模拟的总周期精度。MB-1 与 MB-2 均同时使用 full 后端与 simple 后端运行，以验证完整数据通路并对比两种后端的差异。

5.1.1 MB-1：双核点对点依赖

实验原理。本实验构造一个最小的双核、点对点数据依赖场景，用于验证“消费者核心必须等到生产者数据到达后才能开始计算”的因果一致性。具体而言：第一个核心 (Core 0) 作为生产者，先从片外存储器 (DRAM) 加载权重 32 KB 与输入特征 8 KB，接着进行 500 个仿真周期的计算，产生 128 KB 的输出并可供其他核心读取。第二个核心 (Core 1) 作为消费者，需要先从 DRAM 加载自己的权重 64 KB，同时必须等待 Core 0 产出的 128 KB 数据作为自己的输入特征，二者都就绪后再进行 300 周期计算，最后将结果写回 DRAM。若模拟器正确实现依赖传播，则 Core 1 在加载阶段应一直等待，直到 Core 0 的输出经核间通路到达后，才能结束加载、进入计算阶段。

实验过程。硬件与负载配置如下。硬件配置：双核拓扑。NoC 与 DRAM 分别采用前

文定义的 full 后端 (BookSim2 + Ramulator2, 完整建模 DDR 时序与网络排队) 与 simple 后端 (固定延迟、简化建模) 各运行一次, 以对比两种后端下核间依赖与访存延迟的叠加效果。运行负载: 即上述“Core 0 生产者、Core 1 消费者”的两层调度, 输入与权重规模如上。实验时对两种后端分别执行仿真, 并开启状态记录。从仿真得到的状态时间线中读取每个核心在加载、计算、写回各阶段的起始与结束周期, 以及各阶段持续时长, 用于填表与绘图。

实验结果。表5-1给出两种后端下各核的状态转移时间点与阶段时长。

表 5-1 MB-1 双核点对点依赖实验结果

后端	核/阶段	起始周期	结束周期	时长
full	Core 0 Loading	0	207	207
	Core 0 Computing	207	707	500
	Core 1 Loading	0	2000	2000
	Core 1 Computing	2000	2300	300
	Core 1 Writeback	2300	2456	156
	总周期			2458
simple	Core 0 Loading	0	180	180
	Core 0 Computing	180	680	500
	Core 1 Loading	0	938	938
	Core 1 Computing	938	1238	300
	Core 1 Writeback	1238	1402	164
	总周期			1404

图5-1根据全后端运行得到的状态转移踪迹绘制, 直观展示两核的状态转移时间线: Core 0 先完成 Loading 与 Computing, Core 1 在长时间 Loading (等待 Core 0 的输出与 DRAM 响应) 后才进入 Computing 与 Writeback, 与表5-1一致。数据来源见附录附录 A.5。

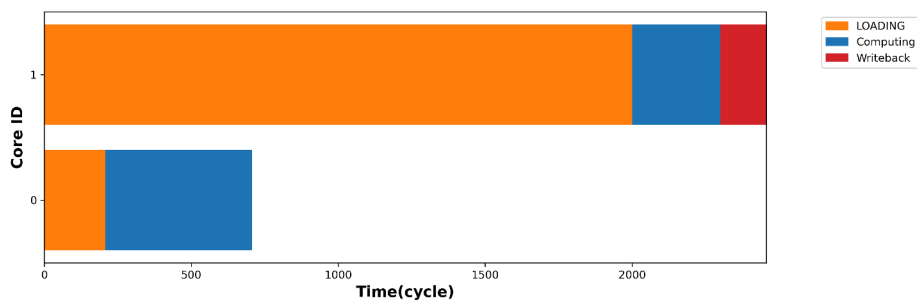


图 5-1 MB-1 状态转移时间线 (全后端。数据见附录附录 A.5)。横轴为仿真周期 (cycle), 纵轴为两核 (上方为 Core 1, 下方为 Core 0)。图中颜色表示各核所处状态: 橙色为 Loading, 蓝色为 Computing, 红色为 Writeback

对结果的分析。

1. 因果一致性验证通过。两种后端下，Core 1 均在 Core 0 完成计算（full 下约 707 周期，simple 下约 680 周期）之后才结束加载阶段、进入计算阶段（full 下 Core 1 加载约 2000 周期，simple 下约 938 周期），说明依赖传播的因果约束被正确执行。仿真输出的状态记录也显示，Core 1 在加载阶段明确标记为“等待来自 Core 0 的 128 KB 数据”，与上述依赖关系一致。
2. 计算时长一致。两种后端下 Core 0 与 Core 1 的计算阶段分别为 500 与 300 周期，与调度中设定的计算量完全吻合，验证了计算建模的正确性。
3. DRAM 后端影响访存阶段。全后端下 Core 1 的 LOADING 阶段为 2000 周期，simple 下仅为 938 周期。差异来自 Ramulator2 的 DDR4 时序约束（tRCD、tCL 等）使 DRAM 响应延迟更高，同时 Core 1 还需等待 Core 0 的计算及 NoC 传输完成，两者串行叠加导致 LOADING 时间显著拉长。
4. NoC 数据包一致。两种后端均产生 2 个 NoC 数据包（Core 0→Core 1 的请求与响应），验证了核间通信路径的正确性。

5.1.2 MB-2: 仅 DRAM 依赖（无核间通信）

实验原理。本实验在单核上执行两层串行计算，且两层所需的输入与权重全部来自片外存储器（DRAM），不涉及核间数据传递，用于验证：（1）单核、仅 DRAM 依赖时，不应产生任何 NoC 数据包。（2）simple 与 full 两种后端在“纯 DRAM 访存”场景下的周期差异，量化 DDR 时序建模对访存阶段的影响。具体负载为：第一层从 DRAM 加载权重 32 KB 与输入特征 8 KB，计算 500 周期后向 DRAM 写回 128 KB。第二层从 DRAM 加载权重 64 KB 与输入特征 128 KB（即第一层写回的数据），计算 300 周期后向 DRAM 写回 32 KB。两层的计算量固定，差异仅体现在 DRAM 的加载与写回阶段上。

实验过程。硬件配置为单核拓扑，NoC 与 DRAM 仍分别采用 full 后端与 simple 后端各运行一次。运行负载即上述“两层、仅 DRAM 依赖”的调度，无核间通信。对两种后端分别执行仿真并记录状态时间线，从时间线中读取每一层在加载、计算、写回各阶段的起止周期与时长，用于填表与对比。

实验结果。两种后端下各层的执行时间线见表5-2。

对结果的分析。

1. NoC 数据包为 0。两种后端下均无 NoC 流量（DRAM 直连核，未经过 NoC），验证了单核 DRAM-only 场景下通信路径的正确性。
2. 计算时长一致。Layer1 与 Layer2 的 COMPUTING 阶段在两种后端下分别为 500 与 300 周期，与调度中给定的计算周期一致。
3. DRAM 模型差异集中在访存阶段。全后端下，Layer1 的 Writeback 为 651 周期

(simple: 356, 增幅 83%), Layer2 的 Loading 为 853 周期 (simple: 484, 增幅 76%)。这反映了 Ramulator2 的 DDR4 时序 (行激活、列读取、bank 冲突、请求排队) 对大块数据传输的影响: Layer1 写回 128 KB、Layer2 加载 192 KB (64+128), 数据量越大时 DDR 时序约束越显著。

4. 总周期差异。全后端总周期 2685, simple 为 1987, 全后端高出 35%。差异完全来自 DRAM 访存阶段, 计算阶段无差异。后续 ZeBu 对比实验统一采用 full 后端, 以在包含完整 DDR 时序建模的条件下评估模拟器的端到端精度。

表 5-2 MB-2 仅 DRAM 依赖实验结果 (单核, 两层串行)

后端	层/阶段	起始周期	结束周期	时长
full	Layer1 Loading	0	207	207
	Layer1 Computing	207	707	500
	Layer1 Writeback	707	1358	651
	Layer2 Loading	1359	2212	853
	Layer2 Computing	2212	2512	300
	Layer2 Writeback	2512	2683	171
	总周期			2685
simple	Layer1 Loading	0	180	180
	Layer1 Computing	180	680	500
	Layer1 Writeback	680	1036	356
	Layer2 Loading	1037	1521	484
	Layer2 Computing	1521	1821	300
	Layer2 Writeback	1821	1985	164
	总周期			1987

MB-1 与 MB-2 小结。两个微基准验证了模拟器三项基本能力: 核间依赖因果一致性 (MB-1)、DRAM 闭环正确性 (MB-2)、以及 simple/full 后端在计算阶段完全一致而在访存阶段体现 DDR 时序差异。这为后续统一采用 full 后端进行 ZeBu 精度校验与多核设计空间探索奠定了基础。

5.1.3 与 ZeBu 硬件模拟的对比 (单核)

实验原理。Synopsys ZeBu 硬件模拟平台在单核配置下执行与本模拟器相同的调度 IR 所生成的汇编指令, 并通过硬件踪迹记录每条指令的执行区间 $[start, end]$ 。以 ZeBu 全程序总周期 ($\max(end)$) 作为基准, 与本模拟器的 Total cycles 对比, 相对误差定义为 $(\text{模拟器} - \text{ZeBu}) / \text{ZeBu} \times 100\%$ 。为使精度评估涵盖完整的系统级时序 (包括 DDR 行列时序与请求排队), 本实验采用 full 后端 (BookSim2 + Ramulator2) 与 ZeBu 硬件

模拟对照。

实验过程。ZeBu 侧的总周期取自 Gemini-Compiler-IR 项目发布的单核预跑结果（与第二章所述一致）。本实验在模拟器一侧采用单核、full 后端配置（BookSim2 + Ramulator2），选取 ResNet34 与 ResNet50 在批大小 1 与 4 下的调度结果分别运行仿真并记录总周期，与上述 ZeBu 参考周期逐项对比。

实验结果。表5-3给出上述四种配置下模拟器与 ZeBu 的总周期及相对误差。

表 5-3 与 ZeBu 硬件模拟的总周期对比（单核，full 后端。相对误差 = (模拟器 - ZeBu)/ZeBu × 100%)

模型	批大小	ZeBu 周期	模拟器周期	相对误差 (%)
ResNet34	1	1 922 067	1 890 695	-1.63
ResNet34	4	4 280 215	4 501 725	+5.18
ResNet50	1	2 545 655	2 432 592	-4.44
ResNet50	4	5 470 662	5 653 354	+3.34

对结果的分析。

1. 误差均在 $\pm 5.2\%$ 以内。4 种配置的相对误差范围为 -4.44% 至 $+5.18\%$ ，说明在包含完整 DRAM 时序建模的条件下，模拟器与 ZeBu 硬件模拟在总周期上吻合良好。
2. 正负偏差并存，无系统性偏移。批大小为 1 时模拟器周期略低于 ZeBu (-1.63% 、 -4.44%)，批大小为 4 时略高 ($+5.18\%$ 、 $+3.34\%$)，偏差方向与批大小有关：批较小时 Ramulator2 的 DDR 时序对小规模访存的建模与 ZeBu 存在微小差异。批增大后，工作负载级阶段划分与 ZeBu 指令级执行在访存重叠上的建模粒度差异被放大。
3. 趋势一致。随批大小从 1 增至 4，两个模型的总周期均近似线性增长（ResNet34: $1.89\text{M} \rightarrow 4.50\text{M}$ ，ZeBu: $1.92\text{M} \rightarrow 4.28\text{M}$ 。ResNet50: $2.43\text{M} \rightarrow 5.65\text{M}$ ，ZeBu: $2.55\text{M} \rightarrow 5.47\text{M}$)，趋势一致。

结论。在 full 后端、单核条件下，本模拟器的总周期与 ZeBu 硬件模拟平台在 4 种 ResNet 配置上的相对误差均在 $\pm 5.2\%$ 以内，验证了包含完整 DRAM 时序建模在内的端到端时序模型的准确性。结合 MB-1/MB-2 对核间依赖与 DRAM 闭环的功能验证，可认为本模拟器具备可信的建模能力，为后续多核瓶颈分析与设计空间探索提供了可靠基础。

5.2 真实 AI 负载瓶颈分析（调度器生成 IR）

本节选取 ResNet34 与 ResNet50 两类 CNN 负载，以调度器生成 IR 驱动模拟，通过端到端阶段占比分解与单核阶段统计，实现对性能瓶颈的可解释归因。

5.2.1 端到端阶段占比分解口径

实验原理。端到端执行时间可拆分为“计算占用”与“等待占用”。记每核在 LOADING、COMPUTING、WRITEBACK 阶段的周期分别为 C_c^{load} , C_c^{comp} , C_c^{wb} ，显式网络背压周期为 C_c^{nocstall} 。本文采用可复现的近似拆分：

$$T_{\text{Compute}} = \sum_c C_c^{\text{comp}}, \quad T_{\text{NoC_Stall}} = \sum_c C_c^{\text{nocstall}}, \quad T_{\text{Mem_Stall}} \approx \sum_c C_c^{\text{wb}} + \sum_c C_c^{\text{load}} - \sum_c C_c^{\text{nocstall}}.$$

即：计算时间取各核 COMPUTING 之和。NoC 背压取各核 StallNoC 之和。存储相关等待近似为写回与加载时间之和再减去已计入的 NoC 背压。单核场景下无核间通信，故 $T_{\text{NoC_Stall}} = 0$ ，瓶颈主要体现在计算与访存（DRAM 读/写）的占比上。

实验过程。采用 Gemini-Compiler-IR 中 ResNet34 与 ResNet50 的调度结果（与第 5.1.3 节 ZeBu 对比实验同源）作为输入，在单核、simple 后端配置下运行模拟器。仿真结束后，从输出的“每核统计”中读取每个核心在加载、计算、写回以及因网络背压而停滞的周期数，按上式汇总为总计算时间 T_{Compute} 、NoC 背压时间 $T_{\text{NoC_Stall}}$ 与访存相关等待时间 $T_{\text{Mem_Stall}}$ ，并计算三者占总执行时间的百分比。单核下无核间通信，NoC 背压恒为 0，瓶颈主要体现在计算与访存的占比上。表中给出批大小 1 的分解结果。

实验结果。表 5-4 给出 ResNet34 与 ResNet50 在批大小 1、单核下的总周期及 T_{Compute} 、 T_{NoC} 、 T_{Mem} 的周期与占比。

表 5-4 真实负载端到端瓶颈分解（单核、批 1。 $T_{\text{NoC}} = 0$ ）

模型	总周期	T_{Compute}	T_{NoC}	T_{Mem}	计算%	NoC%	访存%
ResNet34	1 890 695	1 530 664	0	359 888	81.0	0.0	19.0
ResNet50	2 432 592	1 863 694	0	568 717	76.6	0.0	23.4

对结果的分析。

1. NoC 背压为 0。单核下无核间通信， $T_{\text{NoC}} = 0$ ，与预期一致。
2. 计算为主导瓶颈。ResNet34 与 ResNet50 的计算占比分别为 81.0% 与 76.6%，说明在单核配置下端到端时间主要由计算阶段占据，与两类 CNN 的计算密度相符。
3. 访存占比随模型加深而上升。ResNet50 的访存占比(23.4%)高于 ResNet34(19.0%)，

与 ResNet50 层数更多、中间激活与权重更大、DRAM 读写的相对占比升高一致。多核映射下核间依赖与 NoC 竞争将引入非零 $T_{\text{NoC_Stall}}$ ，届时可用同一分解框架区分计算与通信/访存瓶颈。

结论。通过调度器 IR 驱动模拟并按 T_{Compute} 、 $T_{\text{NoC_Stall}}$ 、 $T_{\text{Mem_Stall}}$ 分解，可对端到端周期做可解释的瓶颈归因。在单核 ResNet34/50 上，计算占比约 77%–81%，访存占比约 19%–23%，为后续多核扩展与资源调配（如增加单核计算单元、调节 NoC 与 DRAM 带宽）提供定量依据。

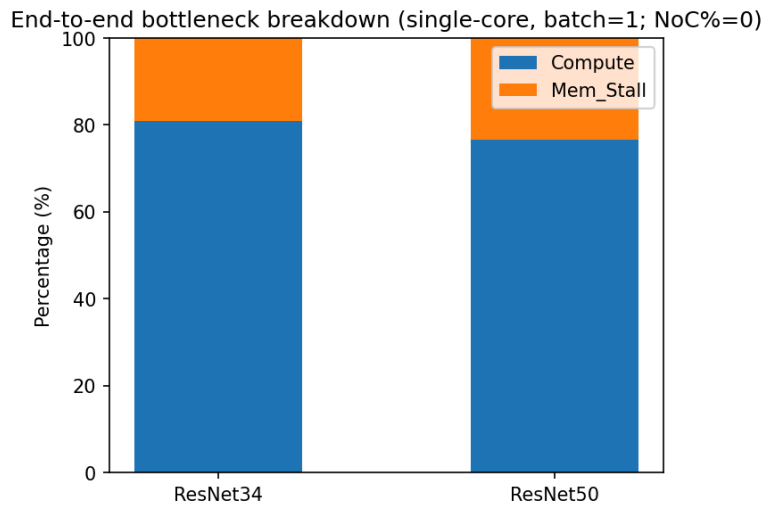


图 5-2 端到端瓶颈占比（单核、批 1。Compute / Mem_Stall 堆叠柱状图，NoC%=0。数据同表 5-4）

5.2.2 热点核与阶段占比可视化

实验目的。5.2.1 节将端到端时间分解为计算、NoC 背压与访存等待并给出占比，但仍是各核加总的宏观结果。本小节在此基础上给出热点核（在总周期或某类等待上占比突出的核心）的可视化表达：在双核微基准上采用各核各阶段占该核活跃时间的比例热力图，在单核真实负载上采用与分解口径一致的阶段占比堆叠条，用于从空间（核）维度补充瓶颈解释，并与 5.2.1 节宏观分解对照。

实验原理。“热点核”在本文中定义为：在给定维度（如总周期占比、计算周期占比、访存等待周期占比或 NoC 背压周期占比）上显著高于其他核心的核。模拟器在运行时可记录每个核心在每一仿真周期内所处的状态（加载、计算、写回或停滞），形成按核、按时间排列的状态时间线。据此可统计各核在各阶段的累计周期数（即 5.2.1 节中单核统计的推广），进而识别计算或等待时间最长的若干核心。对于单核 ResNet，本节采用与表 5-4 一致的阶段占比堆叠条，与 5.2.1 的分解口径严格对照。对双核场景则采用 5.1 节中的 MB-1 微基准，在工作负载汇总上绘制各核各阶段占该核活跃时间的比例热力

图，以验证“按核、按阶段”分解的可行性。

实验配置与结果。(1) 双核方法验证 (MB-1): 采用 5.1 节 MB-1 双核点对点依赖负载与 simple 后端，基于仿真得到的状态轨迹与工作负载汇总。由各核工作负载的加载、计算、写回周期按核归一化，得到各阶段占该核活跃时间的比例，绘制热点热力图 (图 5-3)。(2) 单核真实负载 (ResNet34/50): 采用与 5.2.1 相同的批大小 1、单核统计结果，将 T_{Compute} 与 $T_{\text{Mem_Stall}}$ 占总时间的比例以堆叠条呈现，与表 5-4 及图 5-2 一致。

结果分析。图 5-3 表明，在 MB-1 中 Core 1 的加载阶段占该核活跃时间的比例显著高于 Core 0，与 consumer 需同时等待 DRAM 与 Core 0 数据的设定一致。计算阶段 Core 0 占比更高，体现生产者核以计算为主、消费者核以等待与写回为主的分工。与 5.1 节表 5-1 及因果一致性结论一致，Core 0 先完成加载与计算，Core 1 在长时间加载后方进入计算与写回。图 5-2 与表 5-4 一致地表明，单核 ResNet 端到端时间仍以计算为主、访存相关等待次之，ResNet50 的访存占比高于 ResNet34。单核真实负载上阶段占比条与 5.2.1 节宏观分解一致。

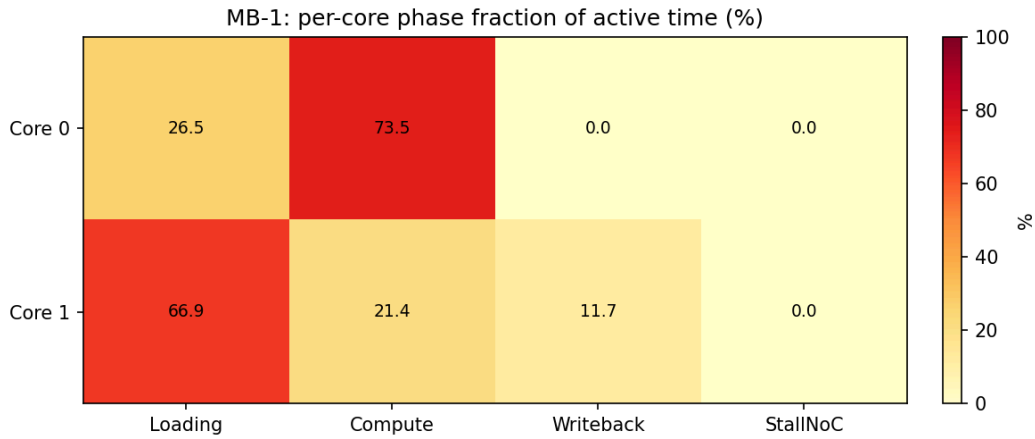


图 5-3 MB-1 (simple 后端) 双核各阶段占该核活跃时间的比例 (热点热力图。该微基准下 StallNoC 为 0)

5.3 多核瓶颈分析与优化实验

前两节在单核配置下验证了模拟器精度并做了端到端瓶颈分解。本节将实验扩展到多核 (2×2 , 4 核) 场景，使用全后端 (BookSim2 + Ramulator2) 运行真实 DNN 负载，展示模拟器在多核 DRAM 带宽瓶颈定位与优化效果验证上的应用价值。

5.3.1 实验设计

实验目标。(1) 构造一个存储受限的多核基线配置，使 DRAM 带宽成为性能瓶颈，在甘特图上直观表现为加载阶段占据大部分时间。(2) 通过将 DRAM 技术从 DDR4 升

级为 HBM（高带宽存储器），展示带宽升级对端到端性能的改善，以此验证模拟器能正确反映 DRAM 子系统差异。

负载与 IR 生成。采用 ResNet50 网络，批大小为 1，通过 Gemini 编译器的 2×2 核映射模式生成调度 IR。IR 共含 288 个工作负载（每核 72 个），涉及约 90 MB 的 DRAM 读取与 288 次 DRAM 写回，涵盖了 ResNet50 全部卷积、池化与全连接层。

硬件配置。两组配置共享相同的核心微架构（MAC 256、Vector 64、SRAM 16 MB、核频 25 MHz）与 NoC 拓扑（ 2×2 Mesh，BookSim2 后端），差异仅在 DRAM 子系统：

表 5-5 多核瓶颈分析实验的两组 DRAM 配置

参数	DDR4 基线	HBM 优化
DRAM 标准	DDR4-2400R	HBM-2Gbps
通道数	1	4
Rank 数	1	1
峰值带宽	19.2 GB/s	128.0 GB/s
等效带宽/核周期	768 B/cycle	5120 B/cycle

DDR4 基线采用单通道以模拟带宽受限场景（峰值 19.2 GB/s），HBM 优化采用 4 通道 HBM（峰值 128.0 GB/s），峰值带宽提升约 6.7 倍。两组均使用 Ramulator2 后端，具备 DDR 行列时序、bank 冲突与请求排队的全真建模。受行/bank 冲突与 FR-FCFS 调度约束，实测有效带宽通常显著低于上述峰值，在 ResNet50 的 DRAM 访问模式下尤为明显，因此 DDR4 单通道仍会构成显著的访存瓶颈。

5.3.2 基线仿真结果（DDR4 单通道）

实验结果。表5-6给出 DDR4 基线配置下的仿真统计。

表 5-6 DDR4 基线仿真每核统计（ResNet50， 2×2 ，批 1）

核	Idle	Loading	Computing	Writeback	Workloads
0	0	140 680	22 352	64 537	72
1	0	153 077	22 352	58 929	72
2	0	144 515	22 352	66 362	72
3	0	153 799	22 352	62 381	72
总计/最大	—	592 071	89 408	252 209	288

总周期为 238 803。各核加载阶段占活跃时间的 60%–65%，计算阶段仅 9%–10%，写回阶段约 26%–28%。Loading/Computing 总比值达 6.6，表明系统处于严重的存储受限状态。图5-4以热力图形式展示了各核各阶段占活跃时间的比例，Loading 列颜色最

深，直观反映 DRAM 带宽瓶颈。

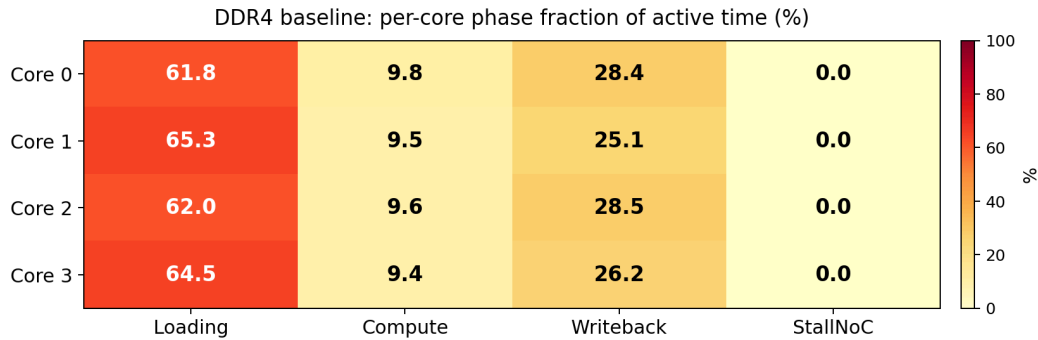


图 5-4 DDR4 基线各核阶段占比热力图 (2×2 , ResNet50 批 1)。Loading 占比 62%–65%，Compute 仅 9%–10%，系统严重存储受限

图5-5给出 DDR4 基线的甘特图。4 核的加载段（橙色，含 DRAM 等待与核间等待子段）占据绝大部分时间线，计算段（蓝色）极为狭窄，直观展示了带宽瓶颈。

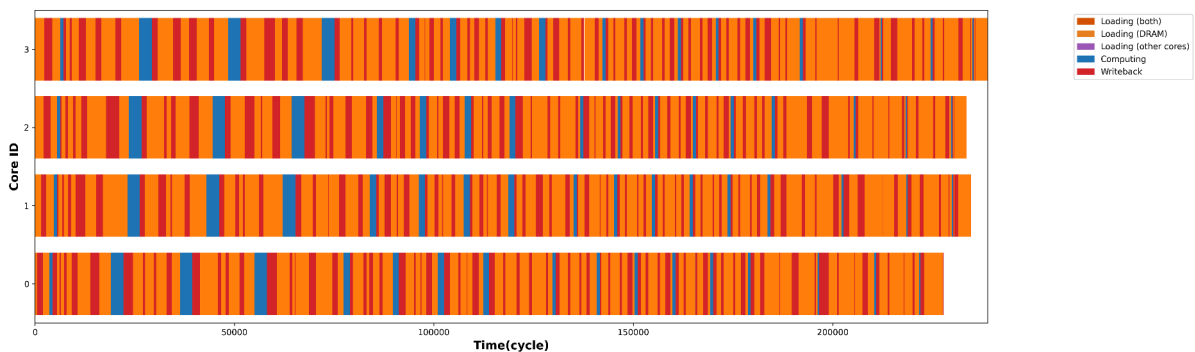


图 5-5 DDR4 基线甘特图 (2×2 , ResNet50 批 1)。橙色为加载（含 DRAM 与核间等待），蓝色为计算，红色为写回。加载阶段占据大部分时间线

5.3.3 优化仿真结果（HBM 4 通道）

将 DRAM 从 DDR4 单通道替换为 HBM 4 通道后重新仿真，表5-7给出结果。

表 5-7 HBM 优化仿真每核统计 (ResNet50, 2×2 , 批 1)

核	Idle	Loading	Computing	Writeback	Workloads
0	0	84 488	22 352	18 092	72
1	0	102 664	22 352	18 713	72
2	0	85 616	22 352	18 093	72
3	0	97 415	22 352	18 533	72
总计/最大	—	370 183	89 408	73 431	288

总周期降至 144 000。Loading/Computing 比值从 6.6 降至 4.1，Writeback 总周期从

252 209 降至 73 431（降幅 70.9%）。图5-6的热力图显示，Loading 占比仍在 67%–71%（因 SRAM 端口瓶颈），但 Writeback 从 25%–28% 降至 13%–15%，Compute 从 9%–10% 升至 16%–18%。

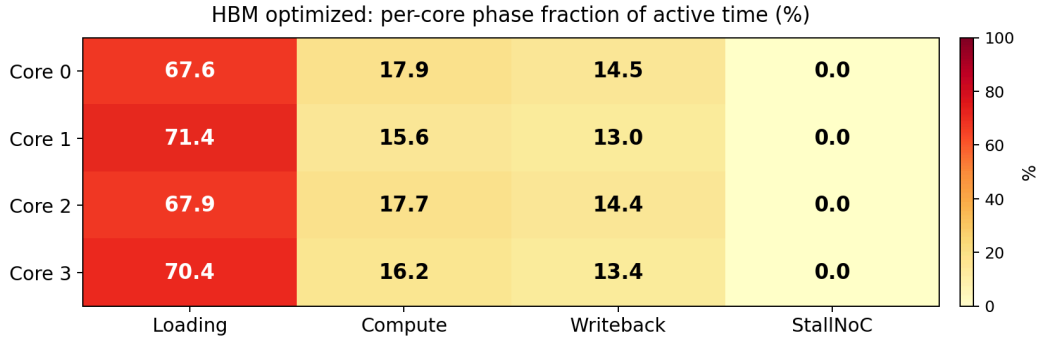


图 5-6 HBM 优化各核阶段占比热力图（ 2×2 ，ResNet50 批 1）。与图5-4对比，Writeback 占比显著下降，Compute 占比上升

图5-7为 HBM 优化甘特图，可见写回段（红色）显著收窄，加载段虽仍为主导但已明显缩短。

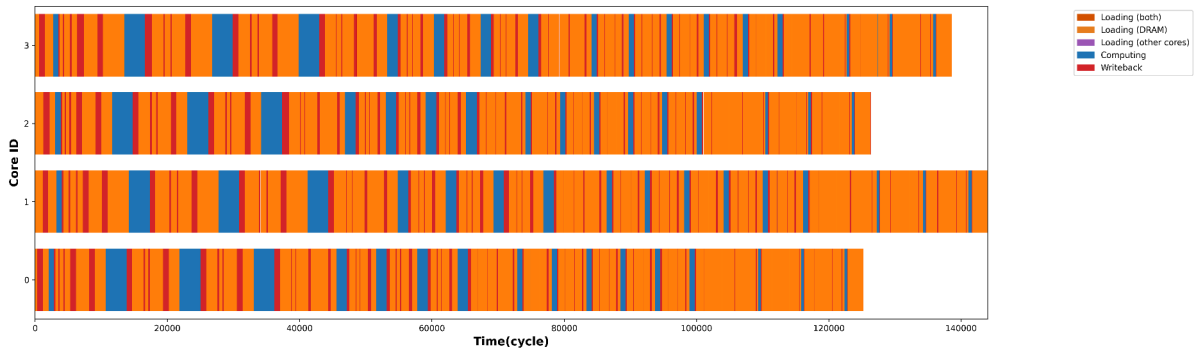


图 5-7 HBM 优化甘特图（ 2×2 ，ResNet50 批 1）。与图5-5对比，加载与写回段均明显缩短

残留瓶颈分析。虽然 HBM 的峰值 DRAM 带宽达到 128 GB/s（5120 B/cycle），远超加载数据所需，但加载阶段仍占各核活跃时间的约 60%。对 288 个工作负载的逐项分析表明：92% 的工作负载的加载时间受 SRAM 写端口带宽限制（256 B/cycle），而非 DRAM 响应延迟。具体而言，DRAM 数据到达核后必须按 SRAM 写带宽逐周期写入片上存储。当 DRAM 带宽远超 SRAM 写入速率时，后者成为新的瓶颈。这一“片上带宽墙”现象在高带宽外部存储配合传统窄端口 SRAM 时普遍存在。

5.3.4 进一步优化：提升 SRAM 端口带宽

基于上述分析，在 HBM 配置基础上将 SRAM 读写带宽从 256 B/cycle 提升至 1024 B/cycle（ $\times 4$ ），对应于在硬件设计上加宽 SRAM 读写端口或采用多 bank 并行访问。其余参数（核微架构、NoC、DRAM）保持不变。

实验结果。表5-8给出 HBM + SRAM 1024 B/cycle 配置下的仿真统计。

表 5-8 HBM + 宽 SRAM 仿真每核统计 (ResNet50, 2×2 , 批 1, SRAM 带宽 1024 B/cycle)

核	Idle	Loading	Computing	Writeback	Workloads
0	0	28 374	22 352	8 589	72
1	0	31 058	22 352	8 668	72
2	0	28 853	22 352	10 588	72
3	0	30 766	22 352	9 429	72
总计/最大	—	119 051	89 408	37 274	288

总周期降至 62 818, 较 HBM (SRAM 256) 再降 56.4%, 较 DDR4 基线降 73.7%。Loading/Computing 比值从 4.1 降至 1.3, 计算阶段占瓶颈核活跃时间的比例从 15.6% 升至 35.7%, 系统从严重存储受限逐步趋向计算—存储平衡。图5-8的热力图显示, Compute 占比升至 36%–38%, Loading 降至 47%–50%, 三阶段占比趋于均衡。图5-9的甘特图中, 计算段 (蓝色) 面积明显增大, 加载与写回段均大幅缩短。

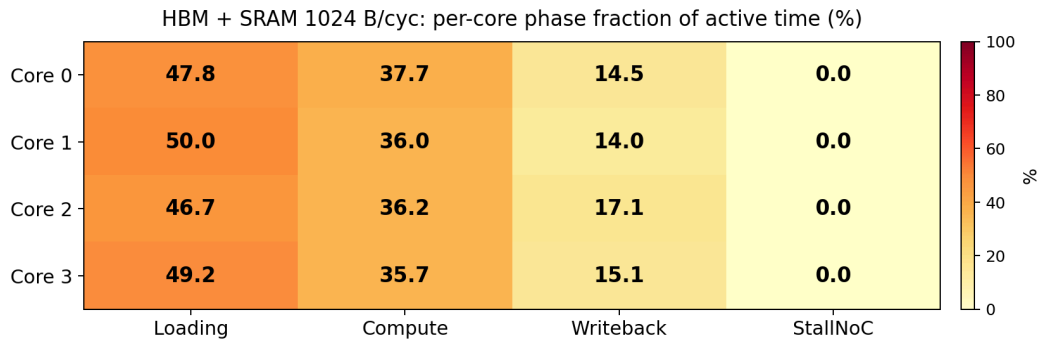


图 5-8 HBM + 宽 SRAM 各核阶段占比热力图 (2×2 , ResNet50 批 1, SRAM 1024 B/cycle)。
与图5-4、5-6对比, Compute 占比从 9% 升至 36%, 系统趋向均衡

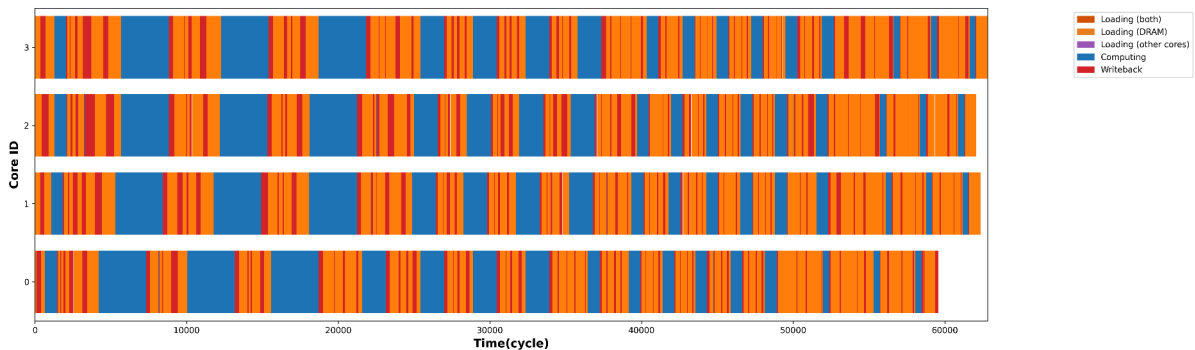


图 5-9 HBM + 宽 SRAM 甘特图 (2×2 , ResNet50 批 1, SRAM 1024 B/cycle)。计算段占比显著提升, 系统趋向计算—存储平衡

5.3.5 三组配置对比分析

表5-9汇总三组配置的关键指标，展示递进式优化的完整路径。

表 5-9 三组配置仿真结果对比（ResNet50， 2×2 ，批 1）

指标	DDR4 基线	HBM 优化	HBM+ 宽 SRAM	总改善
总周期	238 803	144 000	62 818	-73.7%
相对 DDR4 加速比	1.00×	1.66×	3.80×	—
Loading 总周期	592 071	370 183	119 051	-79.9%
Writeback 总周期	252 209	73 431	37 274	-85.2%
Computing 总周期	89 408	89 408	89 408	0%
Loading/Computing	6.62	4.14	1.33	—
瓶颈核计算占比	9.4%	15.6%	35.7%	—
DRAM 峰值带宽	19.2 GB/s	128 GB/s	128 GB/s	—
SRAM 写带宽	256 B/cy	256 B/cy	1024 B/cy	—

结论。

1. DRAM 带宽升级（DDR4 $\rightarrow$ HBM）将总周期从 238 803 降至 144 000（1.66 $\times$ ），写回阶段降幅高达 70.9%。瓶颈从 DRAM 读写带宽迁移至片上 SRAM 写端口。
2. SRAM 端口加宽（256 $\rightarrow$ 1024 B/cycle）在 HBM 基础上将总周期再降 56.4% 至 62 818（相对 DDR4 基线 3.80 $\times$ ）。这揭示了“片上带宽墙”现象：当外部 DRAM 带宽充裕时，SRAM 的读写端口宽度反而成为数据搬运的限速环节。
3. 瓶颈逐级迁移。DDR4 基线下瓶颈在 DRAM 带宽（Loading/Computing = 6.6），HBM 优化后瓶颈迁移至 SRAM 端口（Loading/Computing = 4.1），最终提升 SRAM 端口后系统趋向计算—存储平衡（Loading/Computing = 1.3，计算占比 35.7%）。这一逐级暴露新瓶颈的过程验证了模拟器在迭代式硬件优化中的实用价值。
4. 计算周期恒定。三组实验中各核计算时长均为 22 352 cycles，证明性能提升完全来自存储子系统的改善，且模拟器能正确隔离各子系统的贡献。

5.4 多核吞吐量的设计空间探索

5.3 节在固定 2×2 核、batch=1 的条件下揭示了 DRAM 与 SRAM 带宽瓶颈。然而在实际推理部署中，吞吐量（单位时间完成的推理数）才是核心指标，且受单核算力、核数规模、片上存储以及 die 面积等多因素共同约束。本节沿三个维度展开设计空间探索：（1）单核算力（MAC 规模）；（2）核数扩展；（3）Batch 大小。所有实验均采用 ResNet50 网络、HBM 4 通道 DRAM 与 1024 B/cycle SRAM 带宽（5.3 节确定的最优存

储配置)，并使用全后端（BookSim2 + Ramulator2）。

5.4.1 单核算力扩展（MAC 规模扫描）

在 2×2 核配置下，将单核乘法累加单元数从 256 扫描至 512 和 1024，对 batch=1 和 batch=4 各运行一组仿真。

实验结果。表5-10给出 MAC 扫描结果。

表 5-10 MAC 规模扫描（ 2×2 核，HBM+SRAM1024）

MAC	Batch	总周期	Loading	Computing	Writeback	计算占比
256	1	62 775	119 210	89 408	36 944	35.8%
512	1	62 778	119 220	89 408	36 946	35.8%
1024	1	62 776	119 223	89 408	36 935	35.8%
256	4	258 289	508 269	357 632	153 289	34.8%
512	4	258 289	508 269	357 632	153 289	34.8%
1024	4	257 969	510 067	357 632	153 290	34.8%

分析。三种 MAC 规模下的总周期几乎无差异（batch=1 下均约 62 778，batch=4 下均约 258 000），且 Computing 总周期完全一致。这一看似反直觉的结果源于当前负载的特征：Gemini 编译器生成的 IR 中，288 个工作负载（batch=1）有 75% 的工作负载计算量极小（`compute_cycles` ≤ 1 ），真正有显著计算的仅为 residual 层。即使将 MAC 翻四倍，瓶颈核的活跃时间仍由 Loading 主导（ $L/C \approx 1.33$ ），计算缩减对总周期的贡献被淹没。结论：在当前存储配置下，系统处于存储受限状态，单纯提升单核算力无法提高吞吐量。

5.4.2 核数扩展

固定单核乘加器单元数量 `mac_units`=512，将核数从 $2 \times 2=4$ 核扩展至 $4 \times 4=16$ 核与 $8 \times 8=64$ 核。每种核数的 IR 由 Gemini 编译器针对对应拓扑重新生成。

实验结果。表5-11给出核数扩展结果。

表 5-11 核数扩展 (HBM+SRAM1024, MAC=512)。8×8 batch=1 因死锁未完成

拓扑	Batch	总周期	L/C	计算占比	Workloads	状态
2 × 2	1	62 781	1.33	35.8%	288	完成
4 × 4	1	60 839	5.61	9.2%	1136	完成
8 × 8	1	—	—	—	4544	死锁
2 × 2	4	257 827	1.42	34.8%	1152	完成
4 × 4	4	296 297	6.95	7.6%	4544	完成

分析。

1. $2 \times 2 \rightarrow 4 \times 4$: 总周期几乎不降。Batch=1 下从 62 781 降至 60 839 (仅 -3.1%), batch=4 下反而从 257 827 升至 296 297 (+14.9%)。原因在于: 核数增加后工作负载被分散到更多核 (从 288 增至 1136), 但 16 核共享同一 DRAM 子系统导致读写竞争加剧, Loading 与 Writeback 剧增。L/C 从 1.33 升至 5.61 (batch=1) 和从 1.42 升至 6.95 (batch=4), 系统从接近均衡退回到严重存储受限。
2. 8×8 死锁。64 核配置下仿真无法完成: 99.3% 的工作负载完成后, 剩余 30 个工作负载的 15 个核陷入互等状态。这是因为 ResNet50 的层级结构 (约 72 个 unique 层) 无法均匀分布到 64 核, Gemini 编译器生成的 IR 中大量核的工作负载极少, 导致复杂的依赖链形成环路。结论: 核数不宜超过负载的并行度上限。

图5-10给出 16 核的端到端甘特图, Loading 与 Writeback 段占据绝大部分时间线, DRAM 争用严重。图5-11进一步以热力图形式展示各核阶段占活跃时间的比例: 全部核心的 Loading 占比在 50%–55%、Writeback 在 36%–41%, Compute 仅约 9.3%–9.5%, 与 5.3 节 DDR4 基线 (4 核) 的 9%–10% 接近, 表明核数扩大 4 倍后系统从接近均衡退回到严重存储受限。

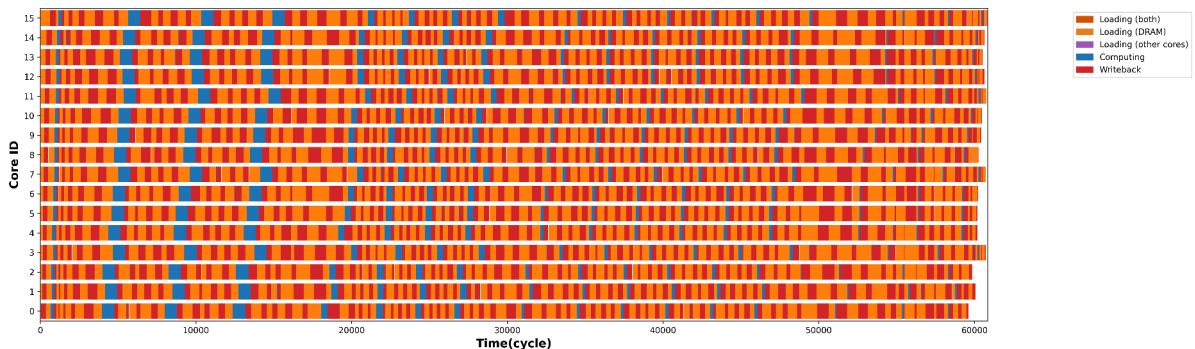


图 5-10 4 × 4 核甘特图 (ResNet50 batch=1, MAC=512)。16 核的 Loading 与 Writeback 占据绝大部分时间, DRAM 争用严重

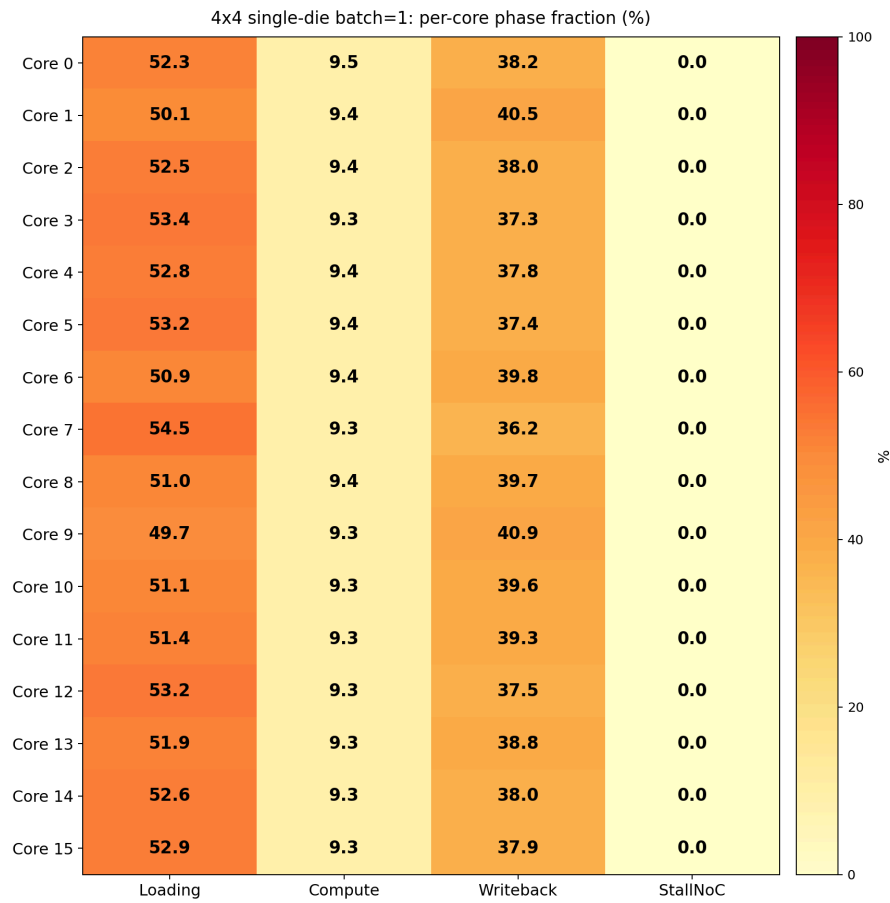


图 5-11 4×4 核各阶段占比热力图 (ResNet50 batch=1, MAC=512)。16 核的 Compute 占比均不足 10%，Loading 与 Writeback 共占约 90%

5.4.3 Batch 扩展效率

在推理部署中，增大 batch 可以提高数据复用率与吞吐量。表5-12对比 batch=1 与 batch=4 在不同核数配置下的吞吐量。吞吐量定义为 $\text{Throughput} = \text{batch} / \text{Total_cycles}$ 。

表 5-12 Batch 扩展效率 (MAC=512, HBM+SRAM1024)

拓扑	Batch	总周期	吞吐量 ($\times 10^{-5}/\text{cyc}$)	相对 b=1 加速	效率
2×2	1	62 781	1.593	—	—
2×2	4	257 827	1.551	$0.97\times$	24.3%
4×4	1	60 839	1.644	—	—
4×4	4	296 297	1.350	$0.82\times$	20.5%

分析。Batch 从 1 增至 4 时，总周期增长约 $4.1 \times (2 \times 2)$ 到 $4.9 \times (4 \times 4)$ ，均超过 batch 本身的 $4 \times$ 倍率，导致吞吐量反而略降。Batch 扩展效率 ($=\text{batch}=4$ 吞吐量 / $\text{batch}=1$ 吞吐量 / 4) 仅约 24%。这是因为在当前存储受限配置下，增大 batch 线性增加

了 DRAM 读写数据量，但 DRAM 带宽并未同步扩展，导致存储争用超线性增长。

5.4.4 综合结论与设计启示

1. 存储受限状态下，提升算力无效。当 $\text{Loading/Computing} > 1$ 时，系统为存储受限，增加 MAC 单元对吞吐量无贡献。设计者应优先解决 DRAM 带宽或 SRAM 端口带宽瓶颈，再考虑计算资源投入。
2. 核数扩展存在明确上限。 $2 \times 2 \rightarrow 4 \times 4$ 在 $\text{batch}=1$ 下仅提升 3%，在 $\text{batch}=4$ 下反而退步 15%， 8×8 直接死锁。核数必须与负载的并行度匹配：ResNet50 的有效并行度约 4–16 核，超过此范围后 DRAM 争用与调度依赖冲突主导性能。
3. Batch 扩展效率受限于存储带宽。Batch=4 相对 $\text{batch}=1$ 的吞吐量效率仅约 24%，说明在存储受限系统中 batch 扩展的收益远不及理想的线性关系。提升 DRAM 通道数或采用更高带宽的 HBM2E/HBM3 可望改善 batch 扩展效率。
4. 模拟器对神经网络芯片设计的价值。本节实验覆盖了 12 种配置组合，每次仿真耗时 3–60 分钟。这一效率使得在流片前对核数、算力、存储带宽等关键参数进行快速定界成为可能。

6 结论与展望

6.1 结论

第五章所开展的验证与分析实验，从功能正确性、时序可比性与瓶颈及热点洞察三个层面支撑了本模拟器的有效性与实用价值。本节对主要结论做简要归纳。

微基准验证。第一个微基准（双核点对点依赖）验证了消费者核心在加载阶段仅在收到对应数据的读响应之后才会进入计算阶段。状态转移与数据追踪中的等待行为与设计一致，模拟能够稳定结束。第二个微基准（所有输入均来自片外存储）在简单模式与高保真模式下的对比表明，在并发度较低时，两种模式给出的总周期趋势一致。当并发度提高时，高保真模式因网络与存储的排队与时序约束会得到更长的总周期，而简单模式仍近似为固定延迟。这一差异印证了可插拔后端在“快速校验”与“高保真评估”之间的不同定位。

与硬件模拟的对比。在单核、调度一致、且统一采用 full 后端（BookSim2 + Ramulator2）以包含完整 DDR 行列时序与请求排队建模的配置下，本模拟器与 ZeBu 硬件模拟平台得到的总周期吻合良好，相对误差可控制在 $\pm 5.2\%$ 以内，验证了工作负载级调度与计算建模的合理性。偏差主要来自工作负载级抽象与指令级抽象在访存重叠刻画上的粒度差异，以及核内算子计算时间估算模型（见 4.1.3 节上界式估算，未显式建模片上供数折损、流水填充与分块对齐损失）的简化。

真实负载的瓶颈分解与热点可视化。在 ResNet34、ResNet50、批大小 1、单核与 full 后端下，通过将端到端时间分解为“纯计算时间”“因网络背压导致的停滞时间”与“访存相关等待时间”，可以对总周期进行可解释的归因。单核场景下不存在核间通信，因此网络背压时间为零，瓶颈主要体现在计算与访存的相对占比上。ResNet50 的访存占比高于 ResNet34，与模型加深一致。双核微基准上辅以各核阶段占比热力图，可从空间维度观察 producer/consumer 分工与等待主导阶段。在多核（ 2×2 ）场景下，同一分解口径进一步区分了计算、通信与访存三类瓶颈的贡献。递进式优化实验（DDR4 $\rightarrow$ HBM $\rightarrow$ SRAM 端口加宽）揭示了“DRAM 带宽 $\rightarrow$ SRAM 端口带宽”的瓶颈逐级迁移现象，最终将总周期从 238 803 降至 62 818（ $3.80 \times$ 加速），系统从严重存储受限趋向计算—存储平衡。设计空间探索表明，存储受限状态下提升 MAC 算力对总周期无可观测改善，核数超过负载并行度上限后性能退化（ 8×8 配置出现死锁），Batch 扩展效率仅约 24%。

6.2 展望

尽管本文完成了端到端模拟平台与实验框架，并在第五章通过微基准、硬件模拟对比与瓶颈及热点分析验证了其有效性，仍存在若干可进一步完善的方向，总结如下并略作展望。

核内建模的精细化。本平台定位于芯片早期设计阶段的架构与配置探索，核心计算时间按 4.1.3 节的上界式估算给出，未显式刻画流水填充、片上供数折损与分块对齐损失，对算子融合与核内数据流变化的敏感性有限。若进一步将调度器中间表示中的核内分块、循环顺序与数据驻留策略映射到更完整的核内模型，或接入 SCALE-Sim、MAESTRO、Timeloop 等数据流仿真器^[20-21,27]，可在保持系统级评估能力的前提下提升核内精度，使设计空间探索中的计算—访存权衡更贴近真实硬件行为，为编译器与架构的联合优化提供更细粒度的反馈。

更细粒度的瓶颈拆分。当前阶段分解将“访存相关等待”近似为“写回时间与加载时间之和再减去已单独计入的网络背压时间”。在加载阶段内部并未区分“等待片上网络投递”与“等待片外存储响应”。若在统计层面对这两类等待分别计数（例如按请求目标是其他核心还是片外存储来分类），可提高瓶颈归因的精度。这样能更好地剥离“通信墙”与“存储墙”的贡献，为互连与存储的协同优化提供更清晰的依据。

更丰富的网络与存储统计。当前输出侧重端到端周期与各核心各阶段占比。若进一步从周期级网络模拟器与存储模拟器中采集尾延迟分布、热点链路利用率、行缓冲命中率与存储体冲突率等指标，可支撑更深入的互连—存储协同优化与敏感性分析。这些指标与现有的状态追踪、每核瓶颈统计等形成互补，为架构师提供更全面的诊断信息。

更深入的设计空间探索。本文第五章已在部分维度上开展了多组配置的设计空间探索。在此基础上，若将模拟器纳入映射或参数搜索的闭环（例如与 Gemini 的模拟退火或贝叶斯优化相结合），通过并行运行多组配置与结果缓存降低探索成本，可将设计空间进一步扩展至 NoC 虚拟通道数、片上缓存容量、时钟比等更多维度，使编译—映射—模拟的一体化优化成为可能。

负载覆盖面的扩展。本文实验采用 ResNet34 与 ResNet50 作为真实负载，选择该系列的主要原因是当前可用的 ZeBu 硬件仿真踪迹仅覆盖 ResNet 系列的调度 IR，精度验证需要与“金标准”对等比较。模拟器的执行层以调度器 IR 为输入，不区分上游模型类型，因此建模机制与瓶颈分析框架对 Transformer 等其他负载同样适用。CNN 负载已在实验中展示了计算受限与存储受限的转换、DRAM 带宽墙、NoC 争用等多核系统级关键现象。然而，Transformer 类负载具有更大的 KV Cache、更强的访存—带宽压力以及自注意力算子带来的序列长度敏感性，其在多核映射下的瓶颈分布可能与 CNN 存在差

异。待上游调度器（如 Gemini）支持 Transformer 类负载的 IR 生成并提供对应 ZeBu 踪迹后，可在不修改模拟器代码的前提下直接运行与验证，进一步检验本文方法论在更广泛负载形态上的适用性。

更大规模多核扩展与互连。本文已在 2×2 至 4×4 核规模上开展了多核瓶颈分析与设计空间探索，初步揭示了 DRAM 争用与核数扩展上限。在更大规模多核（如 8×8 及以上）与更复杂的片上网络拓扑下，核间依赖与网络、存储的争用模式将更为复杂。进一步研究片上网络与存储带宽同映射策略的协同优化、死锁检测与防止机制，以及在带宽与能耗约束下的架构配置，是面向实际多核神经网络芯片产品的重要延伸方向。

6.3 工作总结

本文围绕“大规模多核神经网络芯片的端到端周期级评估与瓶颈分析”这一目标，从问题出发，完成了从中间表示驱动建模到全系统集成、从可插拔后端到可复现实验输出的系统性工作。以下从输入与任务抽象、执行层与全系统耦合、松耦合后端与实验可复现性、以及实验体系四个方面对全文工作加以概括。

输入与任务抽象。论文在第二、三章中建立了以调度器生成中间表示为输入的端到端多核神经网络芯片模拟流程。调度器（如 Gemini）输出的调度方案可直接作为模拟器的输入，无需人工改写或重新描述工作负载。模拟器通过统一的任务抽象，将每个核心上的执行内容表示为“核心级工作负载—显式数据依赖—输出流向”的可解析模型。解析得到的核心网格、每核工作负载列表、片外存储读写信令以及数据块标识与引用计数等信息，为后续执行层的依赖驱动推进与片上缓存的生命周期管理提供了完整的语义基础。这一设计使得编译器与调度器的输出能够与模拟器无缝衔接，有利于在架构探索中保持“同一套调度、同一套评估”的一致性。

执行层与全系统耦合。第四章在统一时间轴下将计算核心、片上网络与片外存储三者耦合在一起。每个计算核心采用“空闲—加载—计算—写回—完成”五态状态机，按数据依赖依次推进。在加载阶段向片上网络或片外存储发起读请求，在写回阶段向片外存储发起写请求，并响应其他核心对本地缓存的读请求。模拟引擎在每一全局周期内按固定顺序执行。先推进所有核心，再统一收集各核心发出的报文并注入网络与存储子系统，接着推进网络与存储一个周期，最后将本周期到达的响应投递给对应核心。这一顺序保证了“请求先于响应、响应在下一周期被消费”的因果关系。多核并发下的 NoC 争用、存储排队与生产者侧背压都能被显式刻画，且结果可完全复现。此外，通过全局周期与多速率推进机制，模拟器支持核心、网络与存储运行在不同时钟频率下的场景，便于分析“存储墙”“通信墙”对端到端性能的敏感性。

松耦合后端与实验可复现性。在架构实现上，本文采用松耦合的接口设计将片上网络与片外存储抽象为可替换的后端。用户可以选择“简单模式”（固定延迟、无排队）

进行快速校验,也可以选择“高保真模式”(接入周期级网络模拟器与周期级存储模拟器)以刻画拥塞与排队效应。两种模式共享同一套执行层逻辑,便于对照实验。在实验管理上,每次运行的结果都会写入单独的结果目录,不覆盖历史数据。输出包括与硬件模拟的对比表、端到端瓶颈分解汇总、各核心阶段统计以及状态随时间变化的追踪数据等,便于事后复现与核对。论文还提供了可一键完成硬件模拟对比与瓶颈分解等的自动化流程,使实验过程规范、结果可追溯。

实验体系。第五章围绕模拟器实验展开:先通过微基准与 ZeBu 对照完成精度与功能验证,再基于真实负载模拟结果做瓶颈与热点分析。微基准部分通过“双核点对点依赖”与“仅依赖片外存储”两类典型场景,验证了依赖满足的因果一致性、核间通信与访存的闭环正确性,以及 simple 与 full 两档后端在访存建模上的差异。在此基础上,将本模拟器的总周期与 Synopsys ZeBu 硬件模拟平台在相同调度、单核与 full 后端配置下的结果进行对比,相对误差控制在较小范围内,为工作负载级时序模型提供了参考依据。在 ResNet34、ResNet50 等真实 DNN 负载上,通过端到端阶段占比分解(计算时间、网络背压时间、访存等待时间)及各核阶段占比热力图等可视化,实现了对性能瓶颈与热点核的可解释归因。在多核(2×2 四核)场景下,以 ResNet50 为负载,通过 DDR4→HBM→SRAM 端口带宽的递进式优化揭示了瓶颈逐级迁移现象(总周期从 238 803 降至 62 818, $3.80 \times$ 加速),并沿 MAC 规模、核数与 Batch 三个维度开展了 12 组配置的设计空间探索,定量给出了存储受限下算力扩展无效、核数存在并行度上限及 Batch 扩展效率约 24% 等设计启示。

6.4 主要贡献

本文的主要贡献可概括为以下三点。

第一,中间表示驱动的端到端周期级模拟框架。设计并实现了以调度器生成中间表示为输入的多核神经网络芯片周期级模拟器,将计算、通信与存储纳入统一时间轴闭环耦合。模拟器既能输出端到端总周期,也能输出每个核心在加载、计算、写回以及因网络背压而停滞等阶段的周期统计,从而支撑可解释的瓶颈分析。模拟器可直接消费调度器输出的调度方案,无需手工改写工作负载描述。这便于与编译—映射流程衔接,为“改映射即改评估”的迭代提供了便利。

第二,松耦合的网络与存储接口与可插拔后端。提出了面向对象的网络接口与存储接口抽象,使得“简单模式”(固定延迟、无排队)与“高保真模式”(周期级网络模拟器与周期级存储模拟器)能够在同一套执行层逻辑下透明替换。研究者可以在不修改核心推进与依赖逻辑的前提下,对比“快速校验”与“高保真排队与拥塞”的结果,实现变量隔离与可复现实验。同时,模拟器还暴露了微架构数据流调度的接口,为进一步探索编译优化提供了基础能力。松耦合的接口式设计提供了统一的扩展点,便于分析研究

不同策略下的系统行为及其效果。

第三，面向验证与瓶颈归因的实验方法。建立了从微基准（依赖因果、simple 与 full 后端对照）、与 ZeBu 的总周期对比、到真实 AI 负载阶段分解及各核阶段占比热力图等可视化分析的实验路径。通过可复现的分解口径与统计输出，支撑了对计算—访存主导关系及双核场景下热点分工的定量结论。实验流程依托自动化脚本、独立结果目录以及仿真数据与图表输出，便于后续扩展与审阅。在此基础上，通过多核瓶颈递进式优化（DDR4→HBM→SRAM 端口加宽， $3.80\times$ 加速）与多维设计空间探索（MAC 规模、核数、Batch 三维度 12 组配置），验证了仿真平台在流片前快速参数定界中的实际价值。

致 谢

本论文的完成离不开许多师长、同学与家人的支持与帮助，在此谨致谢忱。

感谢姚期智教授在学业与科研环境上提供的指导与帮助。

感谢我的导师马恺声教授。从选题立意、方案设计到论文撰写的整个过程中，马老师都给予了悉心指导与耐心点拨。在周期级多核神经网络芯片模拟器这一课题上，马老师帮助我厘清了系统级建模与设计空间探索的研究思路，并在仿真框架设计、实验方案制定与结果分析等方面提出了诸多宝贵意见。马老师严谨的治学态度与对工程与理论结合的重视，使我受益匪浅。

感谢实验室的各位同门，在技术讨论、代码实现与论文修改过程中给予的帮助与建议。感谢在硕士阶段曾与我交流、为我答疑的各位老师与同学。

感谢我的家人，在我求学期间给予的理解、鼓励与支持，让我能够心无旁骛地完成学业与论文工作。

由于本人学识与精力有限，文中难免存在疏漏与不足之处，恳请各位老师与专家批评指正。

参考文献

- [1] VASWANI A, SHAZEER N, PARMAR N, et al. Attention Is All You Need[C/OL]//Advances in Neural Information Processing Systems (NeurIPS 2017): vol. 30. 2017: 5998-6008. DOI: 10.48550/arXiv.1706.03762.
- [2] ZHAO W X, ZHOU K, LI J, et al. A Survey of Large Language Models[J/OL]. Computing Research Repository (CoRR), 2023, abs/2303.18223. DOI: 10.48550/arXiv.2303.18223.
- [3] HENNESSY J L, PATTERSON D A. A New Golden Age for Computer Architecture[J/OL]. Communications of the ACM, 2019, 62(2): 48-60. DOI: 10.1145/3282307.
- [4] SASTRY G, HEIM L, BELFIELD H, et al. Computing Power and the Governance of Artificial Intelligence[J/OL]. Computing Research Repository (CoRR), 2024, abs/2402.08797. DOI: 10.48550/arXiv.2402.08797.
- [5] GHOLAMI A, YAO Z, KIM S, et al. AI and Memory Wall[J/OL]. IEEE Micro, 2024, 44(3): 33-39. DOI: 10.1109/MM.2024.3373763.
- [6] CAI J, WU Z, PENG S, et al. Gemini: Mapping and Architecture Co-exploration for Large-scale DNN Chiplet Accelerators[C/OL]//IEEE International Symposium on High-Performance Computer Architecture (HPCA 2024). 2024: 156-171. DOI: 10.1109/HPCA57654.2024.00022.
- [7] HAM H, YANG W, SHIN Y, et al. ONNXim: A Fast, Cycle-Level Multi-Core NPU Simulator[J/OL]. IEEE Computer Architecture Letters, 2024, 23(2): 219-222. DOI: 10.1109/LCA.2024.3484648.
- [8] LUO H, TUGRUL Y C, BOSTANCI F N, et al. Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator[J/OL]. IEEE Computer Architecture Letters, 2024. DOI: 10.1109/LCA.2023.333759.
- [9] WILLIAMS S, WATERMAN A, PATTERSON D. Roofline: An Insightful Visual Performance Model for Multicore Architectures[J/OL]. Communications of the ACM, 2009, 52(4): 65-76. DOI: 10.1145/1498765.1498785.
- [10] DAO T, FU D Y, ERMON S, et al. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness[C/OL]//Advances in Neural Information Processing Systems (NeurIPS 2022): vol. 35. 2022: 16344-16359. DOI: 10.52202/068431-1189.
- [11] DAO T. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning[C/OL]//International Conference on Learning Representations (ICLR 2024). 2024. DOI: 10.48550/arXiv.2307.08691.
- [12] KWON W, LI Z, ZHUANG S, et al. Efficient Memory Management for Large Language Model Serving with PagedAttention[C/OL]//ACM Symposium on Operating Systems Principles (SOSP 2023). 2023. DOI: 10.1145/3600006.3613165.
- [13] JIANG N, BALFOUR J, BECKER D U, et al. A Detailed and Flexible Cycle-Accurate Network-on-Chip Simulator[C/OL]//IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS 2013). 2013. DOI: 10.1109/ISPASS.2013.6557149.

- [14] FISCHER T, ROGENMOSER M, BENZ T, et al. FlooNoC: A 645-Gb/s/link 0.15-pJ/B/hop Open-Source NoC With Wide Physical Links and End-to-End AXI4 Parallel Multistream Support[J/OL]. IEEE Transactions on Very Large Scale Integration (VLSI) Systems, 2025, 33(4): 1094-1107. DOI: 10.1109/TVLSI.2025.3527225.
- [15] LATTNER C, AMINI M, BONDHUGULA U, et al. MLIR: Scaling Compiler Infrastructure for Domain Specific Computation[C/OL]//IEEE/ACM International Symposium on Code Generation and Optimization (CGO 2021). 2021: 2-14. DOI: 10.1109/CGO51591.2021.9370308.
- [16] Qualcomm Research. Hexagon-MLIR: An AI Compilation Stack for Qualcomm's Neural Processing Units (NPUs)[J/OL]. Computing Research Repository (CoRR), 2026, abs/2602.19762. DOI: 10.48550/arXiv.2602.19762.
- [17] CHEN Y H, EMER J, SZE V. Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks[C/OL]//ACM/IEEE International Symposium on Computer Architecture (ISCA 2016). 2016: 367-379. DOI: 10.1145/3007787.3001177.
- [18] JOUPPIN P, YOUNG C, PATIL N, et al. In-Datacenter Performance Analysis of a Tensor Processing Unit[C/OL]//ACM/IEEE International Symposium on Computer Architecture (ISCA 2017). 2017: 1-12. DOI: 10.1145/3079856.3080246.
- [19] VERONESI A, BERTOZZI D, KRSTIC M. Cross-Layer Hardware/Software Assessment of the Open-Source NVDLA Configurable Deep Learning Accelerator[C/OL]//IEEE International Symposium on Circuits and Systems (ISCAS 2021). 2021. DOI: 10.1109/ISCAS51556.2021.9401134.
- [20] SAMAJDAR A, ZHU Y, WHATMOUGH P, et al. A Systematic Methodology for Characterizing Scalability of DNN Accelerators using SCALE-Sim[C/OL]//IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS 2020). 2020: 58-68. DOI: 10.1109/ISPAS S48437.2020.00016.
- [21] KWON H, CHATARASI P, PELLAUER M, et al. Understanding Reuse, Performance, and Hardware Cost of DNN Dataflows: A Data-Centric Approach[C/OL]//IEEE/ACM International Symposium on Microarchitecture (MICRO 2019). 2019: 754-768. DOI: 10.1145/3352460.3358272.
- [22] GAO M, PU J, YANG X, et al. Tetris: Scalable and Efficient Neural Network Acceleration with 3D Memory[C/OL]//ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2017). 2017: 751-764. DOI: 10.1145/3037697.3037702.
- [23] GHOLAMI A, KIM S, DONG Z, et al. A Survey of Quantization Methods for Efficient Neural Network Inference[G/OL]//Low-Power Computer Vision. 2022. DOI: 10.1201/9781003162810-13.
- [24] LAI C, ZHANG W. gem5-NVDLA: A Simulation Framework for Compiling, Scheduling, and Architecture Evaluation on AI System-on-Chips[J/OL]. ACM Transactions on Design Automation of Electronic Systems (ACM TODAES), 2024, 29(5). DOI: 10.1145/3661997.
- [25] CAI J, WEI Y, WU Z, et al. Inter-layer Scheduling Space Definition and Exploration for Tiled Accelerators[C/OL]//ACM/IEEE International Symposium on Computer Architecture (ISCA 2023). 2023. DOI: 10.1145/3579371.3589048.

-
- [26] CHEN T, MOREAU T, JIANG Z, et al. TVM: An Automated End-to-End Optimizing Compiler for Deep Learning[C/OL]//USENIX Symposium on Operating Systems Design and Implementation (OSDI 2018). 2018: 578-594. DOI: 10.48550/arXiv.1802.04799.
- [27] PARASHAR A, RAINA P, SHAO Y S, et al. Timeloop: A Systematic Approach to DNN Accelerator Evaluation[C/OL]//IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS 2019). 2019: 304-315. DOI: 10.1109/ISPASS.2019.00042.
- [28] WU Y N, DENG R, HE L, et al. Accelergy: An Architecture-Level Energy Estimation Methodology for Accelerator Designs[C/OL]//IEEE/ACM International Conference on Computer-Aided Design (ICCAD 2019). 2019: 1-8. DOI: 10.1109/ICCAD45719.2019.8942143.
- [29] HUANG Q, KALAI AH A, KANG M, et al. CoSA: Scheduling DNN Accelerators via Constrained Optimization[C/OL]//ACM/IEEE International Symposium on Computer Architecture (ISCA 2021). 2021: 106-119. DOI: 10.1109/ISCA52012.2021.00018.
- [30] YANG X, GAO M, LIU Q, et al. Interstellar: Using Halide's Scheduling Language to Analyze DNN Accelerators[C]//ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS 2020). 2020: 369-383.
- [31] WEI Y, CAI J, GAO M, et al. Buffer Prospector: Discovering and Exploiting Untapped Buffer Resources in Many-Core DNN Accelerators[C/OL]//ACM/IEEE Design Automation Conference (DAC 2025). 2025. DOI: 10.1109/DAC63849.2025.11132440.
- [32] SHAO Y S, REITER B, WEI G Y, et al. Aladdin: A Pre-RTL, Power-Performance Accelerator Simulator Enabling Large Design Space Exploration of Customized Architectures[C/OL]//ACM/IEEE International Symposium on Computer Architecture (ISCA 2014). 2014: 97-108. DOI: 10.1109/ISCA.2014.6853196.
- [33] WU B, DAI X, ZHANG P, et al. FBNet: Hardware-Aware Efficient ConvNet Design via Differentiable Neural Architecture Search[C/OL]//IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR 2019). 2019: 10734-10742. DOI: 10.1109/CVPR.2019.01099.
- [34] CAI J, WANG X, GAO M, et al. SoMa: Identifying, Exploring, and Understanding the DRAM Communication Scheduling Space for DNN Accelerators[C/OL]//IEEE International Symposium on High-Performance Computer Architecture (HPCA 2025). 2025. DOI: 10.1109/HPCA61900.2025.00048.
- [35] SZE V, CHEN Y H, YANG T J, et al. Efficient Processing of Deep Neural Networks: A Tutorial and Survey[J/OL]. Proceedings of the IEEE, 2017, 105(12): 2295-2329. DOI: 10.1109/JPROC.2017.2761740.
- [36] LIAO H, TU J, XIA J, et al. DaVinci: A Scalable Architecture for Neural Network Computing[C/OL]//IEEE Hot Chips Symposium (Hot Chips 31, 2019). 2019: 1-44. DOI: 10.1109/HOTCHIPS.2019.8875654.
- [37] LIAO H, TU J, XIA J, et al. Ascend: a Scalable and Unified Architecture for Ubiquitous Deep Neural Network Computing – Industry Track Paper[C/OL]//IEEE International Symposium on High-Performance Computer Architecture (HPCA 2021). 2021: 789-801. DOI: 10.1109/HPCA51647.2021.00071.

附录 A Gemini-Compiler-IR、硬件配置与模拟器输出示例

本章给出论文中涉及的三种附录材料：Gemini 编译器 Scheduler IR 的片段示例、四核芯片的配置文件示例，以及模拟器的控制台输出与 trace 文件格式示例。

A.1 Gemini-Compiler-IR 示例

模拟器接受的输入为 Gemini 编译器调度后生成的 Scheduler IR（JSON 格式）。下面摘录单核、batch=1 的 ResNet50 调度结果中的一部分，用于说明 IR 的大致结构。

A.1.1 顶层结构

IR 的顶层包含：键 "-1" 表示与 DRAM 相关的数据（in 为写入 DRAM 的数据块，out 为从 DRAM 读出的数据块）。键 "0", "1", ... 表示各核心上的 workload 列表。buffersize、top_batch_cut、xlen、ylen 为全局参数。

```
{
  "-1" : { "in" : [ ... ], "out" : [ ... ] },
  "0" : [ ... ],
  "buffersize" : 8388608,
  "top_batch_cut" : 1,
  "xlen" : 1,
  "ylen" : 1
}
```

A.1.2 DRAM 数据块示例 ("-1")

"in" 中每个元素描述一块写入 DRAM 的数据（多为某层的 ofmap）。"out" 中每个元素描述一块从 DRAM 读出的数据（权重、ifmap 等）。例如 out 中的一个权重块：

```
{
  "destination" : [
    { "core_id" : 0, "layer_name" : "Conv_0",
      "type" : "core", "workload_id" : 1 }
  ],
  "lower" : [ 0, 0, 0, 0 ],
  "related_ifmap" : [],
  "size" : 27136,
}
```

```

"transfer_id" : 0,
"type" : "weight",
"upper" : [ 0, 63, 2, 48 ]
}

```

A.1.3 核心 workload 示例 (键 "0")

每个 workload 对应一个计算 tile，包含层名、类型、ifmap/ofmap、tile_info、时间估计等。下面为第一个 workload 的简化示例 (layer_type 为 vp，表示向量运算)：

```

{
  "layer_name" : "src_quantize_1_1",
  "layer_type" : "vp",
  "workload_id" : 0,
  "ifmap" : [
    { "align" : 4, "bitwidth" : 16,
      "lower" : [ 0, 0, 0, 0 ], "upper" : [ 0, 2, 223, 223 ],
      "size" : 401408, "transfer_id" : [ 54 ] }
  ],
  "ofmap" : [
    { "destination" : [ { "core_id" : 0, "type" : "core",
                          "workload_id" : 1 } ],
      "lower" : [ 0, 0, 0, 0 ], "upper" : [ 0, 2, 223, 223 ],
      "size" : 401408, "transfer_id" : 55 }
  ],
  "tile_info" : {
    "tile_num_0" : {
      "ifmap_lower" : [ 0, 0, 0, 0 ],
      "ifmap_upper" : [ 0, 2, 223, 223 ],
      "ofmap_lower" : [ 0, 0, 0, 0 ],
      "ofmap_upper" : [ 0, 2, 223, 223 ],
      "single_tile_time_pred" : 50190.0
    }
  },
  "time" : 50190,
  "workload" : [ [ 0, 0, 0, 0 ], [ 0, 2, 223, 223 ] ]
}

```

A.2 四核芯片配置文件示例

本节给出一个 2×2（共 4 核）的芯片配置示例，用于说明模拟器所描述的硬件参数。配置文件为 JSON，主要字段含义如下。

A.2.1 配置内容

- 核阵列：num_cores_x=2, num_cores_y=2，即 4 个核心。
- 数据位宽：element_size_bits=8（INT8）。
- 单核：mac_units=256, vector_units=64, clock_freq_mhz=1000, use_analytical_time=false（由模拟器按 4.1.3 节的上界式估算，根据张量形状与核心计算资源数量推算计算周期）。
- SRAM：size_kb=512，读写带宽 64 bytes/cycle。
- 网络接口：max_outstanding_reqs=16, injection_queue_size=8, ejection_queue_size=8。
- NoC：backend="booksim2"，接入 BookSim2 周期级网络模拟器。flit_size_bytes=16, num_vcs=8（虚拟通道数），vc_buf_size=8（每虚拟通道缓冲深度）。
- DRAM：backend="ramulator2"，接入 Ramulator2 周期级存储模拟器。standard="DDR4", timing="DDR4_2400R", num_channels=4, num_ranks=2, scheduler="FRFCFS"。
- 拓扑：dram_controllers=[] 表示 DRAM 直连各核（不经 NoC）。

完整 JSON 如下（与 configs/full_config.json 一致）：

```
{
  "num_cores_x": 2,
  "num_cores_y": 2,
  "element_size_bits": 8,
  "core": {
    "mac_units": 256,
    "vector_units": 64,
    "clock_freq_mhz": 1000,
    "use_analytical_time": false
  },
  "sram": {
    "size_kb": 512,
    "read_bandwidth_bytes_per_cycle": 64,
    "write_bandwidth_bytes_per_cycle": 64
  }
}
```

```
  },
  "ni": {
    "max_outstanding_reqs": 16,
    "injection_queue_size": 8,
    "ejection_queue_size": 8
  },
  "noc": {
    "backend": "booksim2",
    "flit_size_bytes": 16,
    "hop_latency_cycles": 1,
    "router_latency_cycles": 1,
    "injection_queue_depth": 16,
    "num_vcs": 8,
    "vc_buf_size": 8,
    "routing_delay": 1,
    "vc_alloc_delay": 1,
    "sw_alloc_delay": 1,
    "st_final_delay": 1
  },
  "dram": {
    "backend": "ramulator2",
    "num_channels": 4,
    "standard": "DDR4",
    "org": "DDR4_8Gb_x8",
    "timing": "DDR4_2400R",
    "num_ranks": 2,
    "scheduler": "FRFCFS",
    "addr_mapper": "RoBaRaCoCh",
    "frontend_clock_ratio": 4
  },
  "topology": {
    "dram_controllers": [],
    "dram_routing_policy": "nearest"
  }
}
```

A.3 模拟器输出与 Trace

模拟器运行时会向标准输出打印运行过程与统计信息。若启用 `-t (trace)` 选项，还会在指定目录下生成 `trace` 文件。

A.3.1 控制台输出示例

运行命令示例：

```
./npu_sim -c config.json -i model_schedule.json -t trace
```

运行过程中会周期性打印进度，例如：

```
[Simulator] Starting simulation...
[Simulator] Cycle 100000, cores done: 0/4, workloads completed: 12
[Simulator] Cycle 200000, cores done: 1/4, workloads completed: 28
...
[Simulator] Simulation complete at cycle 523456
```

模拟结束后会打印统计信息 (`print_stats`)，例如：

```
===== Simulation Statistics =====
Total cycles:          523456
NoC packets:           1245678
DRAM reads:            89012
DRAM writes:           45230
DRAM sub-requests:     234567
DRAM addr allocated:   128.50 MB

--- Per-Core Statistics ---
Core  Idle   Loading  Compute  Writeback  StallNoC  Wloads  PeakSRAM(KB)
  0    12000  45000    380000   12000      12000     22      412.5
  1    10000  42000    390000   11000      10000     22      398.2
  2    11000  44000    385000   11500      11000     22      405.0
  3    11500  43000    388000   11200      10500     22      400.1
=====
```

若启用了 `trace`，还会提示 `trace` 文件路径，例如：

```
[Trace] State transitions: trace/state_trace.csv (12345 events)
[Trace] Workload summary: trace/workload_summary.csv
[Trace] Files written to trace/
```

A.3.2 Trace 文件格式

1) state_trace.csv

记录各核心状态随周期的变化，每行一次状态转移。列含义如下：

- cycle: 发生转移的周期；
- core_id: 核心编号；
- old_state: 前一状态（如 IDLE、LOADING、COMPUTING、WRITEBACK、DONE）；
- new_state: 新状态；
- workload_idx: 当前 workload 索引；
- layer_name: 层名称；
- detail: 附加说明（如依赖的数据源）。

表头与示例行：

```
cycle,core_id,old_state,new_state,workload_idx,layer_name,detail
0,0,IDLE,LOADING,0,Conv_0,"waiting_for=[...]"
12500,0,LOADING,COMPUTING,0,Conv_0,""
51200,0,COMPUTING,WRITEBACK,0,Conv_0,""
```

2) workload_summary.csv

按核心与 workload 汇总各阶段的起止周期与耗时。列包括：core_id、workload_idx、layer_name、op_type、start_cycle、loading_done_cycle、compute_done_cycle、end_cycle、loading_cycles、compute_cycles、writeback_cycles、data_sources。

表头与示例行：

```
core_id,workload_idx,layer_name,op_type,start_cycle,
loading_done_cycle,compute_done_cycle,end_cycle,
loading_cycles,compute_cycles,writeback_cycles,data_sources
0,0,Conv_0,pe,0,12500,51200,53000,12500,38700,1800,
"transfer_id=[0,6]"
0,1,Conv_3,pe,53000,65500,98500,100200,12500,33000,1700,
"transfer_id=[1,7]"
```

以上示例与格式与当前模拟器实现一致，便于复现与对照。

A.4 工作量说明与项目代码结构

A.4.1 工作量说明

本课题工作量主要包括：(1) 周期级多核神经网络芯片模拟器的设计与实现，包括核心状态机、SRAM 与依赖建模、NoC/DRAM 抽象接口及 BookSim2/Ramulator2 的集成；(2) Gemini 编译器 Scheduler IR 的解析与驱动逻辑，将调度结果转化为可执行的 workload 序列与数据依赖；(3) 实验脚本与可视化工具链，涵盖 ZeBu trace 解析、仿真对比、瓶颈分解与论文图表生成；(4) 配置体系与实验设计，包括多种芯片配置、微基准 IR 与真实负载实验流程。除模拟器主体外，脚本与辅助工具保证实验可复现、结果可追溯。下表给出自写代码（含 C++ 源码、头文件、Python 脚本与 Shell 脚本）的规模统计，总规模约一万二千行。

附录 A-表 1 项目自写代码规模统计（约）

模块	说明	代码量（行）
核心模拟器（src/ + include/npu_sim/）	状态机、SRAM、IR 解析、引擎与主程序	约 9200
实验与可视化脚本（scripts/）	ZeBu 解析、对比、瓶颈分解、甘特图与图表	约 2100
构建与入口脚本	build.sh、run.sh 及 CMake 配置	约 200
配置与实验数据	芯片 JSON、微基准 IR、实验运行目录说明	约 400
合计	自写代码（不含第三方与子模块）	约 11900

A.4.2 项目文件结构描述

以下为项目自写部分（不含 third_party/、ext/ 等第三方）的目录与主要文件结构。

```
npu-simulator/
|-- CMakeLists.txt
|-- build.sh
|-- run.sh
|-- include/npu_sim/
|   |-- types.h
|   |-- config.h
|   |-- task.h
|   |-- packet.h
|   |-- sram.h
|   |-- core.h
|   |-- ir_parser.h
|   |-- gemini_parser.h
```

```
|  |-- network_interface.h
|  |-- memory_interface.h
|  |-- booksim2_noc.h
|  |-- ramulator2_dram.h
|  |-- address_allocator.h
|  |-- topology.h
|  \-- simulator.h
|-- src/
|  |-- main.cpp
|  |-- simulator.cpp
|  |-- core.cpp
|  |-- gemini_parser.cpp
|  |-- sram.cpp
|  |-- address_allocator.cpp
|  |-- booksim2_noc.cpp
|  \-- ramulator2_dram.cpp
|-- scripts/
|  |-- parse_zebu_trace.py
|  |-- compare_zebu_simulator.py
|  |-- batch_compare_zebu.py
|  |-- run_zebu_and_bottleneck_experiments.py
|  |-- plot_gantt.py
|  |-- plot_gantt_mb1.py
|  |-- plot_c4_concurrency.py
|  |-- plot_c5_breakdown.py
|  |-- gemini_run.py
|  \-- csv_to_excel.py
|-- configs/
|  |-- default_config.json
|  |-- full_config.json
|  |-- gemini_run_example.json
|  |-- booksim2_mesh.cfg
|  |-- ramulator2_ddr4.yaml
|  \-- chips/
|      |-- quad_core_example.json
|      |-- ascend_910.json
|      \-- ... (其他芯片配置)
\-- experiments/
    |-- simple_single_core_config.json
    |-- mb1_p2p_ir.json
```

```
|-- mb2_dram_only_ir.json
|-- runs/          (各次运行输出: trace、CSV 等)
\-- README.md
```

A.4.3 项目代码结构及模块功能

项目根目录下主要目录与文件如下, 第三方依赖 (third_party/、ext/) 未列入。

根目录。 CMakeLists.txt: 顶层 CMake 配置, 定义可执行目标与依赖。build.sh: 一键配置与编译。run.sh: 示例运行命令与常用参数组合。

include/npusim/ (头文件)。 定义模拟器对外接口与内部数据结构。types.h: 全局类型、枚举与常量。config.h: 从 JSON 加载的配置结构体。task.h: Workload、BufferRequirement、OperatorType 等任务表示。packet.h: NoC 报文格式。sram.h: 片上 SRAM 容量与占用模型。core.h: 核心状态机 (IDLE/LOADING/COMPUTING/WRITE-BACK/DONE) 与周期推进。ir_parser.h、gemini_parser.h: IR 解析接口与 Gemini Scheduler IR 实现。network_interface.h、memory_interface.h: NoC 与 DRAM 抽象接口。booksim2_noc.h、ramulator2_dram.h: BookSim2 与 Ramulator2 的封装声明。address_allocator.h、topology.h: DRAM 地址分配与拓扑/路由。simulator.h: 仿真引擎, 多核推进、trace 导出与统计。

src/ (实现)。 main.cpp: 命令行解析、配置加载、仿真运行与统计输出。simulator.cpp: 全局周期步进、核间报文收集与投递、DRAM 响应注入、trace 导出。core.cpp: 单核状态转移、依赖检查、SRAM 占用、计算周期与 NoC/DRAM 请求发起。gemini_parser.cpp: 解析 Scheduler IR JSON, 生成各核 workload 队列与 DRAM 张量描述。sram.cpp: SRAM 占用与释放。address_allocator.cpp: DRAM 地址分配。booksim2_noc.cpp: 将报文映射为 BookSim2 的 Flit 流并驱动仿真。ramulator2_dram.cpp: 将读写请求转换为 Ramulator2 子请求并回调响应。

scripts/ (脚本)。 parse_zebu_trace.py: 解析 ZeBu 输出的 trace, 汇总周期与统计。compare_zebu_simulator.py: 单次 ZeBu 与模拟器结果对比。batch_compare_zebu.py: 批量对比并输出 CSV/LaTeX。run_zebu_and_bottleneck_experiments.py: 一键运行 ZeBu 对比与瓶颈分解实验。plot_gantt.py、plot_gantt_mb1.py: 根据 state_trace.csv 绘制甘特图。plot_c4_concurrency.py、plot_c5_breakdown.py: 第四章并发与第五章瓶颈分解图表。gemini_run.py: 调用 Gemini 生成 IR 并可选启动模拟器。csv_to_excel.py: 结果 CSV 转 Excel 等辅助功能。

configs/。 chips/ 下为各芯片 JSON 配置 (如 quad_core_example.json、default_config 等)。default_config.json 等为全后端/简单后端示例。experiments/

下为微基准 IR、运行目录及说明，用于复现论文实验。

以上模块共同支撑“IR 驱动—周期推进—NoC/DRAM 闭环—统计与 trace”的端到端仿真流程，并与脚本配合完成 ZeBu 精度对比与设计空间探索实验。

A.5 第五章图 5-1 数据来源

```
cycle,core_id,old_state,new_state,workload_idx,layer_name,detail
0,0,IDLE,LOADING,0,producer,"waiting_for=[t0(DRAM,4096B),t1(DRAM,1024B)]"
0,1,IDLE,LOADING,0,consumer,"waiting_for=[t3(DRAM,8192B),t2(CO,16384B)]"
207,0,LOADING,COMPUTING,0,producer,"compute_cycles=500"
707,0,COMPUTING,IDLE,1,,"workload_done"
708,0,IDLE,DONE,1,,"all_workloads_finished"
2000,1,LOADING,COMPUTING,0,consumer,"compute_cycles=300"
2300,1,COMPUTING,WRITEBACK,0,consumer,"writeback_to_dram"
2456,1,WRITEBACK,IDLE,1,,"writeback_complete"
2457,1,IDLE,DONE,1,,"all_workloads_finished"
```

攻读学位期间取得的研究成果

- [1] Inter-layer Scheduling Space Definition and Exploration for Tiled Accelerators, Proceedings of the 50th International Symposium on Computer Architecture (ISCA)
- [2] Gemini: Mapping and Architecture Co-exploration for Large-scale DNN Chiplet Accelerators, 2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)
- [3] SoMa: Identifying, Exploring, and Understanding the DRAM Communication Scheduling Space for DNN Accelerators, Proceedings of the IEEE International Symposium on High-Performance Computer Architecture (HPCA)
- [4] Buffer Prospector: Discovering and Exploiting Untapped Buffer Resources in Many-Core DNN Accelerators, ACM/IEEE Design Automation Conference (DAC)

答辩委员会会议决议

论文提出了

论文取得的主要创新性成果包括:

- [illegible]

论文工作表明作者在 ×××× 具有 ×××× 知识，具有 ×××× 能力，论文 ××××××××××，答辩 ××××××××××××××。

答辩委员会表决，（×票/一致）同意通过论文答辩，并建议授予×××（姓名）×××（门类）学博士/硕士学位。

常规评阅人名单

本学位论文共接受 7 位专家评阅，其中常规评阅人 5 名，名单如下：

王 XX	教授	西安交通大学
李 XX	教授	XXXXXXX 大学
田 XX	教授	XXXXXXX 大学

学位论文独创性声明（1）

本人声明：所呈交的学位论文系在导师指导下本人独立完成的研究成果。文中依法引用他人的成果，均已做出明确标注或得到许可。论文内容未包含法律意义上已属于他人的任何形式的研究成果，也不包含本人已用于其他学位申请的论文或成果。

本人如违反上述声明，愿意承担以下责任和后果：

1. 交回学校授予的学位证书；
2. 学校可在相关媒体上对作者本人的行为进行通报；
3. 本人按照学校规定的方式，对因不当取得学位给学校造成的名誉损害，进行公开道歉。
4. 本人负责因论文成果不实产生的法律纠纷。

论文作者（签名）：日期：年 月 日

学位论文独创性声明（2）

本人声明：研究生 所提交的本篇学位论文已经本人审阅，确系在本人指导下由该生独立完成的研究成果。

本人如违反上述声明，愿意承担以下责任和后果：

1. 学校可在相关媒体上对本人的失察行为进行通报；
2. 本人按照学校规定的方式，对因失察给学校造成的名誉损害，进行公开道歉。
3. 本人接受学校按照有关规定做出的任何处理。

指导教师（签名）：日期：年 月 日

学位论文知识产权权属声明

我们声明，我们提交的学位论文及相关的职务作品，知识产权归属学校。学校享有以任何方式发表、复制、公开阅览、借阅以及申请专利等权利。学位论文作者离校后，或学位论文导师因故离校后，发表或使用学位论文或与该论文直接相关的学术论文或成果时，署名单位仍然为西安交通大学。

论文作者（签名）：日期：年 月 日

指导教师（签名）：日期：年 月 日

（本声明的版权归西安交通大学所有，未经许可，任何单位及任何个人不得擅自使用）